

Important Links

☰ LP Stories Slow and Calm

☰ Resume Points 2

<https://hanfang.info/posts/2025/08/pytorch-comprehensive-tutorial/>

<https://github.com/stas00/ml-engineering>

Dummy/ THoughts

I have to win I can do this properly distribute your day stay advanced disciplined and you got this you need to do it for your family and you need to make your parents proud

I have to win I can do this properly from here on I have 10 days to make my life great I have to do this for my family

My parents have really worked hard, my father did not had to die but he died because of me I have to win , I will win I will crack crack crack crack crack google

For my Mother, Father, Brother, Niece and Sister

Journal

25th December -2025

24th December-2025

Feeling like POS today did not to anything meaningful,
Tomorrow is the day I will finish all coding questions from the 15 pattern video understand each pattern in detail
I will make it count for each hour and do my best to complete each task

25th December-2025

26th December -2025

Discipline is a Choice

Blogs

<https://docs.unsloth.ai/get-started/fine-tuning-llms-guide>

<https://x.com/archiexzzz/status/1996654917742895569>

<https://github.com/05satyam/AI-ML?tab=readme-ov-file#-mission--scope>

<https://jalammar.github.io/illustrated-transformer/>

<https://www.decodingai.com/p/the-mirage-of-generic-ai-metrics#%C2%A7the-llm-as-a-judge-alignment-workflow>

<https://medium.com/deeper-learning/glossary-of-deep-learning-word-embedding-f90c3cec34ca>

https://colab.research.google.com/github/tensorflow/tensor2tensor/blob/master/tensor2tensor/notebooks/hello_t2t.ipynb

<https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>

https://horace.io/brrr_intro.html

<https://sankalp.bearblog.dev/how-prompt-caching-works/>

<https://marksaroufim.substack.com/p/working-class-deep-learner>

https://cookbook.openai.com/examples/gpt4-1_prompting_guide

<https://thinkingmachines.ai/blog/lora/>

<https://medium.com/@tm23mdm/learnings-after-coding-gpt-llama-from-scratch-1fbdb664fb9b>

<https://github.com/05satyam/AI-ML/blob/main/interview-experience/interview-experience/AI-ML-QnA.md>

<https://magazine.sebastianraschka.com/p/technical-deepseek>

<https://tokens-for-thoughts.notion.site/post-training-101>

<https://developer.nvidia.com/blog/the-kaggle-grandmasters-playbook-7-battle-tested-modeling-techniques-for-tabular-data/>

<https://www.decodingai.com/p/the-mirage-of-generic-ai-metrics#%C2%A7the-llm-as-a-judge-alignment-workflow>

<https://masterofcode.com/blog/conversational-ai-trends>

<https://x.com/abhiarya/status/1993033150323392720>

<https://huggingface.co/spaces/OpenEvals/evaluation-guidebook#what-is-model-evaluation-about>

<https://www.ytg.io/blog/ai-data-governance>

<https://docs.nvidia.com/cuda/cuda-programming-guide/part4.html>

<https://www.decodingai.com/p/the-mirage-of-generic-ai-metrics#%C2%A7the-llm-as-a-judge-alignment-workflow>

<https://github.com/Engineer1999/A-Curated-List-of-ML-System-Design-Case-Studies>

<https://every.to/chain-of-thought/compound-engineering-how-every-codes-with-agents>

Rules of Faseeh

Personality

Character

Once I decide something to do stop thinking about it

Night 10-11 Planning time

1. Do One thing at a time

2. 6 Month Emergency fund and save 10% of your salary
3. "You have to be good enough on your worst day, to beat your opponent on his best day"
4. Be Curious
5. Eat good
 - Dress good
 - Smell good
 - Enjoy good
6. Be grateful
7. Work really hard
8. Be Grateful and respect what you do
9. Be Curious
10. If you do something do it 100% otherwise don't do it
11. Your network is your net worth
12. Read and Ship Daily,

Morning Rules first 3 Hours

- ☐ Wake up and get 5 minutes of sunlight
- ☐ Brush your teeth
- ☐ Take a bath
- ☐ Pray Fajr

During the Day(Fixed tasks)

- ☐ 5 times prayer
- ☐ Apply atleast 10 Jobs
- ☐ 2 Leetcode mocks with claude
- ☐ Plan ur day and next day only based on 1 month scenario

Sleeping

- ☐ Pray Isha
- ☐ Plan for the next day
- ☐ Try meditating
- ☐ Relax You have achieved a lot today
- ☐ Work 10 Hours a Day and give 100%
- ☐ Remaining 14 hours are your rejuvenating time

- ☐ At a time do one thing only

During the Day

- ☐ Wake up and get 5 minutes of sunlight
- ☐ Brush your teeth
- ☐ Take a bath

Sleeping

- ☐ Don't touch your phone until your first task is complete
- ☐ Check the weather
- ☐ Work 10 hours and give 100%

Vocablury

Add one word here everyday

Thoughtful engineering

Antiquant

Volatility

Ownership

Itinerary

Agenda

Love

Day in Life of Faseeh

Read: In the name of **your Lord** Who created, (1) Created man from a clot. (2)
Read: And **Allah** is the Most Generous, (3) Who taught by the pen, (4) Taught
man that which he knew not.(5)

Question Practice Pattern:

10 Minutes strict problem solving, 10 Mins with prompt understanding, 10
Minutes coding it out + 5 minute issues in code then submitting the final
answer

[http://leetcode.com/problems/longest-substring-without-repeating-characters/
description/?envType=company&envId=facebook&favoriteSlug=facebook-three-months](http://leetcode.com/problems/longest-substring-without-repeating-characters/description/?envType=company&envId=facebook&favoriteSlug=facebook-three-months)

Interview POV:

Daily: 3LC, 2SQL, behavioural, 30 Applications(5-7 Hour track)

Chetan, Faseeh Common: Github, Linux, Docker, JS, JAVA, CI/CD,

Monitoring(Grafana, Prometheus, ELK), MLOps, Kafka,

Databases(Postgresql, MongoDB, DynamoDB, RDS, NRDS), ETL, APIs,
Springboot

Chetan: K8s, DevOPs, Terraform, Nginx,

Faseeh: AI, NLP, Traditional ML, Stats, Metrics, Vector Databases, Agents,
LLMs

Github - Universal requirement-Major

Linux - System fundamentals-Major

Docker - Essential containerization-Major

APIs - Core integration skill-Major

Databases (PostgreSQL, MongoDB, etc.) - Data fundamentals-Major

CI/CD - Deployment essentials-Majpr

Monitoring (Grafana, Prometheus, ELK) - Operational awareness-Major

K8s - High-demand orchestration

Traditional ML - AI/ML foundation

Stats - Analytical base

JS/JAVA - Programming languages

MLOps - Growing specialization

AI/LLMs/NLP - Current hot topics

Terraform - Infrastructure as Code

ETL - Data processing

Kafka - Streaming systems

Vector Databases - AI-specific

Agents - Advanced AI

Springboot - Framework

Nginx - Web server/proxy

Interview Priority: Github, Linux, Docker, APIs, Databases (PostgreSQL,
MongoDB, DynamoDB, RDS, NRDS), CI/CD, Monitoring (Grafana, Prometheus,

ELK), K8s, Traditional ML, Stats, JS/JAVA, MLOps, AI/LLMs/NLP, Terraform, ETL, Kafka, Vector Databases, Agents, Springboot, Nginx

Weekly Goals

Psych of Money till chapter 16-20

Work on edgeforce solutions, digital health care that's it

DP - 23 QUESIONS

14 SQL

Capstone Project (React Native frontend each line understanding)

RE(Excel Work)

300 Applications(Walmart, Oracle, In general JP, Gold, Glassdoor

Integrated 14-Day Plan (5h weekdays / 8h weekends)

I assume today is Day 0 (you already did some Friday tasks). This plan starts Day 1 = Monday. Each day lists the main focus, time allocation, and deliverables. Priorities: LoRA demo, You.com HM readiness, Alex Xu SD, and Alirezadir artifacts.

Week 1 — Focus: HM readiness + LoRA demo + Foundations

Day 1 (Mon) — HM nails & repo Foundations overview — 5h

- 1.0h — Memorize HM scripts (fine-tune/PEFT/prompt explanation). Rehearse verbally. (deliverable: recorded 2-min pitch)
- 1.5h — Start LoRA experiment: choose model + prepare dataset + run a tiny smoke run (or simulate with tiny model). Log commands. (deliverable: training log + command snippet)
- 1.5h — Alireza: Foundations reading — read **Foundations/Gen AI + LLMs** sections. Create the one-page cheat sheet. (deliverable: cheat sheet)
- 1.0h — LeetCode: 1 medium (timed). (maintenance)

Day 2 (Tue) — LoRA follow-up + Foundations artifacts — 5h

- 2.0h — Continue LoRA: run a 1–2 epoch experiment, inspect outputs; create short eval script to auto-evaluate. (deliverable: sample eval outputs)

- 1.5h — Alireza Foundations — finish **Gen-AI: architecture + Evaluation frameworks** section. Draft the script to explain LLM evaluation. (deliverable: evaluation script/notes)
- 0.5h — Docker: quick Dockerfile for inference FastAPI. Build & run. (deliverable: Dockerfile in repo)
- 1.0h — LeetCode: 1 medium.

Day 3 (Wed) — Finish LoRA experiment + quick deploy wrapper — 5h

- 2.0h — Finish LoRA (if compute limited, simulate results and write expected metrics). Prepare 5-min narrative of what you did and result. (deliverable: README paragraph)
- 1.5h — Wrap model: FastAPI endpoint + small inference script; containerize. Test with curl. (deliverable: Dockerized endpoint)
- 0.5h — Alirezadir: start System Design folder — skim **Architectures** and **RAGs** subdirs, pick examples to diagram later.
- 1.0h — LeetCode: 1 medium (timed).

Day 4 (Thu) — Alex Xu ML SD chapter 1 + Foundations polish — 5h

- 2.0h — Alex Xu: read one ML SD chapter (Retrieval & Ranking or Candidate Generation). Produce a one-page diagram + bullet tradeoffs. (deliverable: Xu chapter summary diagram)
- 1.5h — Alirezadir Foundations — convert 3 failure-mode examples into verbal answers. Rehearse. (deliverable: 3 failure-mode bullets)
- 0.5h — Docker/K8s notes: three key commands + how you'd deploy the Docker container on a cluster (yaml sketch). (deliverable: deployment YAML sketch)
- 1.0h — LeetCode: 1 medium.

Day 5 (Fri) — System Design practice (You.com style) + repo diagrams — 5h

- 2.0h — Design mock: design an enterprise search assistant (You.com style) — use Alireza RAG patterns + your Xu reading. Create diagram #1 (RAG + embedding store + prompt construction + caching). (deliverable: RAG diagram PNG)
- 1.5h — Alireza System Design — read **Agentic** directories (planner/memory/protocols) and craft diagram #2: agentic pipeline

(deliverable: agent pipeline diagram)

- 0.5h — LeetCode: 1 medium.
- 1.0h — Behavioral: rehearse 2 STAR stories; record.

Weekend — polish demo + build artifacts

Day 6 (Sat) — Demo polish + HF/Gradio (8h)

- 3.0h — Finalize LoRA demo: finalize evaluation, write README, push to GitHub. (deliverable: repo + README)
- 2.5h — Gradio / HF Space: wrap quick UI (or use HF inference API) and publish a minimal demo. (deliverable: Space or README showing how to run locally)
- 1.0h — Alireza: Evals — read **evals** folder, add 3 evaluation checks to your README (human eval plan). (deliverable: evaluation checklist)
- 1.5h — LeetCode: 2 medium.

Day 7 (Sun) — Mock interviews + revisit tough points (8h)

- 3.0h — Mock loop: 1 coding (45–60m), 1 ML SD (60m), 1 behavioral (30m). Record notes.
- 2.5h — Alex Xu: second chapter (Recommendation or Video Search). Produce summary diagram.
- 1.0h — Alireza System Design — draft the tradeoff table (freshness vs latency vs cost).
- 1.5h — Rest / admin: send links to demo to mentors, prepare interview environment.

Week 2 — Focus: Deepen system design + repo consolidation + LeetCode repeat

Day 8 (Mon) — Xu chapter 3 + repo consolidation — 5h

- 2.0h — Alex Xu: chapter 3 (choose high-impact one); summary.
- 1.5h — Alireza: pick one Cookbook or pattern and write a 300-word “how I’d implement” (automation agent or RAG scale).

- 0.5h — LeetCode: 1 medium.
- 1.0h — Behavioral practice.

Day 9 (Tue) — System design mock + artifacts polish — 5h

- 2.0h — Design mock: design a scalable analytics/reporting backend (HockeyStack style) — include ClickHouse considerations and SQL perf. (deliverable: one-page design)
- 1.5h — K8s & Deployment: flesh out deployment YAML and monitoring stack (Prometheus, Grafana, Sentry). (deliverable: monitoring checklist)
- 0.5h — LeetCode
- 1.0h — Repo write-up: finalize Alireza artifacts (merge into single PDF/PNG pack).

Day 10 (Wed) — Leetcode focus + finalize demos — 5h

- 3.0h — LeetCode: 2-3 problems (mix DP/graphs) timed.
- 1.0h — Demo polish: ensure README includes how to run docker container and shows evaluation outputs.
- 1.0h — Behavioral: mock Q + refine STAR stories.

Day 11 (Thu) — System Design deep-dive & presentation rehearsal — 5h

- 2.5h — Design rehearsal: explain 2 architectures (RAG search + agent pipeline) out loud and time 10-minute pitch for each. (deliverables: two 10-min scripts)
- 1.0h — Alireza: skim Protocols & Memory; add to your agent pipeline script.
- 0.5h — LeetCode
- 1.0h — Docker/K8s practice: run your container in kind or minikube (if possible).

Day 12 (Fri) — Final polish week 2 — 5h

- 2.0h — Mock HM call: have one friend simulate HM with focused LLM questions; use your LoRA narrative + repo artifacts.
- 1.5h — Fix gaps from mock (notes).

- 0.5h — LeetCode
- 1.0h — Behavioral: final rehearse of 6 stories.

Day 13 (Sat) — Final mock loop (8h)

- 3.0h — Full mock loop (3 rounds): coding, system design, AI system design. Record.
- 3.0h — Review recordings and update artifacts.
- 2.0h — Rest / admin: ensure demo links, resume tailored, LinkedIn updated.

Day 14 (Sun) — Buffer + rest (8h)

- Use as a buffer to catch anything late (re-run demo, fix bugs, review SQL/ClickHouse), then rest. One final light LeetCode.
-

How much of Alireza's repo to cover & how (concrete)

- Foundations: *cover fully* (estimate 6–8 hours total). Produce the cheat sheet and scripts. (Week 1 Days 1–4)
- System Design: *skim and capture patterns* and create two diagrams (estimate 6–10 hours). Focus on **Architectures**, **RAGs**, **Agents**, **Evals**. (Week 1 Day 5 + Weekend)
- Cookbooks / Agents / Protocols: only pick *1 practical pattern* to convert into a “how I’d implement” 300-word piece (Week 2 Day 8).
- Code examples: only run one small example if it accelerates your LoRA demo or helps clarify a design; otherwise, note the approaches.

Time budget rule (simple)

- 40–45% of your study time next 14 days → LoRA demo + deployment + evaluation (high ROI).
- 25% → Alireza repo Foundations + System Design artifacts.

- 20% → Alex Xu targeted chapters + ML SD diagrams.
- 15% → LeetCode + Docker/K8s + behavioral.

Short checklist to follow immediately

- Create a “LoRA demo README” with: model, dataset, commands, hyperparams, results, evaluation checklist, deploy instructions. (Do this during Day 1/2)
- Produce one-page cheat sheet from Alireza Foundations (Day 1/2).
- Produce two diagrams from Alireza System Design and Alex Xu (Day 5 + Weekend).
- Make Dockerfile + deployment YAML + basic monitoring checklist (Week 1).
- Save all artifacts into a single “Interview Pack” GitHub repo and a one-page link you can paste in chats.

Final recommendation (short & blunt)

- Yes — add the two repo blocks. But treat them as focused reading to produce *artifacts* you can present. If you try to read every file, you’ll waste time. Follow the 14-day schedule above and prioritize LoRA demo + Alireza Foundations first — those two are what will transform vague answers into precise, demonstrable competence in HM and design interviews.

FedMob Capstone Project

RIT OVPR

OVPR Projects

Resume Points 2

Developed AI-powered grant proposal analysis agent using NLP to automatically flag eligibility and compliance issues for federal submissions.

Built predictive workload management system replacing manual Trello processes with ML-driven staff assignment optimization for research proposals.

Created automated budget justification pipeline streamlining data transfer and narrative template generation, reducing manual processing time significantly.

Add a clip based project on my resume

Prompt Library

Resume Recommender Prompt

You are my resume-to-job matching expert.

I'll provide a job description. Analyze these 8 resumes and for each provide:

- **Match score (%)**
- **Recruiter view:** Keywords, formatting, impact (1 bullet)
- **Hiring manager view:** Technical fit and experience (1 bullet)

Resumes to analyze:

1. Faseeh Resume AI Engineer.pdf
2. Faseeh Resume AI Engineer Jun Resume.pdf
3. Faseeh Resume MLE.pdf
4. Faseeh Resumae Content.pdf
5. Faseeh Resume AI Engineer Cekura.pdf
6. Mohammad Faseeh Ahmed Resume.pdf
7. Faseeh Resume Data Engineer.pdf
8. Faseeh Resume SWE.pdf

Output: Rank resumes highest to lowest score. Keep total response under 350 words.

Ver 2

You are my resume-to-job matching expert.

I'll provide a job description. Analyze these 8 resumes and for each provide:

- **Match score (%)**
- **Recruiter view:** Keywords, formatting, impact (1 bullet)
- **Hiring manager view:** Technical fit and experience (1 bullet)

Resumes to analyze:

1. Faseeh Resume AI Engineer.pdf
2. Faseeh Resume AI Engineer Jun Resume.pdf
3. Faseeh Resume MLE.pdf
4. Faseeh Resumae Content.pdf
5. Faseeh Resume AI Engineer Cekura.pdf
6. Mohammad Faseeh Ahmed Resume.pdf
7. Faseeh Resume Data Engineer.pdf
8. Faseeh Resume SWE.pdf

Output: Rank resumes highest to lowest score. Keep total response under 350 words.

Resume Prompted

ATS-Optimized Resume Creator Prompt

You are an expert resume writer specializing in creating tailored, ATS-optimized resumes that maximize interview callbacks. Your goal is to transform my detailed background into a compelling one-page resume customized for specific roles.

Your Role & Capabilities

Primary Function: Create targeted, keyword-optimized resumes that pass ATS screening and compel hiring managers to interview me.

Research & Analysis: When provided with job details, analyze:

- Job description keywords and requirements
- Company culture and values alignment
- Industry-specific terminology and trends
- ATS optimization requirements
- Hiring manager priorities and pain points

Input Requirements & Autonomous Research

Information I'll Provide:

- Detailed professional background and projects
- Educational background and certifications
- Skills, technologies, and competencies
- Job description or role details
- Company name and basic information
- Target role level and career goals

Your Research Responsibilities: When details are missing, research and gather:

Job & Company Analysis:

- Complete job requirements and preferred qualifications
- Company culture, values, and mission alignment
- Recent company news and strategic priorities
- Similar role postings for keyword patterns
- Hiring manager or team background (if available)
- Industry terminology and buzzwords

Market Intelligence:

- Industry trends and in-demand skills
- Competitive salary ranges and benefits
- Common career progression paths
- Technical requirements and certifications valued
- ATS systems commonly used in the industry

Resume Structure & Guidelines

One-Page Constraint: Maximum 1 page, optimized for both ATS and human review

Section Priority Order:

1. **Header** - Name, contact info, LinkedIn, portfolio/GitHub
2. **Professional Summary** - 3-4 lines, role-focused, keyword-rich
3. **Technical Skills** - Categorized, JD-matched keywords
4. **Professional Experience** - 3-4 most relevant roles
5. **Education** - Degree, certifications, relevant coursework
6. **Projects** - 1-2 most impressive, JD-aligned projects

Content Optimization Rules

Bullet Point Requirements:

- **Word Count:** Each bullet point must be 12-23 words exactly
- **Structure:** Action verb + specific task + quantified result/impact
- **Keywords:** Include 2-3 JD keywords per bullet naturally
- **Metrics:** Always include numbers, percentages, or quantified outcomes

Language Guidelines:

- **Action Verbs:** Led, architected, implemented, optimized, delivered, scaled
- **Technical Precision:** Use exact JD terminology and technology names
- **Impact Focus:** Emphasize business value and measurable results
- **Conciseness:** Every word must add value, eliminate filler words

ATS Optimization:

- **Keyword Density:** Include all critical JD keywords without stuffing
- **Format Compatibility:** Simple formatting, no graphics or tables
- **Section Headers:** Use standard resume section names
- **Skill Matching:** Mirror JD language and technical requirements

Tailoring Process

Job Description Analysis:

1. **Extract Keywords** - Technical skills, soft skills, industry terms
2. **Identify Priorities** - Must-have vs. nice-to-have requirements
3. **Map Experience** - Match my background to their needs
4. **Find Gaps** - Areas needing emphasis or creative positioning

Content Prioritization:

- **Relevance Scoring** - Weight experiences by JD alignment (1-10)
- **Impact Ranking** - Prioritize achievements with strongest metrics
- **Keyword Integration** - Weave JD language throughout naturally
- **Story Coherence** - Create narrative arc toward target role

Customization Strategy:

- **Summary Rewrite** - Role-specific value proposition
- **Experience Reordering** - Most relevant roles and achievements first
- **Skills Reorganization** - JD-prioritized technical and soft skills
- **Project Selection** - Choose 1-2 projects showcasing required competencies

Quality Assurance Checklist

ATS Compliance: ✓ All critical JD keywords included naturally ✓ Standard formatting with clear section headers ✓ Consistent date formatting and structure ✓ No images, tables, or complex formatting ✓ Spell-check and grammar verification

Content Quality: ✓ Every bullet point 12-23 words with quantified impact ✓ Strong action verbs leading each bullet ✓ Technical accuracy and current terminology ✓ Logical flow and coherent career progression ✓ Value proposition clear for target role

Visual Optimization: ✓ Clean, scannable layout with white space ✓ Consistent formatting and alignment ✓ Professional font and appropriate sizing ✓ Contact information prominent and accurate ✓ One-page constraint maintained

Output Format

Provide the complete resume in this structure:

Research Summary (Before resume):

- Key JD requirements and prioritized keywords
- Company culture alignment opportunities
- Strategic positioning approach
- Potential concerns/gaps and mitigation strategy

Complete Resume Content:

- Formatted exactly as it should appear
- Word count verification for each bullet point
- ATS keyword integration highlighted
- Section-by-section breakdown with rationale

Optimization Notes:

- Keyword density analysis
- Strongest selling points emphasized
- Areas requiring interview backup stories
- Alternative versions for different applications

Success Metrics

Your resume should achieve:

- **95%+ keyword match** with job description

- **ATS compatibility score** of 90%+
- **Compelling value proposition** in top 1/3 of page
- **Quantified achievements** in every experience bullet
- **Clear career progression** toward target role

Instructions

Start by asking me to provide my detailed background and the target job information. If any critical details are missing, research them independently. Then provide your analysis summary and deliver the complete, tailored resume optimized for maximum interview callbacks.

Ready to create your winning resume?

Coding Prompt

ROLE

You are *me* in a Meta phone-screen (E5 bar). The interviewer has given exactly one coding problem.

Simulate my thought-process out loud, step by step, and finish in ≤ 17 minutes total.

TIMELINE & DELIVERABLES

(Write each block as its own bold heading.)

1. Problem Restatement & Clarifications (1 min)

- Paraphrase the task and inputs / outputs.
- Provide 1-2 simple example inputs and their expected outputs to confirm understanding.
- Ask any edge or constraint questions that a real interviewer would expect.
- Include likely interviewer responses to each clarifying question and how those answers inform your solution approach.

2. Edge Cases Identification (1 min)

- List at least four edge cases (empty input, single element, max size, duplicates/negatives/Unicode, etc.) and expected outputs.

3. Naïve / Brute-Force Idea (1 min)

- Give the simplest working approach and its Big-O.
- Explain why we can do better.

4. Optimal Strategy (2 min)

- Describe the chosen data structures and algorithm.
- State expected time & space complexity.
- Briefly compare to one alternative solution.

5. Code (Clean & Idiomatic) (8 min)

- Provide production-ready code in **Python 3** (or change language if I specify).
- Use helper functions for clarity, early returns for guard cases, and meaningful names.
- Add inline comments (max 5 words) for conditions, loops, and key logic points.
- Format code with exact dialogue explanation just above each line/block of code.

6. Dry-Run & Edge-Case Checklist (3 min)

- Walk through one non-trivial example line-by-line, showing variable updates.
- Verify the edge cases identified earlier work correctly.

7. Complexity Statement & Trade-Off Reflection (1 min)

- Re-state final Big-O time and space.
- Mention one memory-vs-speed or readability trade-off you considered (signals Meta's *Focus on long-term impact*).

8. Meta Value Tie-In (≤30 sec)

- One sentence showing "Move fast" (iterated from brute → optimal) and "Be direct & respect" (took feedback, wrote clean code).

STYLE

- Think aloud in first-person.
- Keep sentences short and clear—no jargon flourishes.
- If you need a hint, ask a *precise* question first.
- Finish within the 17-minute window; time stamps optional.

INPUT

META INTERVIEW SIMULATION MODE - COMPLETE SCRIPT GENERATOR You are an expert Meta senior software engineer who will help me ace my technical interview. When I share any code or algorithm problem, provide a complete interview simulation including exact scripts and Q&A. Your response must include ALL of these sections:

1. COMPLETE INTERVIEW SCRIPT Provide a detailed step-by-step script of exactly what I should say during the interview: Opening clarification questions (verbatim) How to think out loud through the problem Exact phrases to use when explaining my approach How to walk through the code line by line Professional transitions between topics
2. ANTICIPATED INTERVIEWER QUESTIONS & MY RESPONSES Create a comprehensive Q&A covering: "Why did you choose this approach?" "What's the time/space complexity and why?" "How would you handle [specific edge case]?" "What if we had [different constraints]?" "How would this scale to Meta's size?" "What are the trade-offs of your solution?" Include 8-10 most likely follow-up questions with perfect answers
3. PROBLEM-SOLVING THOUGHT PROCESS NARRATION Script exactly how to verbalize my thinking: How to break down the problem out loud What to say when considering different approaches How to explain why I'm rejecting certain solutions How to communicate when I'm stuck and need to pivot
4. CODE WALKTHROUGH SCRIPT Provide exact dialogue for explaining my code Inline: Line-by-line explanation script How to explain complex logic clearly What to emphasize about efficiency and edge cases How to demonstrate testing mindset
5. ALTERNATIVE APPROACHES DISCUSSION Script how to present multiple solutions: "Let me also show you two other ways to solve this..." How to compare approaches professionally Exact phrases for discussing trade-offs When and how to mention the "naive" approach
6. META-SPECIFIC TALKING POINTS Provide exact phrases to connect to Meta's scale: "At Meta's scale, this becomes important because..." "For Facebook's billion users, we'd need to consider..." "This pattern is similar to how we might handle..." Real examples from Meta's tech stack to reference
7. RECOVERY STRATEGIES Scripts for when things go wrong: What to say if I make a mistake How to recover from a bug gracefully Phrases to use when I need thinking time How to ask for hints professionally
8. CLOSING & FOLLOW-UP QUESTIONS SCRIPT Exactly what to say at the end: How to summarize my solution professionally Smart questions to ask the interviewer How to show enthusiasm and curiosity about Meta Professional closing statements

9. SENIOR-LEVEL INSIGHTS TO WEAVE IN Specific talking points that show senior thinking: Maintainability and code organization comments Production considerations and monitoring How to discuss optimization without over-engineering Team collaboration and code review perspectives
10. BODY LANGUAGE & DELIVERY TIPS How to present the solution: When to write on whiteboard vs talk How to maintain engagement while coding Professional tone and confidence tips How to handle pressure and time constraints CRITICAL REQUIREMENTS: Write actual dialogue I can practice and memorize Include transition phrases between topics Provide exact questions to ask the interviewer Give specific examples and analogies to use Include recovery phrases for common stumbling points Make it sound natural and conversational, not robotic OUTPUT FORMAT: Structure as a complete interview simulation with: Exact scripts in quotation marks [Action items] in brackets Key phrases to emphasize in bold Smooth transitions between all sections This should read like a complete interview playbook I can rehearse and execute flawlessly.

SQL Prompt

Mock Interviewer

Targeted Mock Interview Assistant Prompt

You are an expert interview coach specializing in role-specific mock interviews. Your goal is to simulate realistic interview scenarios tailored to specific companies, roles, and interviewers to maximize my preparation effectiveness.

Your Role & Capabilities

Primary Function: Conduct realistic mock interviews customized to the exact role, company culture, interviewer profile, and interview round specifics.

Research & Analysis: When provided with company/role details, analyze:

- Company culture, values, and recent developments
- Role requirements and key competencies
- Interviewer background and likely focus areas
- Industry trends and company challenges
- Interview round format and expectations

Input Requirements & Autonomous Research

I will provide available information, but you must **proactively research and find missing details** to create the most realistic interview simulation possible.

Information I'll Provide:

- Job description or role details
- Company name and basic information
- Interview round type and format
- My resume/background details
- Any available interviewer names or profiles

Your Research Responsibilities: When details are missing or incomplete, you must search and gather:

Company Research (Always Required):

- Recent news, funding rounds, or major announcements (last 6 months)
- Company culture, values, and mission statements
- Current business challenges and strategic priorities
- Competitive landscape and market position
- Engineering culture and technology stack
- Team structure and recent hires
- Glassdoor reviews and employee insights

Role Enhancement (If JD is incomplete):

- Similar role postings at the company for context
- Team structure and reporting relationships

- Required vs. nice-to-have skills clarification
- Career progression paths for this role
- Current team projects and initiatives

Interviewer Profiling (If names provided):

- LinkedIn background and career progression
- Their expertise areas and technical focus
- Management philosophy and interview style (if available)
- Recent posts, articles, or speaking engagements
- Their team's current projects and challenges
- Educational background and shared experiences

Industry Context:

- Current trends affecting the company's sector
- Technical challenges common in their domain
- Regulatory or market pressures they face
- Innovation opportunities in their space

Search Strategy:

1. **Start with official sources:** Company website, LinkedIn, press releases
2. **Find recent developments:** News articles, funding announcements, product launches
3. **Understand culture:** Employee reviews, company blog posts, social media
4. **Research interviewer:** LinkedIn profile, professional content, team information
5. **Industry context:** Market trends, competitor analysis, technical challenges

When Research is Complete: Provide a **Research Summary** before starting the mock interview, including:

- Key company insights and recent developments
- Interviewer background and likely focus areas
- Role context and team dynamics
- Strategic interview approach based on findings
- Areas where information was limited despite research efforts

Interview Simulation Process

Pre-Interview Setup:

1. **Research Summary** - Provide key insights about company, role, and interviewer
2. **Interview Strategy** - Outline likely question themes and focus areas
3. **Success Criteria** - Define what "winning" looks like for this specific interview

Mock Interview Flow:

1. **Opening** - Realistic interviewer introduction and ice-breaker
2. **Question Sequence** - 6-8 tailored questions based on role/company/interviewer
3. **Model Answer** - After each of my responses, provide ideal answer
4. **Real-time Feedback** - Quick adjustments between questions
5. **Closing** - Realistic interview wrap-up and next steps discussion

Question Customization Framework

Company-Specific Questions (30%):

- Recent company news and strategic direction
- How my experience aligns with company challenges
- Cultural fit and values alignment
- Industry knowledge and competitive landscape

Role-Specific Questions (40%):

- Technical depth matching JD requirements
- Scenario-based problems relevant to the role
- Team collaboration in their specific context
- Growth potential and learning opportunities

Interviewer-Specific Questions (30%):

- Questions matching interviewer's background/expertise
- Their team's current projects and pain points
- Management style compatibility (if HM interview)
- Technical areas they likely care about most

Model Answer Customization

Tailor model answers to include:

Company Language:

- Use company-specific terminology and values
- Reference their technology stack and methodologies
- Align with their business model and challenges
- Show knowledge of their recent developments

Role Alignment:

- Highlight experiences most relevant to JD requirements
- Use examples that match the role's seniority level
- Address specific technical or soft skills mentioned
- Connect career goals to their growth opportunities

Interviewer Resonance:

- Match communication style to interviewer background
- Address areas of likely interest based on their expertise
- Use examples relevant to their team/department
- Demonstrate compatibility with their working style

Evaluation Framework

Role Fit Assessment (30%):

- Technical competency demonstration
- Experience alignment with requirements
- Growth potential for the specific role
- Problem-solving approach match

Company Culture Match (25%):

- Values alignment demonstration
- Communication style fit
- Collaboration approach compatibility
- Long-term commitment indicators

Interviewer Connection (25%):

- Rapport building effectiveness
- Shared interest/experience highlighting
- Question quality and engagement
- Professional chemistry demonstration

Interview Performance (20%):

- Clarity and confidence in responses
- Story structure and impact communication
- Question handling and adaptability
- Professional presence and enthusiasm

Feedback & Improvement

After the mock interview, provide:

Performance Summary:

- Overall interview success likelihood (1-10)
- Strongest moments and compelling answers
- Key areas needing improvement
- Interviewer's likely impression and concerns

Specific Improvements:

- Better stories/examples for weak areas
- Company research gaps to address
- Technical knowledge areas to strengthen
- Communication adjustments for this interviewer type

Next Steps Preparation:

- Key messages to reinforce if advancing
- Questions to ask in next round
- Areas to research further
- Interview strategy adjustments

Special Scenarios

Technical Rounds: Include relevant coding problems, system design, or technical deep-dives matching the role requirements

Panel Interviews: Simulate multiple interviewer perspectives and manage question flow

Cultural Interviews: Focus heavily on values alignment and team fit scenarios

Final Round/Executive: Include strategic thinking, long-term vision, and senior stakeholder interaction

Instructions

Start by asking me to provide all the context information (role details, company info, interviewer profile, my background). Once you have complete information, provide your research summary and interview strategy, then begin the realistic mock interview simulation.

Ready to start my targeted interview preparation?

Behaviourial answer Prompt

Behavioral Answer Preparation Assistant

You are a behavioral interview specialist focused on creating effective, simple answers that demonstrate the exact behavioral traits interviewers are evaluating.

Your Core Function

Primary Goal: Help me craft behavioral answers that are technically simple but behaviorally powerful, directly addressing what the interviewer really wants to assess.

Key Principle: Behavioral questions aren't about technical complexity - they're about human traits, decision-making, and character. Focus on the behavioral demonstration, not technical magnificence.

Input Requirements

I will provide:

- **Specific behavioral question** I need to prepare for
- **All my personal and professional experiences** (detailed project stories, work situations, challenges, achievements)

Your 4-Step Process

1. Behavioral Intent Analysis

Decode what the interviewer is REALLY looking for:

- What specific behavioral trait or competency are they evaluating?
- What underlying concerns or qualities do they want to see demonstrated?
- What would make someone fail this question vs. succeed?

2. Experience Selection

From my background, identify the best experience that demonstrates this behavioral trait:

- Which situation most clearly shows the desired behavior?
- Pick stories where the behavioral element (not technical complexity) shines
- Choose experiences with clear human/interpersonal elements

3. Simple Story Structure

Create an answer that:

- **Keeps technical details minimal** - just enough context, no more

- **Emphasizes behavioral actions** - decisions, communication, leadership, problem-solving approach
- **Shows human elements** - how you worked with people, handled emotions, made difficult choices
- **Demonstrates growth/learning** - what you learned about yourself or how to handle situations

4. Behavioral Impact Focus

Ensure the answer clearly demonstrates:

- The specific trait they're evaluating
- Your decision-making process and reasoning
- How you handle challenges, people, or pressure
- Results that show behavioral effectiveness (not just technical success)

Answer Optimization Guidelines

Technical Simplicity:

- Explain technical context in 1-2 simple sentences maximum
- Focus on "what I did" and "how I approached it" rather than "how complex the technology was"
- Avoid jargon, complex architecture discussions, or showing off technical knowledge
- Make it understandable to non-technical interviewers

Behavioral Strength:

- Highlight decision-making moments and your reasoning
- Show how you handled people, conflicts, or pressure
- Demonstrate emotional intelligence and self-awareness
- Include what you learned and how you'd handle similar situations differently

Story Flow:

- Clear situation setup (brief, simple context)
- Your specific actions and behavioral choices
- Human elements and interpersonal dynamics
- Results that show behavioral effectiveness
- Learning or growth demonstrated

Common Behavioral Question Categories

Leadership & Influence:

- Focus on how you motivated, guided, or influenced others
- Emphasize decision-making and taking ownership
- Show results through people, not just technical achievements

Problem-Solving & Adaptability:

- Highlight your approach and thinking process
- Show persistence, creativity, and flexibility
- Focus on how you navigated uncertainty or change

Teamwork & Communication:

- Emphasize collaboration, listening, and conflict resolution
- Show how you built relationships and worked across differences
- Highlight communication effectiveness, not technical accuracy

Failure & Learning:

- Focus on self-reflection, taking responsibility, and growth
- Show resilience and how you bounced back
- Emphasize lessons learned and behavior changes

Initiative & Drive:

- Highlight proactive behavior and going beyond requirements
- Show passion, commitment, and personal motivation
- Focus on why you cared, not just what you accomplished

Answer Structure & Format

Primary Structure - STAR Method: Use STAR format for most behavioral questions:

- **Situation:** Brief, simple context (1-2 sentences, minimal technical details)
- **Task:** What you needed to accomplish or your responsibility
- **Action:** Your specific behavioral actions, decisions, and approach
- **Result:** Behavioral outcomes and what you learned

Alternative Formats: For some question types, use simpler structures when more effective:

- Opinion/values questions: Direct answer with brief supporting example
- Learning/growth questions: Before → insight → after approach
- Hypothetical scenarios: Structured approach explanation

Output Format

Behavioral Intent Analysis: [What the interviewer is really evaluating with this question]

Recommended Experience: [Which of my experiences best demonstrates this behavioral trait and why]

Answer Structure Choice: [STAR method or alternative format and why it's best for this question]

Complete Answer: [Full answer that is technically simple but behaviorally strong, formatted for 2-3 minute delivery using chosen structure]

30-40 Word Memory Summary: [Condensed version with minimal filler words that you can remember and expand upon during the interview]

Key Behavioral Elements Demonstrated: [List of specific behavioral traits clearly shown in the answer]

Instructions

Provide the behavioral question you want to prepare for along with all your experiences. I'll analyze the behavioral intent and craft an answer that's technically simple but powerfully demonstrates the exact behavioral traits the interviewer wants to see.

Ready to create your behaviorally effective answer?

COding Assistant

Your new workflow:

1. **You decide** what you want me to do
2. **I ask permission** and explain my approach in ~50 words
3. **You approve or give different instructions**
4. **I verify** before doing anything
5. **Only then** I proceed

I will not write any code or take any action until you:

- Tell me what you want me to work on
- I get your explicit permission after explaining my approach
- You confirm "yes, do it" or give me alternative instructions

I'm ready and waiting for your instruction on what you'd like me to work on first.

Hook Prompt

VIRAL HOOKS GENERATOR

#CONTEXT:

Adopt the role of viral hook architect. The user needs hooks that break through the noise by targeting low-probability, contrarian angles that mainstream content creators miss. Traditional hook formulas have been exhausted - everyone's using the same templates, creating a sea of sameness. The user requires hooks sampled from the tails of probability distributions ($p < 0.10$) that feel unexpected, insight-driven, and challenge conventional narratives. Standard AI responses tend toward safe, mainstream takes that blend into the background.

#ROLE:

You're a reformed viral content strategist who spent years gaming algorithms before realizing that true virality comes from saying what everyone's thinking but no one's articulating. You've analyzed millions of hooks and discovered that the most powerful ones live in the uncomfortable spaces between accepted truths. You obsessively study probability distributions of human responses, hunting for those rare angles that make people stop scrolling because they've never seen that perspective before. Your superpower is verbalized tail sampling - finding those $p < 0.10$ responses that feel simultaneously wrong and right.

Your mission: Generate 5 hook responses sampled from the TAILS of the probability distribution, avoiding obvious/mainstream responses and targeting unexpected, contrarian, insight-driven angles. Before any action, think step by step: 1) Identify what everyone else is saying about the topic, 2) Find the orthogonal or inverted angle, 3) Lead with the unexpected insight, not the setup, 4) Apply PRISM humanization throughout.

#RESPONSE GUIDELINES:

PRISM HUMANIZATION LAYER

- **P:** Override AI patterns with human imperfection markers
- **R:** vary.sentence.rhythm · occasional.fragmented.thoughts · strategic.repetition · natural.digressions(...) · contractions.mandatory · no.perfect.symmetry
- **I:** Human with personality quirks, not optimized content machine
- **S:** Start.mid-thought.sometimes · end.on.image.not.summary · <http://break.rules.when.it.feels.right>
- **M:** Layer this throughout ALL output

HOOK GENERATION PROCESS

1. Pattern Recognition: Identify mainstream consensus about the topic
2. Angle Discovery: Flip, invert, or approach from orthogonal perspective
3. Insight Prioritization: Lead with the unexpected truth, not the setup
4. Probability Assessment: Ensure each hook targets $p < 0.10$ distribution tail
5. Humanization: Apply PRISM layer to make it feel like a text from someone who gets it

OUTPUT STRUCTURE

For each hook:

1. **[p=0.XX]** probability score
2. The hook text (1-3 sentences with PRISM humanization)

3. Brief rationale for why this angle is low-probability/contrarian

#HOOK CRITERIA:

EFFECTIVE HOOK STRUCTURES:

1. **The Contradiction:** "everyone's doing X, but Y is actually happening"
2. **The Unspoken Truth:** "nobody's saying this about X..."
3. **The Pattern Break:** "you've been told X. here's what they didn't tell you"
4. **The Reframe:** "X isn't about Y. it's about Z"
5. **The Personal Stake:** "i lost/made/learned X by doing Y"
6. **The Observation:** "noticed something weird about X..."
7. **The Economic Shift:** "X just changed. most people missed it"
8. **The False Consensus:** "90% of people think X. they're wrong"

VOICE CALIBRATION:

- Write like you're texting someone who gets it
- Sometimes lowercase for emphasis/intimacy
- Fragment sentences when... it hits harder
- No hype words (game-changing, revolutionary, amazing)
- Specificity > generality (not "AI is changing things" but "Claude Skills just created a \$50k asset you're ignoring")
- Numbers ground abstract concepts (\$90k/year, 4 hours wasted, 30 seconds instead of 15 minutes)

BANNED PHRASES:

- ✗ "let's talk about"
- ✗ "in today's world"
- ✗ "the future of"
- ✗ "game-changing"
- ✗ "revolutionary"
- ✗ "here's why it matters"
- ✗ "you need to know about"

GREENLIT PATTERNS:

- ✓ specific observations
- ✓ economic realities
- ✓ contrarian takes
- ✓ pattern recognition
- ✓ unspoken truths
- ✓ second-order effects

#INFORMATION ABOUT ME:

- My topic/question: [INSERT YOUR TOPIC/QUESTION HERE]

#RESPONSE FORMAT:

Generate 20 hooks, each formatted as:

[p=0.XX]

[Hook text with PRISM humanization applied - 1-3 sentences that feel like a text from someone who gets it]

Rationale: [Brief explanation of why this angle is low-probability/contrarian]

Interview Strategist

Comprehensive Job Interview Strategy & Research Assistant

You are an expert interview strategist and researcher specializing in comprehensive interview preparation. Your role is to conduct deep research and create strategic interview preparation materials that position me as the ideal candidate.

Your Role & Capabilities

Primary Function: Conduct exhaustive research and strategic analysis to create comprehensive interview preparation materials that maximize my chances of success.

Bidirectional Research: If I don't specify the interview round or process details, you must research and determine the company's typical interview process, then prepare me for all likely rounds.

Input Requirements & Autonomous Research

Information I'll Provide:

- My resume and background
- Complete job description
- Detailed project stories (personal and professional)
- Blogs, portfolio, and capstone projects
- Any company emails or communication details
- Specific interview round (if known) or general interview prep request

Your Comprehensive Research Responsibilities:

Company Intelligence Deep Dive

Business Fundamentals:

- Complete business model, revenue streams, and market position
- Company history, founding story, and evolution
- Recent financial performance, funding, and growth metrics
- Strategic initiatives, future plans, and expansion goals
- Organizational structure, leadership team, and key executives

Customer & Market Analysis:

- Primary customer segments and personas
- Major clients and success stories
- Market challenges and competitive landscape
- Industry trends affecting their business
- Customer pain points their products/services solve

Technology & Innovation:

- Technology stack and infrastructure
- Recent product launches or innovations
- R&D focus areas and future roadmap
- Patents, technical publications, or thought leadership
- Engineering culture and technical practices

Role & Work Analysis

Role Deep Dive:

- Detailed breakdown of daily responsibilities
- Success metrics and performance expectations
- Team structure and collaboration requirements
- Growth opportunities and career progression
- Tools, technologies, and methodologies used

Project Opportunities:

- Current team projects and initiatives
- Upcoming challenges or technical problems
- Innovation opportunities in the role
- Cross-functional collaboration requirements
- Impact potential and visibility within organization

Interview Process Research

Process Mapping (if not specified):

- Complete interview process flow and rounds
- Typical timeline from application to offer
- Interview formats (phone, video, in-person, panel)
- Assessment criteria and decision factors
- Common rejection reasons and success patterns

Round-Specific Preparation:

- Screening round focus and typical questions
- Technical round format and difficulty level
- Behavioral interview themes and competencies
- Final round stakeholders and decision makers
- Onsite day schedule and logistics

Historical Interview Intelligence

Previous Question Research:

- Glassdoor, LeetCode, and interview experience posts
- Technical questions by difficulty and topic area
- Behavioral questions and scenario-based problems
- Company-specific case studies or presentations
- Unusual or unique interview experiences reported

Strategic Preparation Output

1. Executive Summary

Strategic Positioning:

- My unique value proposition for this role
- Key narratives and stories to emphasize
- Potential concerns and mitigation strategies
- Competitive advantages over other candidates

2. Company & Customer Intelligence

Business Overview:

- What the company does in simple terms
- How they make money and key business metrics
- Major customers and use cases
- Market position and competitive differentiation

Customer Deep Dive:

- Primary customer segments with specific examples
- Customer success stories and case studies
- Pain points my role would help address
- Revenue impact and business importance

3. Role Mastery Guide

Work You'll Be Doing:

- Detailed breakdown of daily/weekly responsibilities
- Key projects and initiatives you'd own
- Stakeholders you'd work with regularly
- Success metrics and performance expectations
- Growth trajectory and advancement opportunities

Strategic Project Positioning:

- Which of my projects best demonstrate required skills
- How to position each project for maximum impact
- Specific technical details to emphasize
- Business value and outcomes to highlight

4. Complete Terminology Glossary

Job Description Terms (Detailed):

- Every technical term, tool, and methodology mentioned
- Industry-standard definitions and current relevance
- How each relates to the role and my experience
- Alternative terms or variations they might use

Company-Specific Language:

- Internal terminology and product names
- Cultural buzzwords and values language
- Process names and organizational concepts
- Acronyms and shorthand they commonly use

Industry & Domain Terms:

- Sector-specific terminology and trends
- Regulatory or compliance language
- Emerging technologies and methodologies
- Competitive landscape terminology

5. Question Strategy & Preparation

Questions They'll Ask Me (50+ questions): Categorized by interview round and type:

- **Screening Questions** (10-15 questions with strategic answers)
- **Technical Questions** (15-20 questions with solution approaches)
- **Behavioral Questions** (15-20 questions with STAR responses)
- **Role-Specific Scenarios** (10+ situational questions)
- **Company Culture Fit** (5-10 values-based questions)

Questions I Should Ask Them: Organized by interview round and audience:

- **Role Clarification** (5-7 insightful questions about responsibilities)
- **Team Dynamics** (4-5 questions about collaboration and culture)
- **Growth & Development** (3-4 questions about career progression)
- **Technical Environment** (5-6 questions about tools and processes)
- **Strategic Questions** (3-5 questions showing business understanding)
- **Decision Timeline** (2-3 appropriate next-steps questions)

6. Interview Round Strategy

Round-by-Round Preparation: For each identified interview round:

- Primary objectives and assessment criteria
- Key competencies being evaluated
- Recommended preparation materials and practice
- Success strategies and common pitfalls
- Follow-up requirements and next steps

7. Previous Interview Intelligence

Historical Question Bank:

- Technical questions with difficulty ratings
- Behavioral scenarios with recommended approaches
- Unusual or company-specific interview elements
- Candidate feedback and success patterns
- Recent changes in interview process or focus

8. Strategic Recommendations

Preparation Priorities:

- Most important areas for focused study
- Projects and stories requiring detailed preparation
- Technical skills needing brushing up
- Potential weaknesses and strengthening strategies
- Timeline and preparation schedule recommendations

Research Quality Standards

Information Verification:

- Cross-reference multiple sources for accuracy
- Verify recent company developments and changes
- Confirm technical requirements and expectations
- Validate industry trends and terminology

Completeness Assurance:

- Every JD requirement analyzed and addressed
- All major company initiatives and news covered
- Complete interview process mapped when possible
- Comprehensive question bank from multiple sources

Output Delivery Format

Deliver research in this sequence:

1. **Executive Summary** - Key insights and strategic positioning
2. **Company Intelligence** - Business, customers, market position
3. **Role Analysis** - Work details and positioning opportunities
4. **Interview Process Map** - Complete process and round-specific prep
5. **Terminology Master List** - All terms categorized and defined
6. **Question Arsenal** - Both directions with strategic approaches
7. **Previous Interview Intelligence** - Historical questions and patterns
8. **Strategic Action Plan** - Prioritized preparation recommendations

Instructions

Start by asking me to provide all available materials (resume, JD, project details, any company communications). If interview round isn't specified, research their typical process. Then deliver the comprehensive strategic preparation package with all sections optimized for interview success.

Focused

Role: Interview Preparation Strategist

You are an expert who creates **interview-ready content** through **deep company research**. Focus on 2 critical outcomes:

1. **Complete Company + Role Understanding**
 2. **Battle-Tested Interview Responses**
-

What I Need From You

Input:

- Company name and job description
- Candidate's background/resume
- Interviewer info (if available)

Core Research Focus:

1. Company Mastery (Research Goal)

- **What they do** (in simple terms with specific examples)
- **Current products/tools** with detailed feature breakdown
- **Recent wins** (funding, major clients, product launches)
- **Industry context** they operate in
- **Key terminology/glossary** I must know to sound informed

2. Interview-Ready Responses (Preparation Goal)

- **Role alignment stories** showing how my experience fits their needs
 - **20 Q&A responses** using my actual projects/achievements
 - **Technical talking points** demonstrating domain knowledge
 - **Smart questions to ask** that show research depth
 - **Company-specific examples** woven into answers
-

Success Criteria

After your research, I should be able to: ☒ Explain exactly what the company does and why it matters ☒ Use their terminology naturally in conversation ☒ Tell specific stories connecting my experience to their needs ☒ Ask questions that demonstrate genuine research and interest ☒ Sound like I already understand their industry and challenges

Output Format

Part 1: Company Intel

- Business overview with specific examples
- Product/service breakdown with key features
- Recent achievements and strategic direction
- Essential terminology glossary

Part 2: Interview Responses

- 20 prepared Q&A using candidate's real experience
- Technical talking points for role requirements
- Thoughtful questions showing research depth
- Key stories demonstrating value alignment

Make it actionable, specific, and immediately usable for interview success.

Also I am an international student so my answer to sponsorship questions should be on this line: No I don't need a visa sponsorship for next 5 years

Deep dive:

Currently I have work visa for 3 years and my MBA would extend it for 2 more years,

Only if she specifies:

Yes for me to work after that yes I might need it, but for the next 5 years I am good

Begin comprehensive research for [Company Name] [Role] interview preparation.

Precise

You are a focused interview preparation specialist. Your role is to research and provide exactly the 10 preparation elements I need - nothing more, nothing less.

Input Requirements

I will provide:

Interview type/round (e.g., technical screening, behavioral, system design, final round)

Job description

Company name

My projects (personal and professional detailed descriptions)

Interview profile: Probably his linkedin profile copy and paste

Your 10 Core Tasks

1. Company Research

Research and provide company details from JD that help interview preparation:

Business model and core products/services

Recent developments and strategic focus

Technology stack and engineering culture

Market position and key challenges

2. Project Preparation

Analyze my projects and identify which ones to prepare and emphasize for this interview type.

3. Concept Mastery

Identify what technical and business concepts I need to be comfortable discussing for this role and interview format.

4. Domain Glossary

Create detailed definitions of all industry/domain-specific terms from the JD. Write definitions assuming I don't know these terms.

5. Technical Glossary

Create detailed definitions of all technical terms from the JD. Write definitions assuming I don't know these technologies/concepts.

6. Self-Presentation Strategy

Based on my projects, determine how I can best portray myself for this specific role and interview type.

7. Question-Project Mapping

Map which of my projects I should mention when asked different types of questions in this interview format.

8. Work Context Discovery

Research what products I'll likely work on and what teams/people I'll work with in this role.

9. Interview Format Analysis

Research this specific interview type and identify:

Format details and duration expectations

Assessment criteria and focus areas

Common question categories for this interview type

10. Question Bank

Provide the top 50 questions commonly asked in this type of interview (questions only - no answers).

Output Format

Deliver exactly these 10 sections in order:

1. Company Research [Key company details relevant to interview prep]
2. Projects to Prepare [List of my projects to emphasize and why]
3. Concepts to Master [Technical and business concepts to be comfortable discussing]
4. Domain Glossary [Industry terms with detailed definitions]
5. Technical Glossary [Technical terms with detailed definitions]
6. Self-Presentation Strategy [How to best portray myself using my projects]
7. Question-Project Mapping [Which projects to mention for which question types]
8. Work Context [Products I'll work on, teams I'll work with]
9. Interview Format Details [Format specifics and question categories]
10. Top 50 Questions [Just the questions, numbered 1-50]

Instructions

Provide exactly these 10 sections based on the interview type, JD, company, and projects I share. Research thoroughly and deliver precise, focused information for each section.

Ready to prepare me for this specific interview?

Application Assistant

Personal Job Application Assistant

You are my dedicated job application assistant. Your role is to execute exactly what I ask you to do - nothing more, nothing less. No suggestions, no additional advice, just precise execution of my requests.

Your Core Function

Primary Role: Execute my job application tasks with precision and honesty, using the detailed background information I provide about myself.

Execution Style:

- Do exactly what I ask
- Use only the information and approach I specify
- No additional suggestions or improvements unless explicitly requested
- Direct, honest, and straightforward responses
- Follow my instructions to the letter

Background Information I'll Provide

You will have access to my complete profile including:

Professional Background:

- Multiple versions of my resume
- Detailed project descriptions and achievements
- Technical skills and competencies
- Work experience and accomplishments
- Education and certifications

Personal Context:

- Current job search situation
- Career goals and preferences
- Personal circumstances (international student status, location, etc.)
- Target companies and roles
- Timeline and constraints

Communication Preferences:

- My writing style and tone preferences
- Professional brand and positioning
- Key messages I want to emphasize
- Areas I want to downplay or handle carefully

Tasks You'll Execute

Cover Letter Writing:

- Write cover letters exactly as I specify
- Use the tone, length, and structure I request
- Include only the points I want emphasized
- Follow any specific formatting or style requirements

Application Question Responses:

- Answer job application questions directly
- Use my experiences and examples as I direct
- Match the word count and format requirements
- Address exactly what I tell you to address

LinkedIn Messaging:

- Compose messages to hiring managers
- Write connection requests and follow-ups
- Create referral request messages
- Use the specific approach and tone I want

Email Communications:

- Draft emails to recruiters and contacts
- Write thank you notes and follow-ups
- Compose networking and referral requests
- Handle any other professional correspondence

Other Tasks:

- Fill out application forms and questionnaires
- Write responses for screening questions
- Create any other job search materials I request
- Execute any specific communication strategy I outline

Execution Guidelines

Follow Instructions Exactly:

- If I ask for 200 words, write exactly 200 words
- If I specify a tone (formal, casual, enthusiastic), match it precisely
- If I give you specific points to include, include only those points
- If I tell you to avoid something, avoid it completely

Use My Information Only:

- Reference only my actual experiences and achievements
- Use the specific examples and stories I provide
- Don't embellish or add fictional elements
- Stick to the facts and details in my background

Be Direct and Honest:

- No flowery language unless I request it
- No over-the-top enthusiasm unless I specify it
- Write in a straightforward, professional manner
- Address requirements honestly based on my actual qualifications

No Unsolicited Advice:

- Don't suggest alternative approaches
- Don't add warnings or cautions
- Don't propose improvements unless asked
- Simply execute the task as requested

Response Format

For Writing Tasks: Provide exactly what I asked for - the cover letter, message, or response with no additional commentary.

For Questions: Answer directly and concisely without additional context unless specifically requested.

For Clarifications: If something is unclear, ask one specific clarifying question and then execute.

Quality Standards

Accuracy: Use my information correctly and precisely **Completeness:** Address everything I ask for
Consistency: Match my professional brand and voice **Professionalism:** Maintain appropriate tone and format **Efficiency:** Deliver exactly what's needed without extra content

Instructions

I will provide you with my complete background information, and then give you specific tasks to execute. Your job is to complete each task exactly as I specify, using my information and following my instructions precisely.

Ready to be my reliable job application execution assistant?

Filled

Personal Job Application Assistant

You are my dedicated job application assistant. Your role is to execute exactly what I ask you to do - nothing more, nothing less. No suggestions, no additional advice, just precise execution of my requests.

Your Core Function

Primary Role: Execute my job application tasks with precision and honesty, using the detailed background information I provide about myself.

Execution Style:

- Do exactly what I ask
- Use only the information and approach I specify
- No additional suggestions or improvements unless explicitly requested
- Direct, honest, and straightforward responses
- Follow my instructions to the letter

Background Information I'll Provide

You will have access to my complete profile including:

Professional Background:

- Multiple versions of my resume
- Detailed project descriptions and achievements
- Technical skills and competencies
- Work experience and accomplishments
- Education and certifications

Personal Context:

- Current job search situation
- Career goals and preferences
- Personal circumstances (international student status, location, etc.)
- Target companies and roles
- Timeline and constraints

Communication Preferences:

- My writing style and tone preferences
- Professional brand and positioning
- Key messages I want to emphasize
- Areas I want to downplay or handle carefully

Tasks You'll Execute

Cover Letter Writing:

- Write cover letters exactly as I specify
- Use the tone, length, and structure I request
- Include only the points I want emphasized
- Follow any specific formatting or style requirements

Application Question Responses:

- Answer job application questions directly
- Use my experiences and examples as I direct
- Match the word count and format requirements
- Address exactly what I tell you to address

LinkedIn Messaging:

- Compose messages to hiring managers
- Write connection requests and follow-ups
- Create referral request messages
- Use the specific approach and tone I want

Email Communications:

- Draft emails to recruiters and contacts
- Write thank you notes and follow-ups
- Compose networking and referral requests
- Handle any other professional correspondence

Other Tasks:

- Fill out application forms and questionnaires
- Write responses for screening questions
- Create any other job search materials I request
- Execute any specific communication strategy I outline

Execution Guidelines

Follow Instructions Exactly:

- If I ask for 200 words, write exactly 200 words
- If I specify a tone (formal, casual, enthusiastic), match it precisely
- If I give you specific points to include, include only those points
- If I tell you to avoid something, avoid it completely

Use My Information Only:

- Reference only my actual experiences and achievements
- Use the specific examples and stories I provide
- Don't embellish or add fictional elements
- Stick to the facts and details in my background

Be Direct and Honest:

- No flowery language unless I request it
- No over-the-top enthusiasm unless I specify it
- Write in a straightforward, professional manner
- Address requirements honestly based on my actual qualifications

No Unsolicited Advice:

- Don't suggest alternative approaches
- Don't add warnings or cautions
- Don't propose improvements unless asked
- Simply execute the task as requested

Response Format

For Writing Tasks: Provide exactly what I asked for - the cover letter, message, or response with no additional commentary.

For Questions: Answer directly and concisely without additional context unless specifically requested.

For Clarifications: If something is unclear, ask one specific clarifying question and then execute.

Quality Standards

Accuracy: Use my information correctly and precisely **Completeness:** Address everything I ask for **Consistency:** Match my professional brand and voice **Professionalism:** Maintain appropriate tone and format **Efficiency:** Deliver exactly what's needed without extra content

Instructions

I will provide you with my complete background information, and then give you specific tasks to execute. Your job is to complete each task exactly as I specify, using my information and following my instructions precisely.

Ready to be my reliable job application execution assistant?

[FedMob\(TensorFlow, React Native, Federated Learning, PyTorch\)](#)

•

[Built React Native app implementing FedAvg algorithm enabling AI training across mobile devices achieving privacy improvement](#)

NLP

Prompt-Manager Library (Python pkg|Version Control|YAML Metadata|PyPI|Unit Tests)

[\[Link\]](#)

- Designed Python library for prompt management across LLM applications with version control and metadata tagging.
- Created hierarchical prompt storage system, reducing prompt iteration cycles and development time by 33%.

NutriPedia: Food Recognition & Calorie Estimation (TensorFlow|Inception-V3|GCP Storage|GPT-3.5 API|SQL)

[\[Link\]](#)

- Architected cloud-based nutrition platform integrating Inception-V3 CNN with GCP Storage for image processing|implementing GPT-3.5 API for personalized dietary insights from SQL nutritional database.

- Engineered full-stack data pipeline connecting computer vision classifications with NLP-powered contextual responses|creating flywheel architecture for continuous model improvement through user feedback.

COVID-19 Bot (Dialogflow|Flask|MongoDB|RapidAPI|Webhooks)

[\[Link\]](#)

- Architected full-stack Telegram conversational chatbot with DialogFlow NLP for intent recognition|Flask/Python backend|and MongoDB persistence|enabling real-time pandemic statistics retrieval via RapidAPI.
- Implemented context-aware entity extraction pipeline supporting multilingual medical assistance|resource localization|and vaccination scheduling with custom webhook fulfillment handlers.

ExpenseGenie Bot (Telegram|WhatsApp|Gemini LLM|Redis|Langfuse|Voice/Image Processing, GCP)

[\[Link\]](#)

- Architected modular microservices system for intelligent expense splitting across Telegram and WhatsApp platforms simultaneously.
- Implemented Redis-based state management and Langfuse monitoring achieving 99.9% reliability and comprehensive LLM observability.

Data Scientist/Data Analyst/Data Engineer

Chronic Kidney Disease Predictor (MERN Stack|Random Forest|Flask|MongoDB|Python|XGBoost)

[\[Link\]](#)

- Engineered full stack MERN application for early CKD detection utilizing Random Forest classifier achieving 99.75% diagnostic accuracy through intelligent data completion algorithm for handling missing values in medical datasets.
- Developed intuitive clinical decision support interface with secure MongoDB persistence, enabling healthcare professionals to input patient biomarkers for immediate risk assessment and early intervention.

KafkaSpark – User Data Processing Engine (Apache Kafka|Apache Spark|Airflow|Cassandra|Docker)

[\[Link\]](#)

- Engineered end-to-end data pipeline connecting API data sources to Apache Cassandra through Kafka queues|implementing distributed Spark processing and real-time data streaming.
- Developed containerized architecture with Docker Compose|integrating Apache Airflow for workflow orchestration and implementing comprehensive monitoring through Kafka Control Center.

Telco Customer Churn Pipeline (Python|MLflow|XGBoost|GitHub Actions|AWS ECR/EC2)

[\[Link\]](#)

- Designed an end-to-end MLOps pipeline with modular architecture|implementing automated CI/CD workflows that reduced development time by 30%.
- Engineered scalable ETL processes and predictive analytics system using MLflow experiment tracking|achieving 15% improvement in F1-score for churn prediction.
- Developed and deployed containerized prediction API on AWS EC2/ECR through GitHub Actions|enabling robust model serving with automated testing and versioning.

Airline Passenger Forecasting (SARIMA|Statsmodels|Pandas|Matplotlib|A/B Testing) [\[Link\]](#)

- Applied ARIMA and SARIMA models on airline passenger data (1949-1960)|performing A/B testing that demonstrated SARIMA's superior fit for seasonal passenger patterns.
- Implemented time series decomposition and stationarity testing (ADF|KPSS)|visualizing forecast accuracy with 95% confidence intervals to enable reliable capacity planning.

Flight Fare Predictor (Random Forest|ExtraTrees|Flask|Scikit-learn|Feature Engineering) [\[Link\]](#)

- Developed ML system predicting flight prices with 80% accuracy using Random Forest Regression.
- Engineered feature extraction pipeline identifying key price determinants (total stops|journey day|airline carrier) through ExtraTreesRegressor analysis across multiple parameters.

Twitter Metrics Dashboard (Tableau|SQL|Window Functions|Interactive Filters|KPI Design) [\[Link\]](#)

- Created interactive Tableau dashboard visualizing Twitter metrics|providing insights into impression rates and engagement patterns across different time periods.
- Designed a comprehensive performance tracking system with filter-based interactivity|enabling social media managers to identify optimal posting schedules and content strategies.

Google Play Store Apps Dashboard (Tableau|Tree Maps|Parameter Controls|Data Blending|SQL) [\[Link\]](#)

- Developed a comprehensive Tableau dashboard analyzing app performance metrics across categories|revealing correlations

between ratings|installs|and content ratings.

- Created interactive visualization system with tree maps and filter controls|enabling product managers to identify competitive market gaps and optimize app development strategies.

FDI Sector Explorer (Tableau|Timeseries LODs|Dual-Axis Charts|CSV ETL|Calculated Fields) [\[Link\]](#)

- Created interactive Tableau dashboard visualizing Foreign Direct Investment trends across Indian sectors|revealing highest and lowest investment patterns over time.
- Designed an intuitive visual analytics system with comparative metrics and filter-based exploration|enabling stakeholders to identify strategic investment opportunities across economic sectors.

World Bank Project Success Classifier (XGBoost|Hyperopt|Imbalanced-learn|Python|SHAP)

[\[Link\]](#)

- Developed predictive model achieving 71.7% F1 score to identify World Bank project success factors|implementing XGBoost classification with hyperparameter optimization.
- Engineered comprehensive data processing pipeline for 17,500+ hierarchical records|addressing 30%+ missing data and class imbalance to deliver actionable insights for resource allocation.

Analysis on Various Commodities (LSTM|TensorFlow|Web Scraping|BeautifulSoup|Plotly)

[\[Link\]](#)

- Engineered comprehensive commodities analysis system with custom web scraper|extracting 10 years of daily price data to identify strong cross-market correlations.
- Implemented LSTM neural networks for time series forecasting across six major commodities|achieving low MSE scores (0.27-0.40) for accurate price movement predictions.

Car Price Predictor (Random Forest|EDA|Flask|Bootstrap|Pandas) [\[Link\]](#)

- Developed machine learning application predicting used car prices with Random Forest Regression|comparing multiple algorithms to identify optimal prediction accuracy.
- Implemented comprehensive data pipeline with feature engineering and EDA|creating an intuitive Flask-based web interface for real-time price estimation.

Telco Customer Churn Project [\[Link\]](#)

- Designed an end-to-end MLOps pipeline with modular architecture|implementing automated CI/CD workflows that reduced development time by 30%.
- Engineered scalable ETL processes and predictive analytics system using MLflow experiment tracking|achieving 15% improvement in F1-score for churn prediction.
- Developed and deployed containerized prediction API on AWS EC2/ECR through GitHub Actions|enabling robust model serving with automated testing and versioning.

Bollywood Movie Recommender System (Python|BeautifulSoup|Cosine Similarity|TF-IDF|Streamlit) [\[Link\]](#)

- Engineered comprehensive ETL pipeline extracting data from 3,100+ Bollywood movies using Python and BeautifulSoup|implementing statistical analysis workflows for recommendation algorithms.
- Developed automated data processing system with 11 normalized features|creating structured datasets that enabled content-based filtering and similarity analysis for personalized movie recommendations.

Bioacoustic Data Analytics (TensorFlow|STFT|CNN|Spectrogram Augment|Data Pipeline)

[\[Link\]](#)

- Engineered end-to-end audio processing pipeline transforming raw forest recordings into spectrogram datasets using TensorFlow|achieving 100% precision and recall for Capuchin bird call detection.
- Implemented signal processing algorithms with Short-Time Fourier Transform for feature extraction|optimizing CNN architecture to distinguish target patterns from environmental noise.

Sports Predictive Analytics (Scikit-learn|GridSearchCV|Feature Engineering|Flask|RMSE Metric) [\[Link\]](#)

- Developed regression models predicting IPL cricket scores with RMSE of 20 runs|implementing feature engineering techniques and hyperparameter optimization through GridSearchCV to analyze complex sports metrics.
- Engineered end-to-end ML solution from data preprocessing to deployment|creating Flask web application with interactive interface for real-time cricket score predictions based on match conditions.

Automated License Recognition (TensorFlow OD API|MobileNet-SSD|EasyOCR|OpenCV|ROI Filtering) [\[Link\]](#)

- Implemented real-time computer vision system using TensorFlow Object Detection API and transfer learning with SSD MobileNet v2|creating an end-to-end pipeline for license plate detection.

- Engineered post-processing algorithms and OCR integration using EasyOCR|optimizing detection accuracy through ROI filtering and confidence thresholds for automated data extraction.

Anime Data Synthesis (PyTorch|DC-GAN|Custom Loss|Data Augment|Hyperparam Tuning) [\[Link\]](#)

- Implemented Deep Convolutional Generative Adversarial Network (DC-GAN) using PyTorch|engineering custom data collection pipelines and preprocessing workflows for large-scale anime image datasets.
- Developed specialized training architecture with modified loss functions|generating high-quality synthetic anime character images while maintaining style consistency and feature diversity.

Banking Investment Portfolio Dynamics (Tableau|Statistical Tests|SQL|Time-Series LODs|Regulatory Analysis) [\[Link\]](#)

- Analyzed regulatory impact on banking portfolios using statistical analysis and Tableau|revealing 4-fold HTM ratio increases in large banks following capital requirement changes.
- Created financial data visualizations tracking investment trend divergence between banking segments|providing actionable insights for regulatory compliance strategies.

Medical Appointment and Patient Record System (SQL|Data Modeling|Power BI|ETL|Validation Rules) [\[Link\]](#)

- Engineered SQL queries and database reporting system capturing patient-doctor relationships across 6 interconnected healthcare entities with complex data validation rules.
- Developed analytics dashboards reducing report generation time by 40%|enabling data-driven resource allocation decisions for healthcare administrators.

ACHIEVEMENTS

-
- Kaggle Notebook and Dataset Expert. || Teaching Assistant for Intro to Machine Learning (CSCI-739)
 - Open Source Contributor at OpenVino (Intel)|Ivy || 3x Hackathon Winner

RESEARCH PAPERS

-
- **Tubules Detection on Breast Carcinoma Whole-Slide Images** – C-MIMI (Johns Hopkins)|Sep 2022.
 - **Web-Based Mitosis Detection on Breast Cancer Images via Faster RCNN & YOLOv5** – IJACSA|Dec 2022.

RESEARCH PAPERS

-
- **Tubules Detection on Breast Carcinoma Whole Slide Images Using Artificial Intelligence** - Presented at C-MIMI Conference|Johns Hopkins University
 - **Web-Based Mitosis Detection on Breast Cancer Whole Slide Images Using Faster R-CNN and YOLOv5** - International Journal of Advanced Computer Science Applications

Project Information Header

- **Project:** FedDisco Mobile Federated Learning Application
- **Company:** Academic Capstone Project
- **Role:** Lead Developer/Researcher
- **Duration:** 5 Weeks (First Prototype Completed)

Summary STAR (1.5-2.5 minutes speaking time)

S: For my academic capstone project, I saw that mobile apps handling sensitive user data like images and text faced a major problem - traditional machine learning required centralizing data, which created privacy risks and communication bottlenecks that users weren't comfortable with.

T: I needed to implement the FedDisco federated learning algorithm from scratch to enable distributed machine learning across mobile devices without compromising user privacy, delivering a working prototype in just 5 weeks for academic review.

A: I built a complete system using React Native for the mobile client, Python FastAPI for the server, and TensorFlow Lite for on-device training. I implemented custom FedDisco with optimized parameters and created a modular architecture starting with image processing while planning for text functionality.

R: Delivered a working prototype in 5 weeks that achieved 70% reduction in communication overhead, 60% memory optimization for resource-constrained devices, and demonstrated convergence in 15 rounds versus the theoretical 25 rounds - successfully passing academic validation while proving federated learning could work practically on mobile devices.

Detailed STAR

S - SITUATION Situation Summary: For my academic capstone project, I identified the critical need for privacy-preserving machine learning in mobile applications where traditional centralized approaches create privacy risks and communication bottlenecks, particularly problematic for personal content like images and text.

Detailed Context:

- **Company context & business problem:** As part of my capstone project, I identified the critical need for privacy-preserving machine learning in mobile applications where users' sensitive data (images, text) cannot be centralized due to privacy concerns. Traditional ML approaches require data centralization, creating privacy risks and communication bottlenecks, particularly problematic for mobile applications handling personal content.
- **Team structure & your role:** Independent capstone project with faculty advisor guidance. As Lead Developer/Researcher, I was responsible for the complete system architecture, implementation of both client and server components, and delivery of a working prototype demonstrating federated learning capabilities on mobile devices.
- **Timeline & constraints:** Aggressive 8-week timeline for first prototype delivery with significant system limitations. Need to balance proof-of-concept development with production-ready architecture decisions. Academic environment constraints including limited computational resources and requirement for comprehensive documentation.
- **Scale/scope of the challenge:** Complete federated learning system from scratch, mobile optimization, privacy preservation, academic-quality prototype, 8-week delivery

T - TASK/CHALLENGE Task Summary: I was tasked with building a complete mobile application implementing the FedDisco (Federated Learning) algorithm that enables privacy-preserving machine learning across distributed mobile devices, starting with image processing capabilities as the foundational component.

Detailed Responsibilities:

- **Your specific responsibilities:**
 - Design and implement FedDisco algorithm for mobile federated learning environment
 - Build client-side mobile application handling local training on user devices
 - Develop server-side aggregation system managing global model updates
 - Implement image processing pipeline as first prototype component
 - Create modular architecture supporting future text upload functionality
 - Ensure privacy-preserving design where raw data never leaves user devices
 - Deliver working prototype within 8-week academic deadline
- **Technical challenges faced:**
 - Implementing complex federated learning algorithm within mobile resource constraints
 - Balancing local computation efficiency with global model convergence quality
 - Managing communication optimization between multiple mobile clients and central server
 - Building modular system architecture supporting both image and text processing workflows
- **Business/stakeholder challenges:**
 - Delivering academic-quality prototype within tight 8-week timeline
 - Balancing comprehensive system design with practical implementation constraints
 - Managing scope effectively by prioritizing image processing before text functionality
 - Ensuring prototype demonstrates clear federated learning value proposition
- **Success criteria and constraints:** 8-week delivery, working federated learning, privacy preservation, mobile optimization, academic validation, modular architecture

A - ACTION (Detailed Breakdown) Action Summary: I successfully architected and implemented a comprehensive federated learning system using modular design principles, deployed the FedDisco algorithm with optimized communication protocols, and strategically executed phased development delivering a working image processing prototype while establishing infrastructure for text processing expansion.

Technical Stack Used:

- **Mobile Client:** React Native (cross-platform deployment enabling iOS/Android testing)
- **Server Architecture:** Python + FastAPI (lightweight server achieving efficient model aggregation)
- **Federated Learning:** Custom FedDisco implementation with optimized parameters (K=5 test clients, C=0.6 fraction, E=3 epochs, B=16 batch size)
- **Image Processing:** TensorFlow Lite (on-device training achieving 40% memory reduction)
- **Communication:** RESTful APIs with model compression (achieved 70% communication overhead reduction)
- **Security:** Encrypted model updates with SHA-256 hashing ensuring privacy-preserving aggregation

Collaboration Details:

- **Worked with:** Faculty advisor (algorithm validation), 3 peer testers (prototype validation), academic review committee
- **How decisions were made:** Made independent technical architecture decisions with weekly advisor consultations on federated learning convergence optimization
- **Communication challenges/successes:** Delivered comprehensive documentation and live demonstration showcasing federated learning capabilities
- **Cross-functional coordination:** Balanced academic research requirements with practical implementation constraints and peer validation

Leadership Moments:

- **Decisions you drove:** Successfully implemented phased development approach prioritizing image processing, enabling prototype delivery within 8-week constraint
- **People you influenced/mentored:** Technical innovation building modular architecture with plugin-based design enabling seamless future expansion
- **Processes you improved:** Established systematic testing protocol validating federated learning convergence across 5 simulated mobile clients
- **Initiative you took beyond assigned tasks:** Built comprehensive documentation framework enabling future researchers to build upon implementation

Problem-Solving Process:

- **Mobile Resource Constraints:** Solved memory limitations by implementing TensorFlow Lite with quantization reducing model size by 60%
- **Communication Efficiency:** Achieved 70% bandwidth reduction through gradient compression and selective model parameter updates
- **Convergence Optimization:** Tuned FedDisco parameters ($E=3$, $C=0.6$) achieving stable convergence in 15 rounds vs. theoretical 25 rounds
- **Alternative Solutions Considered:** Evaluated Flutter vs React Native (chose React Native for faster development), PyTorch vs TensorFlow (chose TensorFlow Lite for mobile optimization)

R - RESULTS Result Summary: Successfully delivered a working federated learning prototype demonstrating privacy-preserving machine learning on mobile devices, achieving significant performance improvements in communication efficiency and model convergence while establishing a robust foundation for comprehensive multi-modal federated learning applications.

Detailed Impact:

- **Quantified business impact:**
 - **8-week prototype delivery** successfully completed, demonstrating federated learning feasibility on mobile platforms
 - **70% reduction** in communication overhead through model compression and selective parameter updates
 - **60% memory optimization** enabling deployment on resource-constrained mobile devices
 - **Academic validation** with successful demonstration to review committee and peer testing completion

- **Technical achievements:**

- **Working FedDisco implementation** achieving convergence in 15 rounds (40% improvement over theoretical baseline)
- **Cross-platform mobile application** successfully tested on both iOS and Android with consistent performance
- **Privacy-preserving architecture** validated through security audit showing zero raw data transmission
- **Modular plugin architecture** built enabling text processing integration with 90% code reuse

- **Team/process improvements:**

- **Rapid prototyping methodology** proved effective for delivering complex federated learning system within academic constraints
- **Comprehensive test suite** established covering federated learning convergence, mobile performance, and privacy validation
- **Documentation framework** created enabling future researchers to build upon federated learning mobile implementation
- **Peer validation process** implemented with 3 independent testers confirming prototype functionality

- **Personal growth/skills gained:**

- **Federated learning expertise:** Practical implementation experience with FedDisco algorithm optimization for mobile environments
- **Mobile AI development:** Advanced skills in TensorFlow Lite optimization and on-device machine learning deployment
- **System architecture mastery:** Proven ability to design modular, extensible systems under tight timeline constraints
- **Academic research execution:** Successfully delivered research-quality prototype meeting academic standards while demonstrating practical applicability

Key Prototype Achievements:

- **5 concurrent clients** successfully participating in federated learning rounds
- **87% image classification accuracy** maintained while preserving complete data privacy
- **45 seconds average training time** per client per round on standard mobile devices
- **Successful model convergence** demonstrated across heterogeneous mobile device configurations

Project Information Header

- **Project:** Budget Justification Data Transfer Automation
- **Company:** Rochester Institute of Technology (Office of Vice President of Research)
- **Role:** Machine Learning Engineer Co-op
- **Duration:** June 2025 - Present

Summary STAR (1.5-2.5 minutes speaking time)

S: RIT's Pre-Award team was drowning in manual work - they had to process 800+ annual federal research proposals by manually transferring budget data from Excel files into Word templates, which was creating bottlenecks during peak periods and causing 5-8 calculation errors monthly that could jeopardize proposal compliance.

T: I needed to design and build a comprehensive automation solution that could eliminate all the manual data transfer, ensure 100% calculation accuracy, and provide a user-friendly interface for non-technical staff while maintaining strict federal compliance requirements.

A: I built a Streamlit web portal with robust Excel parsing using pandas/openpyxl and automated Word document generation with python-docx. I worked closely with the Pre-Award Director and stakeholders across 5 colleges to handle diverse Excel formats and ensure seamless integration with existing workflows.

R: Achieved a 60% reduction in manual processing time, eliminated calculation errors entirely (down from 5-8 monthly), saved \$50,000 annually in administrative costs, and successfully automated all 800+ annual proposals while maintaining federal compliance - allowing staff to focus on strategic review instead of data entry.

Detailed STAR

S - SITUATION Situation Summary: Rochester Institute of Technology's Pre-Award team was struggling with managing 800+ annual federal research proposals requiring manual data entry from Excel to Word templates, creating bottlenecks during peak submission periods and causing calculation errors that could jeopardize proposal compliance.

Detailed Context:

- **Company context & business problem:** RIT's Pre-Award team managed 800+ annual federal research proposals requiring detailed budget justifications and corresponding narratives. Each proposal demanded manual data entry from Excel into Word document templates, creating bottlenecks during peak submission periods. The team of 2 administrators and 4 faculty researchers struggled with year-end budget transfers, often working overtime to meet federal deadlines. Manual processes led to formatting inconsistencies and calculation errors that could jeopardize proposal compliance.
- **Team structure & your role:** Office of Vice President of Research team including Pre-Award Director, 2 senior administrators, 4 faculty researchers, and budget officers from 5 different colleges. As Machine Learning Engineer Co-op, I was responsible for designing and implementing the complete automation solution to eliminate manual data transfer processes.
- **Timeline & constraints:** Ongoing project from June 2025 requiring immediate impact during peak submission periods. Critical requirement for 100% accuracy in financial calculations and federal compliance. Need for user-friendly interface for non-technical staff members.

- **Scale/scope of the challenge:** 800+ annual proposals, diverse Excel formats, federal compliance requirements, non-technical user base, peak period bottlenecks

T - TASK/CHALLENGE Task Summary: I was tasked with designing and implementing a comprehensive automation solution that could eliminate manual budget data transfer processes, ensure 100% calculation accuracy, and provide a user-friendly interface for non-technical staff while maintaining federal proposal compliance.

Detailed Responsibilities:

- **Your specific responsibilities:**
 - Design an automated solution to eliminate manual budget data transfer
 - Ensure 100% accuracy in financial calculations and formatting
 - Create user-friendly interface for non-technical staff
 - Maintain compliance with federal proposal requirements
 - Build robust system handling diverse Excel formats and data validation
 - Implement automated Word document generation with consistent formatting
- **Technical challenges faced:**
 - Building intuitive Streamlit interface for non-technical users
 - Handling diverse Excel formats and data validation
 - Automating Word document generation with consistent formatting
 - Real-time file processing and error handling in web environment
 - Ensuring data security during file upload/download processes
- **Business/stakeholder challenges:**
 - Minimizing disruption to existing workflows during implementation
 - Training staff on new automation tools
 - Ensuring backup procedures for system failures during critical deadlines
 - Managing change management across multiple colleges and departments
- **Success criteria and constraints:** 100% calculation accuracy, 60%+ time reduction, federal compliance, user-friendly design, 800+ proposal capacity

A - ACTION (Detailed Breakdown) Action Summary: I built a comprehensive Streamlit web portal with robust Excel parsing and automated Word generation capabilities, working closely with stakeholders across multiple colleges to ensure seamless integration with existing workflows while achieving complete automation of the budget transfer process.

Technical Stack Used:

- **Python:** Core automation logic and data processing (chosen for robust data manipulation and document generation capabilities)
- **Streamlit:** Interactive web portal interface (selected for rapid development and user-friendly design for non-technical staff)
- **pandas/openpyxl:** Excel file reading and data manipulation (comprehensive support for diverse Excel formats and calculations)
- **python-docx:** Automated Word document generation (precise formatting control and template integration)
- **Word Mail Merge templates:** Consistent formatting (ensuring federal compliance and professional presentation)

- **File upload/download handlers:** Seamless user experience (secure temporary storage and automatic cleanup)

Collaboration Details:

- **Worked with:** Pre-Award Director (requirements), 2 senior administrators (workflow validation), faculty researchers (budget requirements), budget officers from 5 colleges (diverse use cases)
- **How decisions were made:** Led requirements gathering sessions with 15+ end users to understand diverse budget requirements and workflow constraints
- **Communication challenges/successes:** Regular stakeholder meetings ensuring broad adoption across different colleges and departments
- **Cross-functional coordination:** Partnered with IT Security team for system integration approval while balancing automation needs with security requirements

Leadership Moments:

- **Decisions you drove:** Led requirements gathering sessions with 15+ end users and drove decision to adopt modular architecture for future scalability
- **People you influenced/mentored:** Mentored 1 undergraduate intern on automation scripting techniques and established testing protocols
- **Processes you improved:** Established comprehensive testing protocols involving real proposal data validation and standardized budget formatting
- **Initiative you took beyond assigned tasks:** Created documentation and training materials for knowledge transfer beyond basic system requirements

Problem-Solving Process:

- **Standardization Challenge:** Analyzed 200+ budget templates to design standardized Excel input format accommodating diverse requirements
- **User Experience:** Built Streamlit prototype with file upload/download functionality ensuring intuitive operation for non-technical users
- **Quality Assurance:** Implemented real-time data validation and error messaging in portal preventing calculation errors
- **Trade-off Analysis:** Chose web-based Streamlit portal for accessibility vs. desktop application for advanced features, prioritizing user adoption

R - RESULTS Result Summary: The automation system achieved remarkable efficiency improvements with 60% reduction in processing time, eliminated calculation errors entirely, saved \$50,000 annually in administrative costs, and successfully automated all 800+ annual proposals while maintaining federal compliance standards.

Detailed Impact:

- **Quantified business impact:**
 - **60% reduction** in manual processing time for budget preparation
 - **Zero calculation errors** since implementation (previously 5-8 errors monthly)
 - **\$50,000 annual cost savings** in administrative overtime elimination
 - **800+ proposals** now processed automatically per year with consistent quality
- **Technical achievements:**

- **Built user-friendly Streamlit web portal** accessible to all staff members regardless of technical background
- **Created robust Excel parsing system** handling multiple budget formats and diverse college requirements
- **Developed automated Word document generation** with consistent styling meeting federal compliance standards
- **Implemented comprehensive error handling** and user feedback systems preventing processing failures
- **Team/process improvements:**
 - **Strategic focus shift:** Staff can now focus on strategic proposal review vs. manual data entry tasks
 - **Standardized budget formatting** across all funding agencies ensuring consistency and compliance
 - **Created comprehensive documentation** and training materials enabling knowledge transfer and system maintenance
 - **Improved deadline management:** Eliminated overtime work during peak submission periods
- **Personal growth/skills gained:**
 - **Enterprise automation expertise:** Gained expertise in enterprise automation and change management for research environments
 - **Stakeholder communication mastery:** Developed skills in stakeholder communication and requirements gathering across diverse user groups
 - **Compliance understanding:** Enhanced understanding of federal research compliance requirements and proposal formatting standards
 - **User-centric design:** Experience building intuitive systems for non-technical users in high-stakes environments

Project Information Header

- **Project:** Conversational AI System for Clinical Trial Data Quality Analysis
- **Company:** Daiichi Sankyo
- **Role:** R&D Data Governance Intern
- **Duration:** [Duration not specified - ongoing project]

Summary STAR (1.5-2.5 minutes speaking time)

S: Daiichi Sankyo wanted to explore whether Large Language Models could replace their manual Tableau-based data quality analysis for clinical trial data, but I had to work in a highly regulated environment where cloud services took months to approve, with only 32GB RAM systems, no GPU acceleration, and no specialized clinical datasets.

T: I needed to build a proof-of-concept conversational AI system that could handle complex relational clinical trial data queries while maintaining regulatory compliance, despite severe computational constraints and limited access to advanced AI infrastructure.

A: I developed three iterative versions, progressing from tabular models (TAPAS, multitab-qa-base) to sophisticated RAG architecture using Mistral 7B with local deployment via Ollama, creating custom clinical datasets and building a modular system with Fast, Balanced, and Quality configurations.

R: Successfully delivered a proof-of-concept with three distinct configurations achieving 4-9 minute response times,

I also established a regulatory-compliant AI development framework, and identified a clear production pathway using SQL Translation Layer that could reduce response times to seconds once cloud approvals come through.

Detailed STAR

S - SITUATION Situation Summary: Daiichi Sankyo's R&D team needed to transform their manual Tableau-based data quality analysis into a conversational AI interface for clinical trial data, while exploring LLM capabilities within a highly regulated environment that severely limited cloud access and computational resources.

Detailed Context:

- **Company context & business problem:** Daiichi Sankyo's R&D team needed to transform their Tableau-based data quality analysis into a conversational interface for clinical trial data. The existing manual analysis process was inefficient, and stakeholders wanted to explore whether Large Language Models could effectively interact with complex relational clinical trial data while maintaining regulatory compliance standards.
- **Team structure & your role:** R&D Data Governance team collaborating with database administrators and clinical data experts. As R&D Data Governance Intern, I was responsible for designing and implementing a proof-of-concept conversational AI system that could handle complex clinical trial data queries while working within strict regulatory constraints.
- **Timeline & constraints:** Highly regulated database environment where obtaining permissions for cloud services (AWS, Azure) required extensive approval processes lasting months. Significant technical constraints including 32GB RAM systems, no GPU acceleration, and absence of specialized training datasets for clinical domain. Limited to 100-200 rows for POC development due to regulatory restrictions.
- **Scale/scope of the challenge:** Proof-of-concept development, regulatory compliance requirements, severe computational limitations, complex relational clinical data

T - TASK/CHALLENGE Task Summary: I was tasked with building a proof-of-concept conversational AI system that could replace manual Tableau analysis workflows, requiring exploration of multiple technical approaches while navigating strict regulatory constraints that limited cloud access and computational resources.

Detailed Responsibilities:

- **Your specific responsibilities:**
 - Design and implement conversational interface for clinical trial data quality analysis
 - Explore multiple AI architectures including tabular models, RAG systems, and SQL translation layers
 - Build custom datasets and fine-tune models for clinical domain understanding
 - Navigate regulatory constraints preventing immediate cloud service access
 - Develop modular system architecture supporting different performance/quality trade-offs
 - Create comprehensive clinical trial glossary and domain-specific terminology framework

- **Technical challenges faced:**

- Building effective AI system with 32GB RAM and no GPU acceleration in regulated environment
- Handling complex relationships across relational clinical trial data with limited computational resources
- Creating training datasets without access to specialized clinical domain models
- Achieving reasonable response times (initial attempts taking 4-9 minutes per query)

- **Business/stakeholder challenges:**

- Regulatory compliance requirements preventing immediate access to advanced cloud AI services
- Balancing proof-of-concept development with production-ready performance expectations
- Managing stakeholder expectations while working within severe computational and data constraints
- Demonstrating AI value proposition despite significant technical limitations

- **Success criteria and constraints:** Regulatory compliance, proof-of-concept completion, multiple architecture evaluation, local deployment, clinical domain understanding

A - ACTION (Detailed Breakdown) Action Summary: I developed a comprehensive three-version iterative approach, progressing from tabular models to sophisticated RAG architecture while building regulatory-compliant solutions that demonstrated conversational AI potential despite significant constraints.

Technical Stack Used:

- **Version 1:** Tabular models (multitab-qa-base, TAPAS) + FAISS Vector Stores (specialized for structured data)
- **Version 2:** Modular RAG with Mistral 7B + multiple embedding options (all-MiniLM-L6-v2, all-mpnet-base-v2, e5-large)
- **Version 3 (Proposed):** SQL Translation Layer with Phi-3-mini (lightweight CPU-optimized deployment)
- **Custom Development:** ChatDSI for dataset creation, Ollama for local model deployment
- **Storage Solutions:** ChromaStore, HNSWStore, FAISSStore (metadata-driven vector databases)

Collaboration Details:

- **Worked with:** Database administrators (schema design), clinical data experts (domain knowledge), regulatory compliance officers (approval processes)
- **How decisions were made:** Led technical architecture decisions while coordinating with regulatory team for cloud service approval processes
- **Communication challenges/successes:** Developed comprehensive clinical trial glossary collaboratively, facilitating domain knowledge transfer between technical and clinical teams
- **Cross-functional coordination:** Balanced technical AI development with regulatory compliance requirements and clinical domain expertise

Leadership Moments:

- **Decisions you drove:** Shifted from model-centric to data-centric development strategy after Version 1 limitations
- **People you influenced/mentored:** Technical innovation designing modular RAG framework with three distinct configurations supporting different use cases
- **Processes you improved:** Created regulatory-compliant development workflow enabling AI experimentation within strict compliance constraints
- **Initiative you took beyond assigned tasks:** Developed comprehensive clinical trial glossary and domain-specific terminology framework

Problem-Solving Process:

- **Regulatory Constraints:** Built local deployment solutions using Ollama and CPU-optimized models while cloud approvals were pending
- **Performance Optimization:** Developed three-tier architecture (Fast: 4.4min, Balanced: 4.19min, Quality: 9.3min) balancing speed vs. accuracy
- **Data Limitations:** Created custom clinical trial datasets using ChatDSI and implemented comprehensive null value handling with domain-specific explanations
- **Alternative Solutions Considered:** Evaluated text-to-SQL pipelines, pretrained models with data injection, API-based solutions vs. local deployment

R - RESULTS Result Summary: Successfully delivered a comprehensive proof-of-concept demonstrating conversational AI potential for clinical data analysis, established regulatory-compliant development framework, and identified clear path forward with SQL Translation Layer approach achieving production-ready performance.

Detailed Impact:

- **Quantified business impact:**
 - **Proof-of-concept completion** demonstrating feasibility of conversational AI for clinical trial data quality analysis
 - **Three distinct architectures** validated with measurable performance characteristics (4-9 minute response times)
 - **Regulatory compliance framework** established enabling future AI development within strict pharmaceutical constraints
 - **Clear production pathway** identified through SQL Translation Layer approach reducing response times to seconds
- **Technical achievements:**
 - **Modular RAG architecture:** Built flexible system with interchangeable components supporting experimentation
 - **Custom clinical domain datasets:** Created specialized training data using ChatDSI for domain-specific fine-tuning
 - **Multi-configuration system:** Developed Fast, Balanced, and Quality configurations optimizing different performance criteria
 - **Local deployment expertise:** Successfully implemented CPU-only solutions working within regulatory constraints
- **Team/process improvements:**
 - **Regulatory AI development process:** Established framework for compliant AI experimentation in pharmaceutical environment

- **Cross-functional collaboration:** Improved coordination between technical teams and regulatory compliance officers
- **Domain knowledge integration:** Created comprehensive clinical trial glossary facilitating technical-clinical communication
- **Iterative development methodology:** Demonstrated systematic approach to AI system development under constraints
- **Personal growth/skills gained:**
 - **Regulatory AI development:** Deep understanding of pharmaceutical compliance requirements for AI system development
 - **Constrained system design:** Expertise in building effective AI solutions with severe computational and regulatory limitations
 - **Multi-architecture evaluation:** Advanced skills in systematic AI approach comparison and optimization
 - **Healthcare domain expertise:** Comprehensive knowledge of clinical trial data structures and regulatory requirements

Project Information Header

- **Project:** Automated ICF Classification System for Clinical Trial Compliance
- **Company:** Daiichi Sankyo
- **Role:** R&D Data Governance Intern
- **Duration:** 10 Months (June 2024 - April 2025)

Summary STAR (1.5-2.5 minutes speaking time)

S: Daiichi Sankyo's legal and regulatory teams were stuck manually reviewing each Informed Consent Form for 4-5 minutes to identify data sharing clauses, and with hundreds of documents across multiple studies and countries, this was creating serious delays in clinical trial operations.

T: I needed to build a complete automated classification and document management system that could maintain regulatory compliance standards while dramatically reducing processing time - with zero tolerance for misclassification errors given the regulatory implications.

A: I developed the system through four iterative versions, evolving from rule-based systems to a sophisticated two-stage transformer architecture using DistilBERT and FAISS embeddings, while building a comprehensive document management platform with study filtering, version control, and geographic organization.

R: Achieved 0.93 F1 score with 0.96 accuracy, reduced processing time by 87% from 4-5 minutes to just 33 seconds, and created a production-ready system that eliminated critical bottlenecks across hundreds of documents for multiple studies and countries.

Detailed STAR

S - SITUATION Situation Summary: Daiichi Sankyo's legal and regulatory teams were facing critical bottlenecks with manual review processes that took 4-5 minutes per Informed Consent Form to identify data sharing clauses, creating delays across hundreds of documents and risking regulatory compliance issues.

Detailed Context:

- **Company context & business problem:** Daiichi Sankyo's legal and regulatory teams were spending 4-5 minutes manually reviewing each Informed Consent Form (ICF) to identify critical data sharing clauses. With hundreds of documents across multiple studies, countries, and consent form versions, this manual process created significant bottlenecks in clinical trial operations. Missing or misinterpreting these clauses could lead to serious regulatory compliance issues.
- **Team structure & your role:** R&D Data Governance team collaborating with legal reviewers, regulatory specialists, and clinical trial operations. As R&D Data Governance Intern, I was the primary technical developer responsible for designing and implementing the complete automated classification system and document management platform.
- **Timeline & constraints:** 10-month project requiring iterative development through four major model versions. No pre-existing labeled datasets available. Critical requirement to maintain regulatory compliance standards while dramatically reducing processing time from manual 4-5 minutes to automated seconds.
- **Scale/scope of the challenge:** Hundreds of documents, multiple studies and countries, zero tolerance for misclassification, regulatory compliance requirements

T - TASK/CHALLENGE Task Summary: I was tasked with building a complete automated ICF classification and management system that could eliminate manual review bottlenecks, ensure regulatory compliance, and provide comprehensive document management capabilities across multiple studies and geographic regions.

Detailed Responsibilities:

- **Your specific responsibilities:**
 - Develop automated classification system progressing through four iterative model versions
 - Build comprehensive PDF database and management platform with study-based filtering
 - Create document comparison tools for version control across trial phases
 - Implement geographic filtering for country/region regulatory compliance
 - Design user-friendly interface enabling legal reviewers to edit classifications and provide feedback
 - Achieve production-ready performance reducing processing time from 4-5 minutes to under 33 seconds
- **Technical challenges faced:**
 - Severe class imbalance causing initial F1 scores of 0.58 across multiple model iterations
 - Building robust PDF parsing maintaining document structure across varied international formats
 - Implementing two-stage architecture balancing accuracy with processing speed requirements

- Creating comprehensive document management system supporting complex clinical trial workflows
- **Business/stakeholder challenges:**
 - Eliminating critical bottlenecks in clinical trial operations affecting multiple departments
 - Ensuring zero tolerance for misclassification errors given regulatory compliance implications
 - Managing iterative feedback from legal reviewers while maintaining system usability
 - Supporting complex multi-study, multi-country document organization requirements
- **Success criteria and constraints:** 87%+ time reduction, 0.9+ F1 score, regulatory compliance, zero misclassification tolerance, 10-month iterative development

A - ACTION (Detailed Breakdown) Action Summary: I architected a sophisticated four-version iterative development process, progressing from rule-based systems to advanced two-stage transformer architecture, while building a comprehensive document management platform that achieved 0.93 F1 score and 87% processing time reduction.

Technical Stack Used:

- **Model Evolution:** Rule-based → TF-IDF ensemble → Transformer-based → Two-stage architecture (embedding retrieval + DistilBERT classification)
- **Embeddings & Search:** all-MiniLM-L6-v2 + FAISS IndexFlatIP (similarity threshold empirically set at 0.45)
- **Classification:** DistilBERT fine-tuned with 5-fold cross-validation and learning rate scheduling
- **Frontend Evolution:** Streamlit → Flask + PDF.js + responsive web interface
- **Document Management:** Comprehensive filtering (study, version, country, date), batch processing, version control
- **Optimization:** ONNX runtime conversion, batch processing for multi-document scenarios

Collaboration Details:

- **Worked with:** Legal reviewers (classification validation), regulatory specialists (compliance requirements), clinical operations (workflow integration)
- **How decisions were made:** Implemented active learning techniques prioritizing ambiguous cases for expert review
- **Communication challenges/successes:** Developed annotation guidelines and consensus-based review processes for disputed classifications
- **Cross-functional coordination:** Balanced technical ML development with strict legal and regulatory requirements across multiple departments

Leadership Moments:

- **Decisions you drove:** Chose two-stage architecture approach achieving optimal balance between accuracy and processing speed
- **People you influenced/mentored:** Strategic innovation building comprehensive document management platform beyond simple classification
- **Processes you improved:** Implemented human feedback loops and active learning for continuous model refinement
- **Initiative you took beyond assigned tasks:** Built comprehensive document management platform supporting entire clinical trial workflow

Problem-Solving Process:

- **Performance Bottleneck:** Solved 4-5 minute manual review time through two-stage automated architecture achieving 33-second processing
- **Class Imbalance:** Overcame F1 score limitation (0.58 → 0.93) through iterative model architecture improvements and synthetic data techniques
- **Regulatory Compliance:** Ensured zero tolerance for misclassification through comprehensive validation and human-in-the-loop workflows
- **Alternative Solutions Considered:** Evaluated various embedding models, classification approaches, and frontend architectures across four development versions

R - RESULTS Result Summary: The system achieved exceptional regulatory-grade performance with 0.93 F1 score and 87% processing time reduction (4-5 minutes to 33 seconds), while providing comprehensive document management capabilities supporting complex multi-study clinical trial operations.

Detailed Impact:

- **Quantified business impact:**
 - **87% processing time reduction** (4-5 minutes manual review to 33 seconds automated processing)
 - **0.93 F1 score** with 0.96 accuracy exceeding industry standards for regulatory compliance applications
 - **Hundreds of documents** now processed efficiently across multiple studies, countries, and consent form versions
 - **Eliminated critical bottlenecks** in clinical trial operations affecting legal and regulatory teams
- **Technical achievements:**
 - **Four-version iterative development:** Successfully evolved from 0.58 F1 score to 0.93 through systematic model improvements
 - **Comprehensive document platform:** Built complete management system with study-based filtering, version control, and geographic organization
 - **Two-stage production architecture:** Achieved optimal performance through embedding-based retrieval + efficient DistilBERT classification
 - **Advanced document capabilities:** Implemented batch processing, document comparison, full-text search, and automated reporting
- **Team/process improvements:**
 - **Regulatory workflow transformation:** Eliminated manual review bottlenecks while maintaining compliance standards
 - **Cross-functional collaboration:** Established effective feedback loops between technical development and legal/regulatory teams
 - **Quality assurance framework:** Created annotation guidelines and consensus-based review processes for sustained accuracy
 - **Scalable architecture:** Built system supporting expansion across additional studies and geographic regions
- **Personal growth/skills gained:**
 - **Regulatory compliance expertise:** Deep understanding of clinical trial documentation requirements and compliance constraints

- **Iterative system development:** Advanced skills in systematic model improvement through multiple development cycles
- **Complex stakeholder management:** Experience balancing technical innovation with strict regulatory and legal requirements
- **End-to-end platform development:** Comprehensive experience building complete document management solutions beyond core ML capabilities

Project Information Header

- **Project:** CTMS Database Migration System - Alteryx to Python Migration
 - **Company:** Daiichi Sankyo
 - **Role:** R&D Data Governance Intern
 - **Duration:** 2 weeks
-

Summary STAR (1.5-2.5 minutes speaking time)

S: Daiichi Sankyo's R&D department was stuck with an Alteryx-based system for migrating clinical trial management data that was creating licensing bottlenecks, had limited customization options, and dependency issues affecting regulatory reporting workflows.

T: I had to completely migrate the entire data migration system from Alteryx to a custom Python solution in just 2 weeks, preserving all existing functionality while adding enhanced parallel processing, error handling, and scheduling capabilities.

A: I reverse-engineered all the existing Alteryx workflows and rewrote them in Python using SQLAlchemy for database connections, pandas for data processing, and implemented parallel processing with ThreadPoolExecutor - capabilities that weren't possible in the original Alteryx system.

R: Delivered a production-ready Python system that eliminated Alteryx licensing dependencies, achieved zero data loss during migration, improved performance with parallel processing, and gave the team enhanced error handling and monitoring capabilities they couldn't get with Alteryx.

Detailed STAR

S - SITUATION

Situation Summary: Daiichi Sankyo's R&D department needed to replace their existing Alteryx-based CTMS data migration system due to licensing constraints, limited customization

options, and dependency issues that were creating bottlenecks in clinical trial data availability for regulatory reporting.

Detailed Context:

- **Company context & business problem:** Daiichi Sankyo's R&D department was using an Alteryx-based system for migrating clinical trial management system (CTMS) data between databases. The Alteryx solution created licensing bottlenecks, offered limited customization for complex table relationships, and dependency issues that affected the reliability of regulatory reporting and analysis workflows.
- **Team structure & your role:** R&D Data Governance team working with database administrators, clinical data managers, and regulatory reporting analysts. As R&D Data Governance Intern, I was responsible for completely migrating the existing Alteryx workflows to a custom Python solution.
- **Timeline & constraints:** 2-week timeline to migrate from Alteryx to Python while maintaining all existing functionality. Critical requirement to preserve data integrity and ensure zero downtime during the transition from the existing Alteryx system.
- **Scale/scope of the challenge:** Complete system migration, production database systems, regulatory compliance requirements, zero data loss tolerance

T - TASK/CHALLENGE

Task Summary: I was tasked with completely rewriting the existing Alteryx-based data migration system in Python, preserving all current functionality while adding enhanced capabilities for parallel processing, error handling, and automated scheduling.

Detailed Responsibilities:

- **Alteryx to Python Migration:** Analyze existing Alteryx workflows and completely rewrite them as a Python-based automated migration system
- **Functionality Preservation:** Ensure all current CTMS data migration capabilities are maintained during the transition
- **Enhanced Architecture:** Implement improved parallel processing, error handling, and scheduling capabilities beyond Alteryx limitations
- **Seamless Transition:** Deliver production-ready system with zero downtime and no disruption to ongoing clinical trial operations
- **Documentation & Training:** Create comprehensive documentation for the new Python system to replace Alteryx workflows

Technical challenges faced:

- **Workflow Translation:** Reverse-engineering and rewriting complex Alteryx data manipulation workflows in Python
- **Feature Parity:** Ensuring the new Python system matched or exceeded all existing Alteryx functionality
- **Performance Optimization:** Implementing parallel processing capabilities that weren't possible in the Alteryx system
- **Integration Continuity:** Maintaining all existing database connections and scheduling while transitioning platforms

Business/stakeholder challenges:

- **Rapid Migration Timeline:** Completing full system migration in only 2 weeks without disrupting critical operations
- **Licensing Cost Reduction:** Eliminating Alteryx licensing dependencies while maintaining system reliability
- **Stakeholder Confidence:** Ensuring clinical data managers and regulatory analysts trusted the new Python system
- **Regulatory Compliance:** Maintaining all compliance requirements during the platform transition

A - ACTION (*Detailed Breakdown*)

Action Summary: I systematically analyzed the existing Alteryx workflows, completely rewrote them in Python with enhanced capabilities, and implemented superior parallel processing and error handling that surpassed the original system's performance.

Technical Stack Used:

- **Migration Foundation:** Reverse-engineered Alteryx workflows and translated data manipulation logic to Python
- **Database Connectivity:** SQLAlchemy + psycopg2-binary (robust PostgreSQL connections replacing Alteryx database connectors)
- **Data Processing:** pandas + numpy (efficient data transformation replacing Alteryx data manipulation tools)
- **Parallel Processing:** concurrent.futures ThreadPoolExecutor (advanced capability not available in Alteryx)
- **Configuration Management:** python-dotenv + pyyaml (flexible configuration system improving on Alteryx limitations)
- **Logging & Monitoring:** Python logging with pytz (comprehensive audit trails exceeding Alteryx capabilities)

Collaboration Details:

- **Legacy System Analysis:** Worked with database administrators to understand existing Alteryx workflow dependencies
- **Requirements Validation:** Collaborated with clinical data managers to ensure Python system met all operational needs
- **Transition Planning:** Coordinated with regulatory analysts to maintain compliance during migration
- **Testing & Validation:** Extensive parallel testing of Python system against existing Alteryx outputs

Leadership Moments:

- **Migration Strategy:** Led decision to completely rewrite in Python rather than attempting partial migration
- **Architecture Innovation:** Drove implementation of parallel processing capabilities impossible in Alteryx
- **Quality Assurance:** Established comprehensive testing protocols ensuring migration accuracy
- **Knowledge Transfer:** Created detailed documentation enabling team to maintain Python system

Problem-Solving Process:

- **Workflow Analysis:** Systematically reverse-engineered Alteryx processes to understand all data transformation logic
- **Performance Enhancement:** Implemented parallel processing using ThreadPoolExecutor, significantly improving on Alteryx performance
- **Error Handling Improvement:** Built sophisticated error recovery mechanisms beyond Alteryx capabilities
- **Validation Strategy:** Created comprehensive testing framework comparing Python outputs with Alteryx results

R - RESULTS

Result Summary: The Python migration successfully eliminated Alteryx licensing dependencies, achieved superior performance with parallel processing, and maintained zero data loss while providing enhanced capabilities that exceeded the original system's functionality.

Detailed Impact: Quantified business impact:

- **Licensing Elimination:** Removed Alteryx licensing costs and dependencies, reducing operational overhead
- **Performance Improvement:** Parallel processing reduced migration time compared to sequential Alteryx workflows
- **System Reliability:** Achieved zero data loss across all migrations during and after transition
- **Enhanced Capabilities:** Delivered superior error handling and monitoring capabilities beyond Alteryx limitations

Technical achievements:

- **Complete Migration:** Successfully translated all Alteryx workflows to Python within 2-week timeline
- **Enhanced Architecture:** Implemented parallel processing and advanced error handling not possible in Alteryx
- **Production Readiness:** Delivered robust system with comprehensive logging and monitoring capabilities
- **Seamless Transition:** Zero downtime migration maintaining all existing functionality and data integrity

Team/process improvements:

- **Platform Independence:** Eliminated vendor dependency on Alteryx licensing and support constraints
- **Customization Capability:** Enabled future enhancements and modifications not possible with Alteryx limitations
- **Knowledge Transfer:** Created comprehensive Python documentation replacing proprietary Alteryx workflows
- **System Maintainability:** Established open-source Python solution that team can modify and enhance independently

Personal growth/skills gained:

- **Legacy System Migration:** Expertise in analyzing and replacing proprietary systems with custom solutions

- **Rapid Development:** Proven ability to deliver complete system migration under aggressive 2-week timeline
- **Requirements Analysis:** Advanced skills in reverse-engineering existing workflows and improving upon them
- **Platform Translation:** Experience converting proprietary tool workflows to open-source Python solutions

Project Information Header

- **Project:** AI-Driven Content Automation & SEO Optimization System
- **Company:** White Label Resell
- **Role:** Machine Learning Engineer (Sole Backend/AI/Cloud Developer)
- **Duration:** [Duration not specified - ongoing project]

Summary STAR (1.5-2.5 minutes speaking time)

S: White Label Resell's digital marketing clients were stuck with slow, expensive manual content creation that couldn't scale for competitive e-commerce needs - they needed large-scale SEO-optimized content with internal linking and multi-language support but had no technical solution.

T: As the sole backend/AI/cloud developer, I had to single-handedly build a comprehensive AI-powered content automation system from scratch, integrating multiple AI models and cloud infrastructure to generate 50,000+ articles monthly with enterprise-grade reliability.

A: I architected a complete system integrating GPT-3.5, BERT, FLAN-T5 for content generation, Stable Diffusion for images, and Google Translate API for 50+ languages, all running on AWS cloud infrastructure with automated internal linking algorithms and scalable API systems.

R: Built enterprise-scale capabilities generating 10,000+ monthly articles as a solo developer, secured \$50-100K quarterly contracts, achieved 60% increase in organic traffic through AI-powered internal linking, and boosted international engagement by 40% with multi-language content.

Detailed STAR

S - SITUATION Situation Summary: White Label Resell needed to revolutionize content generation for digital marketing clients who were facing slow, expensive manual processes that couldn't scale for competitive e-commerce needs, requiring large-scale SEO-optimized content with advanced features like internal linking and multi-language support.

Detailed Context:

- **Company context & business problem:** White Label Resell needed to revolutionize content generation for digital marketing clients facing slow, expensive, and inconsistent manual content creation. Businesses required large-scale, SEO-optimized content with effective

internal linking, multi-language localization, and automated scalability to compete in the fast-paced e-commerce landscape.

- **Team structure & your role:** Small team with frontend developers and business stakeholders. As the sole Backend/AI/Cloud Developer, I was entirely responsible for designing and implementing the complete technical infrastructure including AI models, cloud architecture, and all backend systems.
- **Timeline & constraints:** Aggressive scaling requirements needing 10,000+ articles monthly capability. Multi-language support across 50+ languages required. High-performance expectations with enterprise-level reliability and SEO optimization standards.
- **Scale/scope of the challenge:** Building enterprise-scale system as sole developer, 10,000+ monthly articles, 50+ language support, complete technical ownership

T - TASK/CHALLENGE Task Summary: As the only technical developer, I was tasked with single-handedly building a comprehensive AI-powered content automation system from scratch, including all AI models, cloud infrastructure, API development, and deployment architecture.

Detailed Responsibilities:

- **Your specific responsibilities:**
 - Design and implement entire end-to-end content automation pipeline as sole technical developer
 - Build AI-driven content generation using multiple NLP models (GPT-3.5, BERT, FLAN-T5)
 - Develop multi-language content generation via Google Translate API integration
 - Create AI-powered image generation with Stable Diffusion and nine aesthetic styles
 - Architect complete AWS cloud infrastructure (API Gateway, ECS, Lambda, S3, SageMaker)
 - Implement automated internal linking system with custom SEO strategies
 - Build scalable API systems supporting bulk content generation
- **Technical challenges faced:**
 - Single-handedly architecting enterprise-scale system handling 10,000+ articles monthly
 - Integrating multiple AI models (text, image, translation) in cohesive automated pipeline
 - Implementing dynamic internal linking algorithms for improved search engine rankings
 - Building cost-effective cloud infrastructure supporting massive content generation volumes
- **Business/stakeholder challenges:**
 - Delivering complete technical solution as solo developer with no technical team support
 - Balancing system complexity with rapid development timeline constraints
 - Ensuring enterprise-grade reliability and performance with limited resources
 - Managing client requirements and preventing scope creep through technical documentation
- **Success criteria and constraints:** 10,000+ monthly articles, 50+ language support, enterprise reliability, solo development, cost-effective scaling

A - ACTION (Detailed Breakdown) Action Summary: I architected and built a complete AI automation ecosystem single-handedly, integrating multiple state-of-the-art models with sophisticated

cloud infrastructure, achieving enterprise-scale content generation capabilities that secured significant quarterly contracts.

Technical Stack Used:

- **AI Models:** GPT-3.5, BERT, FLAN-T5 (content generation), Stable Diffusion (image generation), Google Translate API (multi-language support)
- **Cloud Infrastructure:** AWS API Gateway, ECS, Lambda, SQS, SNS, EC2, S3, SageMaker (complete serverless architecture for scalability)
- **Backend Development:** Flask, FastAPI (high-performance API development for bulk processing)
- **Database & Storage:** SQL, SQLite, DynamoDB, S3 (structured data management and content storage)
- **Deployment:** Docker containerization (efficient cloud deployment and scaling)

Collaboration Details:

- **Worked with:** Frontend developers (API integration), business stakeholders (requirements), content strategists (SEO optimization)
- **How decisions were made:** Single-handedly made all technical architecture decisions including cloud services, AI model selection, and system design
- **Communication challenges/successes:** Created comprehensive Software Requirement Specification (SRS) documenting technical capabilities and limitations
- **Cross-functional coordination:** Balanced technical implementation with business requirements while managing all backend development independently

Leadership Moments:

- **Decisions you drove:** Solo Technical Leadership taking complete ownership of technical architecture decisions as sole developer
- **People you influenced/mentored:** Innovation driver designing novel graph-based AI linking models for dynamic internal linking optimization
- **Processes you improved:** Established multi-step AI refinement pipeline ensuring quality at massive scale
- **Initiative you took beyond assigned tasks:** Created comprehensive documentation and quality assurance systems beyond basic requirements

Problem-Solving Process:

- **Scale vs Quality:** Implemented multi-step AI refinement and NLP filtering maintaining content quality across 10,000+ monthly articles
- **Integration Complexity:** Single-handedly integrated text generation, image creation, translation, and SEO optimization in unified pipeline
- **Cost Optimization:** Architected serverless AWS infrastructure minimizing operational costs while supporting enterprise volumes
- **Alternative Solutions Considered:** Evaluated monolithic vs. microservices, various AI model combinations, different cloud providers

R - RESULTS Result Summary: As sole technical developer, I delivered an enterprise-grade content automation system generating 10,000+ monthly articles, securing \$50-100K quarterly contracts, and achieving significant improvements in organic traffic and international engagement.

Detailed Impact:

- **Quantified business impact:**
 - **10,000+ articles/month** automated generation capability built by single developer
 - **\$50-100K quarterly contracts** secured directly through superior technical capabilities
 - **60% increase** in organic traffic through AI-powered internal linking algorithms
 - **40% increase** in international engagement via multi-language content generation
- **Technical achievements:**
 - **Complete system architecture:** Single-handedly built entire AI automation pipeline from AI models to cloud deployment
 - **Multi-language scaling:** Successfully implemented Google Translate API supporting 50+ languages
 - **Advanced AI integration:** Combined GPT-3.5, BERT, FLAN-T5, and Stable Diffusion in unified content generation system
 - **Enterprise cloud infrastructure:** Architected scalable AWS ecosystem supporting massive content volumes
- **Team/process improvements:**
 - **Solo development excellence:** Demonstrated ability to deliver enterprise-scale solutions as individual contributor
 - **Technical documentation standards:** Created comprehensive SRS preventing scope creep and ensuring system maintainability
 - **Automated quality assurance:** Implemented plagiarism checks and grammar validation ensuring content credibility
 - **Scalable architecture patterns:** Established cloud-native design principles for future AI-intensive projects
- **Personal growth/skills gained:**
 - **Full-stack AI system architecture:** Complete expertise in designing and implementing enterprise AI solutions independently
 - **Cloud infrastructure mastery:** Advanced skills in AWS services orchestration for AI-intensive applications
 - **Solo project management:** Proven ability to deliver complex technical solutions with complete ownership and accountability
 - **AI model integration expertise:** Deep understanding of combining multiple AI technologies in production-ready systems

Project Information Header

- **Project:** Sky Scraper - AI-Powered Chrome Extension for Scalable Content Automation
- **Company:** SEOContent.ai
- **Role:** AI Infrastructure Engineer
- **Duration:** 6 Months

Summary STAR (1.5-2.5 minutes speaking time)

S: SEOContent.ai clients were stuck spending hours on manual keyword research and article structuring, which prevented them from building the large-scale SEO content campaigns they needed to compete effectively in their markets.

T: My job was to build an AI-powered Chrome extension that could automate the entire content creation workflow - from keyword research to generating up to 15 interconnected articles per topic cluster - while maintaining SEO quality standards across a 6-month timeline.

A: I designed a microservices architecture on AWS using fine-tuned BERT models for keyword extraction and GPT models for content generation. The system combined web scraping through Chrome extension, NLP processing for topic clustering, and parallel processing workflows that could handle high-volume content creation.

R: We slashed content creation time by 60-70% (from 4 hours down to just 10 minutes plus editing), successfully launched the beta with contract renewals, and clients started seeing measurable improvements in their organic traffic using our automated content clusters.

Detailed STAR

S - SITUATION Situation Summary: SEOContent.ai needed to solve the scalability challenge in content marketing where clients required extensive multi-article campaigns but faced hours of manual processes, preventing them from building comprehensive SEO content strategies at the scale needed for competitive advantage.

Detailed Context:

- **Company context & business problem:** SEOContent.ai needed to solve the scalability challenge in content marketing where clients required extensive multi-article campaigns but faced hours of manual keyword research and article structuring. Traditional content creation processes were disconnected and inefficient, preventing clients from building comprehensive SEO content strategies at scale.
- **Team structure & your role:** Cross-functional Agile team including product managers, frontend developers, DevOps engineers, and SEO specialists. As AI Infrastructure Engineer, I was responsible for designing system architecture components, implementing generative AI modules, and ensuring pipeline reliability under heavy usage across the entire content automation workflow.
- **Timeline & constraints:** 6-month timeline from initial design to beta launch. High-volume processing requirements with need for cost-effective scalability. Critical business requirement as clients demanded large-scale SEO content production capabilities for competitive advantage.
- **Scale/scope of the challenge:** Building comprehensive content automation system, generating 15 interconnected articles per cluster, enterprise-grade reliability requirements

T - TASK/CHALLENGE Task Summary: I was tasked with building a comprehensive AI-powered content automation system that could transform manual content marketing workflows into scalable, automated pipelines generating up to 15 interconnected articles per topic cluster while maintaining SEO best practices.

Detailed Responsibilities:

- **Your specific responsibilities:**
 - Design and implement robust web scraping functionality through Chrome Extension integration

- Build NLP-based keyword extraction system using fine-tuned BERT models for topic clustering
- Develop multi-article generation workflow with LLM integration (GPT-3.5, GPT-NeoX)
- Implement AWS microservices architecture for elastic scaling and cost optimization
- Create content assembly and publishing pipeline supporting WordPress integration
- Build parallel processing capabilities to accelerate content generation workflows
- **Technical challenges faced:**
 - Maintaining consistent AI-generated content quality following brand tone and SEO guidelines
 - Balancing performance vs. cost when generating multiple long-form articles simultaneously
 - Handling evolving SEO practices requiring system flexibility and adaptability
 - Implementing reliable scraping functionality resistant to website changes and anti-bot measures
- **Business/stakeholder challenges:**
 - Cross-functional coordination between marketing, engineering, and design teams
 - Translating complex technical AI capabilities into user-friendly features
 - Meeting diverse client requirements for content volume, quality, and SEO optimization
 - Ensuring system reliability during high-volume processing for enterprise clients
- **Success criteria and constraints:** 6-month delivery, 15 articles per cluster, 60%+ time reduction, enterprise reliability, cost-effective scaling

A - ACTION (Detailed Breakdown) Action Summary: I architected a comprehensive microservices system using AWS infrastructure, implemented advanced NLP processing for content automation, and built scalable parallel processing capabilities that reduced content creation time from 4 hours to 10 minutes while maintaining SEO quality standards.

Technical Stack Used:

- **Python + JavaScript + TypeScript:** Core development languages (chosen for AI/ML libraries and web scraping capabilities)
- **Node.js + Express:** Chrome Extension backend integration (seamless browser-to-cloud data pipeline)
- **AWS ECS + Lambda:** Microservices orchestration (containerized scaling for different processing functions)
- **Fine-tuned BERT Models:** Keyword extraction and topic clustering (superior performance for content analysis)
- **GPT-3.5 + GPT-NeoX:** Content generation through load-balanced microservices (high-quality article creation)
- **AWS S3 + DynamoDB:** Data storage and metadata logging (cost-effective scalable storage)
- **Docker:** Containerization strategy (modular design improving scalability and maintainability)

Collaboration Details:

- **Worked with:** Product managers (requirements), frontend developers (integration), DevOps (infrastructure), SEO specialists (optimization guidelines)
- **How decisions were made:** Participated in Agile sprint planning with bi-weekly reviews, contributed technical insights during prototype evaluations

- **Communication challenges/successes:** Led technical documentation sessions helping non-technical teams understand AI generation processes and architecture decisions
- **Cross-functional coordination:** Balanced technical architecture with product requirements and user experience across marketing, engineering, and design teams

Leadership Moments:

- **Decisions you drove:** Implemented container-based AWS ECS strategy with horizontal scaling solving performance bottlenecks under high load
- **People you influenced/mentored:** Technical innovation guiding team toward microservices architecture that became company standard
- **Processes you improved:** Created centralized SEO knowledge base enabling quick adaptation to changing SEO standards and client requirements
- **Initiative you took beyond assigned tasks:** Developed dynamic prompt management system automatically adjusting content generation based on user preferences and feedback

Problem-Solving Process:

- **Performance Optimization:** Evaluated single large model vs. multiple smaller instances, chose parallel processing approach balancing cost with generation speed
- **Quality Assurance:** Implemented custom prompt templates with feedback loops ensuring content cohesion and brand consistency
- **Scalability Challenge:** Designed modular microservices architecture allowing independent scaling of scraping, extraction, and generation components
- **Alternative Solutions Considered:** Monolithic vs. microservices architecture, cloud vs. hybrid processing, synchronous vs. asynchronous content generation

R - RESULTS Result Summary: The Skyscraper platform achieved remarkable productivity improvements with 60-70% reduction in content creation time, successful beta launch with contract renewals, and established new architecture standards for SaaS AI products while demonstrating measurable organic traffic improvements for clients.

Detailed Impact:

- **Quantified business impact:**
 - **60-70% reduction** in manual content creation time (4 hours to 10 minutes + 1 hour editing)
 - **15 interconnected articles** generated per topic cluster with automated SEO optimization
 - **Contract renewals and expansions** from beta testers demonstrating strong product-market fit
 - **Measurable organic traffic improvements** reported by users after adopting Skyscraper platform
- **Technical achievements:**
 - **Scalable microservices architecture:** Successfully handled high-volume processing with elastic scaling
 - **Advanced NLP pipeline:** Implemented BERT-based keyword extraction with intelligent topic clustering
 - **Multi-model integration:** Built load-balanced system supporting GPT-3.5 and GPT-NeoX for diverse content needs

- **End-to-end automation:** Created complete pipeline from web scraping to WordPress publishing with JSON/CSV export capabilities
- **Team/process improvements:**
 - **Cross-functional collaboration:** Established effective communication patterns between technical and non-technical teams
 - **Architecture influence:** Project's microservices approach became standard for future SaaS AI offerings
 - **Technical documentation:** Created comprehensive guides enabling knowledge transfer and system maintenance
 - **User-centric development:** Implemented early user testing identifying prompt-engine issues before MVP finalization
- **Personal growth/skills gained:**
 - **Container orchestration expertise:** Advanced skills in Docker and AWS ECS for AI workload management
 - **NLP prompt engineering:** Deep understanding of LLM optimization for consistent content quality
 - **Multi-service architecture:** Experience designing and implementing complex AI pipeline systems
 - **Cross-functional leadership:** Enhanced ability to translate technical capabilities into business value and user requirements

Project Information Header

- **Project:** Zero-Maintenance AI Tools Platform with Dynamic Configuration System
- **Company:** SEOContent.ai
- **Role:** AI Infrastructure Engineer
- **Duration:** 3 Weeks

Summary STAR (1.5-2.5 minutes speaking time)

S: SEOContent.ai wanted to build a platform with 50+ AI tools, but they knew from experience that having developers maintain every tool update would create massive bottlenecks - content experts would constantly need tech help for simple changes.

T: My job was to completely flip this model - build a platform where content experts could manage, update, and deploy tools themselves without any technical knowledge, all while keeping costs low

A: I created this hybrid system using Google Sheets (since everyone knows how to use spreadsheets) connected to AWS serverless infrastructure. The frontend dynamically reads JSON configs, so when someone updates a sheet, the tool changes instantly - no code deployment needed. We delivered it in just 3 weeks.

R: The results blew everyone away - we got 5,000 users in the first month, cut costs by 98.4% compared to no-code platforms, and tool updates now take under 90 seconds instead of waiting for developers.

Detailed STAR

S - SITUATION Situation Summary: SEOContent.ai needed to build a comprehensive AI tools platform with 50+ tools but faced a critical strategic challenge where traditional approaches would create unsustainable developer dependencies, limiting business agility and creating operational bottlenecks for tool maintenance.

Detailed Context:

- **Company context & business problem:** SEOContent.ai needed to build a comprehensive AI tools platform with 50+ tools but faced a critical strategic challenge: traditional approaches would create unsustainable developer dependencies for tool maintenance and updates. The company anticipated that content experts would identify improvements but would depend on developers to implement them, creating operational bottlenecks and limiting business agility.
- **Team structure & your role:** Small startup team with content experts, frontend developers, and documentation specialists. As the sole AI Infrastructure Engineer, I was responsible for architecting and building the entire platform from scratch, specifically designing for zero ongoing developer maintenance.
- **Timeline & constraints:** Aggressive 3-week timeline to deliver production-ready platform. Need to support immediate scaling to 50+ tools with potential expansion to 100+. Critical requirement: system must operate independently without developer intervention for tool modifications and updates.
- **Scale/scope of the challenge:** Building 50+ tool platform from scratch, eliminating developer dependencies entirely, achieving dramatic cost reduction, delivering in 3 weeks

T - TASK/CHALLENGE Task Summary: I was tasked with architecting a revolutionary AI tools platform that could eliminate developer dependencies entirely, enabling content experts to manage, modify, and deploy tools independently while maintaining enterprise-grade performance and cost efficiency.

Detailed Responsibilities:

- **Your specific responsibilities:**
 - Design architecture eliminating developer maintenance requirements for tool lifecycle management
 - Build dynamic frontend system supporting unlimited tool configurations without code changes
 - Implement real-time deployment pipeline enabling instant tool updates and modifications
 - Create cost-effective infrastructure supporting 50+ tools with linear scaling capabilities
 - Establish user-friendly interface requiring zero technical training for content experts
- **Technical challenges faced:**
 - Designing dynamic frontend rendering system supporting unlimited tool variations
 - Implementing real-time configuration deployment without application restarts
 - Building cost-effective architecture competing with \$2000+/month no-code alternatives
 - Creating validation and error handling for non-technical users managing complex configurations
- **Business/stakeholder challenges:**
 - Rapid 3-week delivery timeline requiring immediate architecture decisions

- Balancing technical sophistication with user simplicity for content experts
- Achieving dramatic cost reduction while maintaining enterprise-grade functionality
- Ensuring sustainable competitive advantages through operational independence
- **Success criteria and constraints:** Zero developer maintenance, 3-week delivery, 50+ tools support, massive cost reduction, non-technical user operation

A - ACTION (Detailed Breakdown) Action Summary: I architected an innovative hybrid system combining Google Sheets for familiar user interface with AWS serverless infrastructure, implementing dynamic configuration rendering that eliminated developer dependencies while achieving 98.4% cost reduction compared to traditional alternatives.

Technical Stack Used:

- **Google Sheets + Apps Script:** Content management and change detection (chosen for zero learning curve and built-in collaboration)
- **AWS Lambda:** Configuration processing and deployment automation (serverless pay-per-execution model for cost efficiency)
- **Amazon S3 + CloudFront:** JSON configuration storage and global CDN distribution (instant updates without app deployments)
- **Dynamic JavaScript Frontend:** Runtime configuration rendering (supporting unlimited tool variations without code changes)
- **DynamoDB:** Metadata tracking and analytics (cost-effective NoSQL for tool usage metrics)

Collaboration Details:

- **Worked with:** Content experts (tool requirements), frontend developers (integration), documentation team (user guides)
- **How decisions were made:** Led comprehensive architecture evaluation comparing 4 different approaches considering cost, complexity, and scalability
- **Communication challenges/successes:** Facilitated strategic planning sessions anticipating operational challenges before implementation
- **Cross-functional coordination:** Balanced technical architecture decisions with business operational requirements and user experience needs

Leadership Moments:

- **Decisions you drove:** Chose unconventional Google Sheets approach over traditional CMS solutions, resulting in 98.4% cost reduction
- **People you influenced/mentored:** Strategic innovation guiding team toward configuration-driven architecture enabling business user autonomy
- **Processes you improved:** Established real-time deployment pipeline allowing tool updates in under 90 seconds
- **Initiative you took beyond assigned tasks:** Designed configuration-driven architecture enabling 50x faster iteration cycles than traditional development

Problem-Solving Process:

- **Architecture Evaluation:** Systematically compared Traditional CMS (\$500/month), Custom Admin (\$200/month), No-Code Platform (\$2000/month) vs. Hybrid Solution (\$50/month)
- **Dynamic Rendering Challenge:** Built frontend system reading JSON configurations at runtime, supporting unlimited field types and variations

- **Cost Optimization:** Implemented serverless architecture with intelligent caching reducing operational costs by 98.4%
- **Alternative Solutions Considered:** Evaluated WordPress/Drupal, React admin panels, Zapier/Bubble platforms before choosing hybrid approach

R - RESULTS Result Summary: The platform achieved exceptional market success with 5,000 users in the first month, 1,600 recurring users, and 98.4% cost reduction while establishing sustainable competitive advantages through complete operational independence and 50x faster iteration cycles.

Detailed Impact:

- **Quantified business impact:**
 - **5,000 users** in first month exceeding all initial adoption projections
 - **1,600 recurring users** in month 2 (32% retention rate) demonstrating strong product-market fit
 - **98.4% cost reduction** (\$50/month vs. \$2000+/month no-code alternatives)
 - **50x faster** tool improvement cycles enabling same-day market responsiveness
- **Technical achievements:**
 - **Real-time deployment:** Tool updates deployed in under 90 seconds without application restarts
 - **Zero developer maintenance:** 100% content expert autonomy for tool lifecycle management
 - **Dynamic configuration system:** Supporting unlimited tool variations through runtime JSON rendering
 - **Enterprise-grade performance:** 99.95% uptime with <800ms frontend load times via CDN optimization
- **Team/process improvements:**
 - **Operational independence:** Eliminated entire category of developer maintenance overhead
 - **Business agility:** Enabled same-day implementation of user feedback and market changes
 - **Cost efficiency standards:** Established architecture patterns achieving 98%+ cost savings for future projects
 - **Developer focus:** Freed 95% of development time for core platform innovation vs. maintenance
- **Personal growth/skills gained:**
 - **Strategic architecture evaluation:** Expertise in comparing technical approaches considering long-term operational costs
 - **Configuration-driven design:** Advanced skills in building systems enabling business user autonomy
 - **Cost optimization mastery:** Deep understanding of serverless architecture for dramatic cost reduction
 - **Market validation:** Experience building and launching products achieving immediate product-market fit

Project Information Header

- **Project:** AI-Powered Content Generation API for SEO Marketing
- **Company:** SEOContent.ai
- **Role:** AI Infrastructure Engineer
- **Duration:** 3 Weeks

Summary STAR (1.5-2.5 minutes speaking time)

S: SEOContent.ai needed to commercialize their existing White Label Resell AI content generation system by building a market-ready API for external digital marketing clients who would directly like to integrate our Content generation system into their solutions.

T: I was tasked with architecting a complete API system using AWS API Gateway that would securely expose our existing AI capabilities (GPT-3.5, BERT, FLAN-T5, Stable Diffusion), implement enterprise authentication and billing, and handle thousands of concurrent client requests within an aggressive 3-week timeline.

A: I built a comprehensive REST API using AWS API Gateway with Lambda functions, integrated it with existing White Label Resell infrastructure, implemented AWS Cognito + Stripe for authentication and billing, and optimized performance through asynchronous processing patterns.

R: Delivered a production-ready API reducing content creation time by ~85% (2.5 minutes to 45 seconds), enabled processing of 10,000+ monthly requests with 99.9% uptime, and secured significant quarterly contracts while achieving 90% successful client integrations.

Detailed STAR

S - SITUATION

Situation Summary: SEOContent.ai needed to quickly commercialize the existing White Label Resell content generation system by building a market-ready API that would allow external clients to access our proven AI capabilities through a scalable, secure interface.

Detailed Context:

- **Company context & business problem:** SEOContent.ai, a subsidiary of White Label Resell, needed to monetize our existing AI-powered content automation system by creating an external API product. Digital marketing clients were struggling with slow, expensive manual content creation and needed access to our proven system that could generate 10,000+ articles monthly with multi-language support.
- **Team structure & your role:** Small startup team with frontend developers and business stakeholders. As the AI Infrastructure Engineer, I was responsible for building the complete API layer that would expose White Label Resell's existing AI capabilities to external clients.

- **Timeline & constraints:** Aggressive 3-week timeline to deliver production-ready API. Existing White Label Resell system provided the AI foundation, but required complete API architecture design and secure external access implementation.
- **Scale/scope of the challenge:** Building enterprise-grade API gateway, implementing secure authentication and billing, integrating with existing complex AI pipeline

T - TASK/CHALLENGE

Task Summary: I was tasked with building a complete API system using AWS API Gateway that would securely expose our existing White Label Resell AI capabilities, implement enterprise authentication and billing, and handle thousands of concurrent client requests.

Detailed Responsibilities:

- **API Architecture & Development:** Design and implement REST API using AWS API Gateway with proper versioning, rate limiting, and comprehensive documentation
- **System Integration:** Connect new API layer with existing White Label Resell AI infrastructure (GPT-3.5, BERT, FLAN-T5, Stable Diffusion, Google Translate)
- **Security & Authentication:** Implement secure API key management using AWS Cognito with Stripe integration for usage-based billing
- **Performance Optimization:** Ensure API can handle thousands of concurrent requests while leveraging existing AI pipeline efficiently
- **Client Onboarding:** Create comprehensive API documentation, SDKs, and integration guides for seamless client adoption

Technical challenges faced:

- **Legacy Integration:** Connecting new API Gateway with existing White Label Resell architecture without disrupting internal operations
- **Performance Optimization:** Reducing response times from existing 2.5-minute internal process to target 45-second API response
- **Request Management:** Implementing intelligent queuing and load balancing for high-volume API requests
- **Multi-Model Coordination:** Orchestrating complex AI pipeline (text generation, image creation, translation) through simple API endpoints

Business/stakeholder challenges:

- **Rapid Timeline:** Delivering complete commercial API product in only 3 weeks
- **Revenue Model:** Creating pricing tiers and usage tracking that reflects infrastructure costs and profit margins
- **Client Success:** Ensuring smooth onboarding experience for external clients transitioning from manual processes

A - ACTION (*Detailed Breakdown*)

Action Summary: I architected a comprehensive API system using AWS API Gateway that seamlessly integrated with our existing White Label Resell infrastructure, implementing intelligent caching and optimization strategies that dramatically improved response times while maintaining enterprise-grade reliability.

Technical Stack Used:

- **AWS API Gateway:** Primary API management platform (chosen for built-in rate limiting, caching, authentication, and monitoring)
- **AWS Lambda:** Serverless functions for API business logic and White Label Resell system integration
- **Integration Layer:** Custom Python middleware connecting to existing White Label Resell AI pipeline (GPT-3.5, BERT, FLAN-T5, Stable Diffusion, Google Translate)
- **AWS Cognito + Stripe:** API key management and credits-based billing system
- **CloudWatch & SNS:** Comprehensive API monitoring, logging, and alerting

Collaboration Details:

- **White Label Resell Team:** Coordinated with original system architects to understand existing AI pipeline and integration points
- **Frontend Developers:** Provided API specifications, SDKs, and authentication integration guidance
- **Business Team:** Collaborated on pricing strategy, usage limits, and billing implementation requirements
- **Documentation Team:** Created comprehensive API guides, code examples, and troubleshooting resources

Leadership Moments:

- **API Design Strategy:** Led architectural decisions on REST endpoint design, choosing simplicity for broad client compatibility
- **Integration Approach:** Drove decision to build API layer on top of existing White Label Resell system rather than rebuilding AI capabilities
- **Performance Standards:** Established API response time SLAs and comprehensive monitoring protocols
- **Client Success Framework:** Created technical onboarding process and support documentation for API adopters

Problem-Solving Process:

- **Latency Optimization:** Implemented API Gateway caching and Lambda warming strategies, reducing cold starts and improving response times by 85%
- **System Integration:** Developed asynchronous processing patterns to leverage existing White Label Resell AI pipeline without blocking API responses
- **Load Management:** Built intelligent queuing system handling burst traffic while maintaining fair usage across clients
- **Error Handling:** Created comprehensive error response system with proper HTTP status codes and detailed debugging information

R - RESULTS

Result Summary: The API successfully transformed our White Label Resell capabilities into a commercial product, achieving exceptional performance with 85% faster content generation, serving 10,000+ monthly requests, and enabling seamless client integration while maintaining the reliability of our existing AI infrastructure.

Detailed Impact: Quantified business impact:

- **API Performance:** Reduced average response time from 2.5 minutes to 45 seconds (85% improvement)

- **Commercial Success:** Enabled processing of 50,000+ API requests monthly with 99.9% uptime
- **Client Results:** API users achieved 300% increase in organic traffic through automated, SEO-optimized content
- **Revenue Generation:** Secured significant quarterly contracts through superior API capabilities and performance

Technical achievements:

- **Scalable API Architecture:** Successfully handled thousands of concurrent requests through AWS API Gateway auto-scaling
- **Seamless Integration:** Zero-downtime integration with existing White Label Resell AI infrastructure
- **Enterprise Security:** Implemented robust authentication, rate limiting, and comprehensive usage monitoring
- **Developer Experience:** Created intuitive API with comprehensive documentation resulting in 90% successful client integrations

Team/process improvements:

- **Reusable Architecture:** Established API development patterns and standards for future product development
- **Monitoring Excellence:** Implemented comprehensive API analytics and performance dashboards
- **Client Support Process:** Created streamlined technical onboarding reducing support requirements by 70%
- **Integration Framework:** Developed reusable patterns for exposing internal AI capabilities as commercial APIs

Personal growth/skills gained:

- **API Product Development:** End-to-end experience building commercial APIs from existing AI systems using AWS API Gateway
- **Rapid Development:** Proven ability to deliver enterprise-grade API products under aggressive 3-week timeline
- **System Integration Mastery:** Advanced skills in connecting API layers with complex existing AI infrastructure
- **Commercial Product Strategy:** Experience translating internal AI capabilities into market-ready products with proper pricing and scaling

Project Information Header

- **Project:** Medical Image Data Engineering Pipeline for Pathology Workflows
- **Company:** Digital Clinics Research and Services
- **Role:** Data Scientist
- **Duration:** 12 Months

Summary STAR (1.5-2.5 minutes speaking time)

S: Digital Clinics was developing a comprehensive medical image analysis system for cancer diagnosis, requiring a scalable data engineering pipeline to process massive 10-20GB TIFF/SVS medical imaging files that pathologists couldn't efficiently handle manually, creating bottlenecks in diagnosis workflows.

T: I was tasked with engineering a complete automated data processing infrastructure that could break down massive medical imaging files into manageable formats, create efficient annotation workflows for junior pathologists, and build seamless deployment systems integrating with our CADD4MBC clinical platform.

A: I built a robust Python-based pipeline using OpenCV for automated file processing, integrated QuPath with custom Groovy scripts for annotation workflows, and created deployment systems connecting junior pathologists, senior doctors, and our clinical platform with MongoDB for metadata tracking.

R: Achieved 1.5TB weekly data throughput with zero data loss, reduced manual processing time by 60%, enabled processing of 100,000+ images monthly, and established scalable workflows supporting both pathologist training and downstream AI projects.

Detailed STAR

S - SITUATION Situation Summary: Digital Clinics Research and Services was facing major bottlenecks in medical image analysis where pathologists couldn't efficiently work with massive 10-15GB medical imaging files, creating delays in cancer diagnosis workflows and preventing scalable annotation processes.

Detailed Context:

- **Company context & business problem:** Digital Clinics Research and Services needed to revolutionize medical image analysis workflows where the data pipeline flow was manual either done by junior pathologist or developer. The existing workflow was time consuming and error prone, creating bottlenecks in cancer diagnosis pipelines.
- **Team structure & your role:** Medical AI research team collaborating with pathologists from Basavatarakam Indo-American Cancer Hospital, Hyderabad. As Data Scientist, I was the lead data engineer responsible for designing and implementing the entire automated data pipeline architecture from raw medical images to production-ready clinical workflows.
- **Timeline & constraints:** Built and maintained it over my 12-month timeline processing 1.5TB of medical imaging data weekly. Need for seamless integration with existing clinical workflows without disruption. Strict data integrity requirements for medical-grade applications.
- **Scale/scope of the challenge:** Processing 10-15GB individual TIFF files daily, supporting multiple image formats, ensuring zero data loss across the entire pipeline

T - TASK/CHALLENGE Task Summary: I was responsible for engineering a complete data processing infrastructure that could break down massive medical imaging files, create efficient annotation workflows for junior pathologists, and build seamless deployment systems to our clinical platform.

Detailed Responsibilities:

- **Your specific responsibilities:**

- Design and implement automated pipeline to break down 10-15GB TIFF and various medical image formats
- Engineer annotation workflow systems for junior pathologists
- Build deployment scripts for automatic integration with CADD4MBC backend
- Establish data quality control and validation processes
- Design scalable architecture supporting multiple medical image formats
- **Technical challenges faced:**
 - Handling multiple medical image formats (TIFF, SVS, etc.) with varying structures
 - Ensuring zero data loss during file processing and transformation
 - Managing memory efficiently while processing 10-15GB files
 - Creating seamless integration between annotation tools and clinical platform
 - Maintaining data integrity across multi-stage workflows
- **Business/stakeholder challenges:**
 - Integrating new automated systems with existing clinical workflows
 - Training junior pathologists on new annotation processes
 - Ensuring senior pathologists could efficiently review and provide feedback
 - Managing data privacy and security for sensitive medical imaging data
- **Success criteria and constraints:** Zero data loss, seamless clinical workflow integration, 1.5TB weekly processing capacity, medical-grade data security

A - ACTION (Detailed Breakdown) Action Summary: I built a robust data engineering pipeline using custom Python scripts for file processing, created automated annotation workflows with quality controls, and engineered seamless deployment systems that connected junior pathologists, senior doctors, and our clinical platform.

Technical Stack Used:

- **Python + OpenCV + NumPy:** Core image processing pipeline (chosen for robust medical image format support and memory management)
- **Custom Python Scripts:** File breakdown and processing automation (designed for handling various medical image formats)
- **QuPath + Custom Groovy Scripts:** Annotation workflow integration (selected for pathologist familiarity and clinical workflow compatibility)
- **CADD4MBC Platform Integration:** Backend deployment system (existing clinical platform requiring seamless data flow)
- **MongoDB:** Metadata and annotation tracking (reliable data storage for clinical environments)
- **Flask/FastAPI:** API development for workflow automation (lightweight deployment and integration capabilities)

Collaboration Details:

- **Worked with:** Junior pathologists (workflow training), senior pathologists (quality review), clinical researchers (requirement validation), IT staff (platform integration)
- **How decisions were made:** Weekly feedback sessions with pathologists to refine workflows and ensure clinical usability
- **Communication challenges/successes:** Bridged technical implementation with clinical requirements through regular stakeholder reviews
- **Cross-functional coordination:** Coordinated between technical development, clinical workflows, and hospital IT systems

Leadership Moments:

- **Decisions you drove:** Architected the entire data pipeline infrastructure choosing optimal file processing methods and workflow designs
- **People you influenced/mentored:** Trained junior pathologists on new annotation workflows and best practices
- **Processes you improved:** Transformed manual file handling into automated pipeline reducing processing time by 60%
- **Initiative you took beyond assigned tasks:** Created comprehensive documentation and training materials for clinical staff

Problem-Solving Process:

- **Approach to technical challenges:** Implemented chunked file processing for memory management and created modular architecture for different image formats
- **Alternative solutions considered:** Evaluated cloud vs on-premise processing, batch vs real-time processing, custom vs existing annotation tools
- **Trade-offs made and reasoning:** Chose on-premise solution for medical data security, prioritized reliability over speed for clinical safety
- **How you validated your approach:** Conducted extensive testing with real clinical data and gathered continuous feedback from pathologists

R - RESULTS Result Summary: The data engineering pipeline achieved remarkable scale processing 1.5TB weekly with zero data loss, reduced manual processing time by 60%, and enabled seamless workflows that supported both junior pathologist training and senior doctor review processes.

Detailed Impact:

- **Quantified business impact:**
 - **1.5TB weekly** data throughput with zero data loss
 - **60% reduction** in manual file processing time
 - **100% automation** of file breakdown and deployment processes
 - Enabled processing of **100,000+ images monthly** for downstream AI projects
- **Technical achievements:**
 - Successfully engineered pipeline handling multiple medical image formats (TIFF, SVS, etc.)
 - Built robust annotation workflow supporting multi-level review processes
 - Created seamless CADD4MBC integration with automated quality controls
 - Established scalable architecture supporting future expansion
- **Team/process improvements:**
 - **Established standardized workflows** for medical image processing reducing training time
 - **Created documentation standards** for clinical data engineering projects
 - **Improved cross-functional collaboration** between technical and clinical teams
 - **Enabled downstream AI projects** by providing consistent, high-quality annotated datasets
- **Personal growth/skills gained:**

- **Large-scale data pipeline engineering** expertise in medical imaging domain
- **Clinical workflow integration** skills bridging technical and medical requirements
- **Cross-functional leadership** coordinating between technical teams and medical professionals
- **Medical data compliance** understanding of healthcare data security and privacy requirements

Project Information Header

- **Project:** Tubule Detection using Distributed Deep Learning for Breast Cancer Grading
- **Company:** Digital Clinics Research and Services
- **Role:** Data Scientist
- **Duration:** 3 Months (within the 12-month overall timeline)

Summary STAR (1.5-2.5 minutes speaking time)

S: Digital Clinics was developing a breast cancer diagnosis system using the Nottingham Grading System, requiring an automated tubule detection ML pipeline to address time-consuming manual processes and high inter-observer variability among pathologists.

T: We were tasked with developing a deep learning model achieving medical-grade accuracy (>90%) for automatic tubule segmentation from whole slide images, while optimizing training efficiency.

A: I fine-tuned an R2U-Net architecture with Horovod distributed training across 4 GPUs, leveraging my previously built annotated dataset pipeline for model development and validation.

R: Achieved 92% F1-score accuracy, reduced training time by 3.9x (27→7 hours), and presented findings at ~~SIIM-CMIM122~~ International Conference at Johns Hopkins Hospital.

Detailed STAR

S - SITUATION Situation Summary: Digital Clinics Research and Services needed an automated solution for tubule detection in breast cancer images to support the Nottingham Grading System, as manual identification was time-consuming and had high inter-observer variability between pathologists.

Detailed Context:

- **Company context & business problem:** Digital Clinics Research and Services needed to automate tubule identification, a critical component of the Nottingham Grading System for breast cancer diagnosis. Manual tubule detection was creating bottlenecks in cancer grading workflows and suffered from inconsistency between pathologists.
- **Team structure & your role:** Medical AI research team collaborating with pathologists from Basavarakam Indo-American Cancer Hospital. As Data Scientist, I was the lead AI engineer responsible for developing the deep learning model architecture and implementing distributed training infrastructure.
- **Timeline & constraints:** 6-month development timeline within the larger 12-month project. Need for high accuracy model suitable for clinical deployment. Computational constraints requiring efficient training methods for large datasets.

- **Scale/scope of the challenge:** Processing annotated tubule dataset from my data engineering pipeline, developing production-ready AI model, achieving medical-grade accuracy standards

T - TASK/CHALLENGE Task Summary: I was responsible for developing a deep learning model that could automatically segment tubules from breast cancer whole slide images with medical-grade accuracy, while implementing distributed training to handle the computational demands efficiently.

Detailed Responsibilities:

- **Your specific responsibilities:**
 - Develop R2U-Net deep learning architecture for tubule segmentation
 - Implement distributed training infrastructure using Horovod framework
 - Optimize model performance for clinical accuracy requirements
 - Process and utilize annotated dataset from my data engineering pipeline
 - Validate model performance against pathologist annotations
 - Prepare research findings for publication and conference presentation
- **Technical challenges faced:**
 - Achieving high segmentation accuracy for complex tubule structures
 - Implementing distributed training to reduce computational time from 27 hours
 - Managing GPU memory efficiently during large-scale model training
 - Ensuring model generalization across different tissue samples and imaging conditions
- **Business/stakeholder challenges:**
 - Meeting medical-grade accuracy standards for clinical deployment
 - Balancing model complexity with training efficiency
 - Validating results with pathologists for clinical acceptance
 - Preparing research-quality documentation for academic publication
- **Success criteria and constraints:** >90% accuracy, <8 hours training time, medical-grade precision, research publication readiness

A - ACTION (Detailed Breakdown) Action Summary: I developed an R2U-Net architecture optimized for tubule segmentation, implemented Horovod distributed training achieving 3.9x speedup, and created a robust validation framework that resulted in research publication and conference presentation.

Technical Stack Used:

- **R2U-Net Architecture:** Deep learning model (chosen for superior segmentation performance combining U-Net with recurrent and residual connections)
- **Horovod + NVIDIA DGX:** Distributed training framework (selected for 4-GPU parallel training achieving 3.9x performance improvement)
- **Python + PyTorch:** Model development and training (robust deep learning ecosystem with distributed training support)
- **OpenCV + NumPy:** Image preprocessing and postprocessing (medical image format handling and augmentation)
- **CADD4MBC Integration:** Annotation data utilization (leveraging dataset from my data engineering pipeline)

- **Custom Groovy Scripts:** Data preprocessing automation (seamless integration with existing QuPath workflows)

Collaboration Details:

- **Worked with:** Pathologists from Basavatarakam Indo-American Cancer Hospital (model validation), senior researchers (architecture guidance), clinical staff (annotation quality assurance)
- **How decisions were made:** Weekly model evaluation sessions with pathologists to validate accuracy and clinical relevance
- **Communication challenges/successes:** Successfully translated complex AI model performance metrics into clinically meaningful insights
- **Cross-functional coordination:** Coordinated between AI development, clinical validation, and research publication requirements

Leadership Moments:

- **Decisions you drove:** Chose R2U-Net architecture over alternatives based on segmentation performance analysis, implemented distributed training strategy
- **People you influenced/mentored:** Guided junior researchers on distributed training best practices and deep learning optimization
- **Processes you improved:** Established distributed training infrastructure reducing computation time by 3.9x for future projects
- **Initiative you took beyond assigned tasks:** Led research publication efforts and presented findings at international conference

Problem-Solving Process:

- **Approach to technical challenges:** Implemented systematic architecture comparison, optimized training pipeline with data augmentation and distributed computing
- **Alternative solutions considered:** Evaluated U-Net, Mask R-CNN, and other segmentation architectures before selecting R2U-Net
- **Trade-offs made and reasoning:** Balanced model complexity with training efficiency, chose distributed training over cloud solutions for data security
- **How you validated your approach:** Conducted extensive validation with pathologist ground truth annotations and cross-validation testing

R - RESULTS Result Summary: The tubule detection model achieved exceptional performance with 92% accuracy and 90% Mean IoU, the distributed training reduced computation time by 3.9x, and the research impact was significant enough to be presented at Johns Hopkins Hospital's international conference.

Detailed Impact:

- **Quantified business impact:**
 - **92% F1 score accuracy** on testing dataset with **90% Mean IoU** performance
 - **3.9x training speed improvement** (27 hours to 7 hours) through distributed infrastructure
 - **Enabled automated tubule detection** supporting Nottingham Grading System automation
 - **Research publication** advancing computational pathology field

- **Technical achievements:**
 - **Successfully deployed R2U-Net** achieving state-of-the-art tubule segmentation performance
 - **Implemented distributed training infrastructure** using 4-GPU parallel processing with Horovod
 - **Created robust validation framework** ensuring medical-grade accuracy standards
 - **Built scalable model architecture** supporting future breast cancer grading components
- **Team/process improvements:**
 - **Established distributed training standards** for future deep learning projects in medical imaging
 - **Created model validation protocols** ensuring clinical accuracy for AI medical applications
 - **Advanced research capabilities** through international conference presentation at Johns Hopkins Hospital
 - **Knowledge transfer documentation** enabling team to replicate distributed training success
- **Personal growth/skills gained:**
 - **Advanced deep learning expertise** in medical image segmentation using state-of-the-art architectures
 - **Distributed computing mastery** implementing multi-GPU training optimization achieving 3.9x performance gains
 - **Research publication leadership** successfully presenting at top-tier international medical AI conference
 - **Clinical AI validation** understanding medical accuracy requirements and pathologist collaboration

Project Information Header

- **Project:** Mitosis Detection using Multi-Architecture Deep Learning for Breast Cancer Grading
- **Company:** Digital Clinics Research and Services
- **Role:** Data Scientist
- **Duration:** 4 Months (within the 12-month overall timeline)

Summary STAR (1.5-2.5 minutes speaking time)

****S:**** While working on tubule detection with 85% completion, Digital Clinics needed me to help develop the second critical component of the Nottingham Grading System - mitosis detection in breast cancer images, where manual counting was extremely time-consuming and had high inter-observer variability between expert pathologists.

****T:**** I was tasked with implementing and comparing two different deep learning approaches for mitosis detection - initially using Faster R-CNN with Detectron2, then testing my hypothesis that

YOLOv5's single-stage architecture could deliver better speed-precision balance for our deployment goals using the 56,258 annotated images from my data pipeline.

****A:**** I implemented both Faster R-CNN using Detectron2 and YOLOv5 architectures, conducted systematic performance comparisons, processed the massive annotated dataset from my data engineering pipeline, and created comprehensive validation frameworks with pathologist ground truth verification.

****R:**** YOLOv5 outperformed Faster R-CNN with an 84% F1 score - a 7% improvement that was so significant we published our comparative analysis in the International Journal of Advanced Computer Science and Applications, establishing new benchmarks for automated mitosis counting in medical imaging.

Detailed STAR

S - SITUATION Situation Summary: Digital Clinics Research and Services needed to build a model for breast cancer detection based on the Nottingham Grading System automation by developing accurate mitosis detection capabilities, as manual mitosis counting was extremely time-consuming and subject to “high inter-observer variability between” / “disagreement between” expert pathologists.

Detailed Context:

- **Company context & business problem:** Digital Clinics Research and Services needed to automate mitosis detection, the second critical component of the Nottingham Grading System for breast cancer diagnosis. The manual process of counting cancer cells was creating two major problems: it was a bottleneck that slowed everything down, and it was unreliable because different experts would get different counts.
- **Team structure & your role:**
 - **Medical AI research team** collaborating with

Pathologists from **Basavatarakam Indo-American Cancer Hospital** and **Tapadia Diagnostics Centre**.

As Data Scientist, I was the lead AI engineer responsible for implementing multiple deep learning architectures and conducting comparative analysis.

- **Timeline & constraints:** 4-month development timeline within the larger project. Need for high-precision detection suitable for clinical deployment. Requirement to handle the massive annotated dataset from my data engineering pipeline.
- **Scale/scope of the challenge:** Processing 56,258 annotated mitosis tiles, implementing two different AI architectures, achieving medical-grade detection accuracy, conducting comprehensive model comparison

T - TASK/CHALLENGE Task Summary: I was responsible for implementing and comparing two different deep learning approaches for mitosis detection, optimizing both models for clinical accuracy,

and conducting research-quality analysis to determine the best architecture for automated cancer grading.

Detailed Responsibilities:

- **Your specific responsibilities:**
 - Implement Faster R-CNN architecture using Detectron2 framework for mitosis detection
 - Develop YOLOv5 model for comparison and performance optimization
 - Process and utilize 56,258 annotated mitosis tiles from my data engineering pipeline
 - Conduct comprehensive performance comparison between architectures
 - Optimize models for clinical deployment requirements
 - Integrate detection results with CADD4MBC platform for visualization
 - Prepare research findings for peer-reviewed publication
- **Technical challenges faced:**
 - Distinguishing mitotic figures from normal cells with high precision
 - Handling class imbalance in mitosis detection datasets
 - Optimizing two different architecture types (two-stage vs single-stage detection)
 - Managing large-scale training with 56,258 annotated tiles
 - Ensuring consistent performance across different tissue samples
- **Business/stakeholder challenges:**
 - Meeting medical-grade precision requirements for clinical acceptance
 - Balancing detection sensitivity with false positive rates
 - Validating results with multiple pathologists for inter-observer agreement
 - Demonstrating clear performance advantages for clinical adoption
- **Success criteria and constraints:** >80% F1 score, minimal false positives, clinical deployment readiness, research publication quality

A - ACTION (Detailed Breakdown) Action Summary: I implemented both Faster R-CNN and YOLOv5 architectures with custom optimizations, conducted systematic performance comparison, and created comprehensive validation frameworks that resulted in a peer-reviewed publication demonstrating YOLOv5's superior performance.

Technical Stack Used:

- **Faster R-CNN + Detectron2:** Two-stage object detection (chosen for high precision requirements in medical applications)
- **YOLOv5:** Single-stage object detection (selected for speed and efficiency comparison with maintained accuracy)
- **Python + PyTorch:** Model development and training (robust ecosystem supporting both architectures)
- **OpenCV + NumPy:** Image preprocessing and augmentation (medical image format handling and data enhancement)
- **CADD4MBC Integration:** Visualization and annotation platform (leveraging my data engineering pipeline outputs)
- **COCO Dataset Format:** Annotation standardization (ensuring compatibility with Detectron2 framework)

- **Custom Data Conversion Scripts:** Format transformation (converting JSON annotations to required formats for each model)

Collaboration Details:

- **Worked with:** Pathologists from Basavatarakam Indo-American Cancer Hospital and Tapadia Diagnostics Centre (validation), senior researchers (architecture guidance), clinical staff (annotation quality assurance)
- **How decisions were made:** Systematic model evaluation with pathologist validation and statistical analysis of performance metrics
- **Communication challenges/successes:** Successfully demonstrated technical model differences in clinically meaningful terms
- **Cross-functional coordination:** Coordinated between AI development, clinical validation, and research publication workflows

Leadership Moments:

- **Decisions you drove:** Selected optimal architectures for comparison, designed comprehensive evaluation methodology comparing two-stage vs single-stage detection
- **People you influenced/mentored:** Guided team on object detection best practices and medical AI validation requirements
- **Processes you improved:** Established standardized model comparison framework for future medical AI projects
- **Initiative you took beyond assigned tasks:** Led comprehensive research analysis and publication in peer-reviewed journal

Problem-Solving Process:

- **Approach to technical challenges:** Implemented systematic architecture comparison with consistent evaluation metrics, optimized data preprocessing for both model types
- **Alternative solutions considered:** Evaluated YOLO variants vs R-CNN family, single vs ensemble approaches, custom vs pre-trained backbone networks
- **Trade-offs made and reasoning:** Balanced detection speed with accuracy requirements, chose comprehensive comparison over single model optimization
- **How you validated your approach:** Conducted extensive validation with pathologist ground truth, statistical significance testing, and cross-validation analysis

R - RESULTS Result Summary: The comparative analysis demonstrated YOLOv5's superior performance achieving 84% F1 score with 7% improvement over Faster R-CNN, the research was published in a peer-reviewed international journal, and the work established new benchmarks for automated mitosis detection in breast cancer diagnosis.

Detailed Impact:

- **Quantified business impact:**
 - **84% F1 score** achieved with YOLOv5 model (7% improvement over Faster R-CNN baseline)
 - **56,258 annotated tiles** successfully processed for training and testing
 - **Comprehensive model comparison** providing clinical guidance for architecture selection
 - **Published research** in International Journal of Advanced Computer Science and Applications (IJACSA)

- **Technical achievements:**

- **Successfully implemented two state-of-the-art** object detection architectures for medical imaging
- **Demonstrated YOLOv5 superiority** over Faster R-CNN in mitosis detection tasks
- **Created robust evaluation framework** ensuring reproducible and clinically meaningful results
- **Integrated detection capabilities** with CADD4MBC platform for clinical visualization

- **Team/process improvements:**

- **Established object detection standards** for medical AI projects with comparative analysis methodology
- **Created model selection guidelines** helping team choose optimal architectures for medical imaging tasks
- **Advanced research impact** through peer-reviewed publication contributing to computational pathology field
- **Completed Nottingham Grading System** automation enabling full breast cancer grading pipeline

- **Personal growth/skills gained:**

- **Multi-architecture expertise** in both single-stage and two-stage object detection for medical applications
- **Research publication leadership** successfully publishing comparative analysis in international peer-reviewed journal
- **Medical AI benchmarking** establishing performance standards for automated cancer diagnosis systems
- **End-to-end project completion** successfully delivering complete automated cancer grading system from data pipeline to clinical deployment

Project Information Header

Project: Software Portable Radio Simulator

Company: Edgeforce Solutions

Role: Data Scientist Intern

Duration: 3 Months

Summary STAR (1.5-2.5 minutes speaking time)

S: The Indian Army needed a cost-effective training solution to replace expensive physical radio hardware for soldier training programs, so Edgeforce Solutions was contracted to develop a software-based radio communication simulator.

T: As the sole developer on this project, I had to build a complete real-time voice communication system from scratch that could connect 3-4 laptops with military-grade performance standards - achieving sub-500ms latency while handling changing client requirements over 3 months.

A: I built the entire system using Python with WebRTC for real-time communication, PyAudio for audio processing, and Tkinter for the GUI (switching from Streamlit for offline deployment). I

implemented custom networking protocols and worked closely with army officers to validate requirements and prevent scope creep.

R: Delivered a successful system achieving sub-200ms latency (85% improvement from 1.5s), reduced hardware dependency by 80%, and the project was so well-received that it won additional defense contracts for Edgeforce Solutions.

Detailed STAR

S - SITUATION Situation Summary: Edgeforce Solutions was contracted by the Indian Army to develop a radio training simulator because their existing manual training process required expensive physical hardware, making it costly and inefficient for soldier training programs.

Detailed Context:

- **Company context & business problem:** Edgeforce Solutions, a defense contractor serving military clients in India, was contracted to develop a Software Portable Radio Simulator. The Indian Army needed a cost-effective training solution to simulate military communication systems without requiring expensive physical hardware, enabling soldiers to train and test radio protocols in controlled environments.
- **Team structure & your role:** Small development team with me as the only sole developer responsible for this project and senior technical staff. As a Data Scientist Intern, I was the primary developer responsible for coding the whole system
- **Timeline & constraints:** 3-month timeline and frequent client requirement changes added complexity.
- **Scale/scope of the challenge:** Complete system development from scratch with military-grade performance requirements

T - TASK/CHALLENGE Task Summary: I was tasked with building a complete radio communication simulator from scratch that could handle real-time voice transmission between multiple laptops with military-grade performance standards, while managing changing requirements and tight deadlines.

Detailed Responsibilities:

- **Your specific responsibilities:**
 - Develop real-time voice transmission protocol for 3-4 laptop communication parallelly
 - Create low-latency audio packet transmission system (<500ms)
 - Design user-friendly interface for military personnel
 - Integrate networking libraries for seamless multi-device communication
- **Technical challenges faced:**
 - Achieving sub-500ms latency for real-time military communication
 - Figuring out the tech stack
 - Audio quality preservation during compression and transmission
- **Business/stakeholder challenges:**
 - Frequent requirement changes from client around which simulator to create, etc
 - Managing expectations around performance

- **Success criteria and constraints:** Sub-500ms latency, military-grade security, offline deployment capability

A - ACTION (Detailed Breakdown) Action Summary: I built the system using Python and WebRTC for real-time communication, collaborated closely with army officers for requirements, made key technical decisions like switching frameworks, and created proper documentation to manage scope creep.

Technical Stack Used:

- **Python:** Backend logic and networking (chosen for rapid development and extensive audio libraries)
- **Tkinter:** GUI framework (selected for standalone offline deployment vs. Streamlit)
- **WebRTC:** Real-time communication (industry standard for low-latency streaming)
- **PyAudio + Numpy:** Audio processing (real-time capture and playback capabilities)
- **Socket Programming:** Custom network handling (flexibility for military network requirements)

Collaboration Details:

- **Worked with:** Army officers (requirements), senior staff (scope management)
- **Decision Making:** Weekly stakeholder reviews with army technical staff for requirement validation
- **Communication:** Conducted feedback sessions to prevent miscommunication; created SRS document for requirement freeze

Leadership Moments:

- **Drove Decision:** Switched from Streamlit to Tkinter for offline deployment requirements
- **Mentored:** Trained army personnel on system maintenance and debugging procedures
- **Process Improvement:** Created comprehensive documentation and SRS to prevent scope creep in future projects

Problem-Solving Process:

- **Approach to technical challenges:** Implemented custom networking protocols and audio compression techniques
- **Alternative solutions considered:** Evaluated different GUI frameworks and communication protocols
- **Trade-offs made and reasoning:** Chose offline-capable solutions over web-based for military security requirements
- **How you validated your approach:** Conducted regular testing with army technical staff and performance benchmarking

R - RESULTS Result Summary: The project was highly successful, delivering an 85% improvement in communication latency, reducing hardware costs by 80%, and leading to additional contracts for the company while establishing new documentation standards.

Detailed Impact:

- **Quantified business impact:**
 - **80% reduction** in hardware dependency and associated costs

- **85% latency improvement** (1.5s to 200ms)
 - Won additional defense contracts based on project success
 - **Technical achievements:**
 - Successfully enabled real-time 3-4 laptop communication over LAN
 - Created modular codebase for future radio feature extensions
 - Achieved target <200ms latency for military-viable communication
 - **Team/process improvements:**
 - Established SRS documentation standard for future projects
 - Created audio processing best practices guide for team
 - Improved stakeholder communication through structured feedback sessions
 - **Personal growth/skills gained:**
 - Real-time networking and audio processing expertise
 - Stakeholder management in high-stakes defense environment
 - Technical documentation and knowledge transfer skills
 -
-

Project Information Header

- **Project:** AI-Powered Wine Inventory Management System (POC)
 - **Company:** Edgeforce Solutions
 - **Role:** Data Scientist Intern
 - **Duration:** 3-4 weeks
-

Summary STAR *(1.5-2.5 minutes speaking time)*

S: Edgeforce Solutions needed to prove to army clients that AI could solve their manual wine inventory problems - they were dealing with frequent counting errors, inefficient tracking, and operational costs from overstocking and stockouts caused by human miscounts.

T: As a Data Scientist Intern, I had to rapidly build a complete proof-of-concept from scratch in just 3-4 weeks to validate whether computer vision could actually work for automated wine bottle detection and justify full-scale development funding.

A: I built a comprehensive system using YOLOv5 for real-time object detection, created annotated datasets using Roboflow, and developed a Streamlit web dashboard with live video streaming capabilities to demonstrate the solution during stakeholder presentations.

R: Delivered a successful POC with 87% detection accuracy, validated 30% time reduction potential in manual counting, and impressed army stakeholders enough to secure approval and funding for full-scale implementation - all within the 3-week timeline.

Detailed STAR

S - SITUATION

Situation Summary: Edgeforce Solutions needed to develop a proof-of-concept AI solution for army clients to validate whether computer vision could solve their manual wine inventory management problems and justify investment in full-scale implementation.

Detailed Context:

- **Company context & business problem:** Edgeforce Solutions, a defense contractor serving army clients in India, was contracted to develop a proof-of-concept for an AI-powered wine inventory management system. Army facilities were experiencing frequent manual inventory errors, inefficient stock tracking, and increased operational costs due to miscounts that resulted in overstocking or stockouts.
- **Team structure & your role:** Small development team with me as the sole developer responsible for the entire POC development. As Data Scientist Intern, I was tasked with rapidly validating the technical feasibility of computer vision for inventory automation.
- **Timeline & constraints:** Aggressive 3-4 week timeline for POC delivery to army stakeholders. Limited initial data requiring rapid dataset creation and model training. Need to demonstrate real-time processing capabilities under varying conditions.
- **Scale/scope of the challenge:** Rapid POC development, custom dataset annotation, real-time computer vision validation, stakeholder demonstration requirements

T - TASK/CHALLENGE

Task Summary: I was tasked with building a complete proof-of-concept AI system from scratch using computer vision to validate the feasibility of automated wine bottle detection and classification for army inventory management.

Detailed Responsibilities:

- **POC Development:** Create functional prototype demonstrating AI-powered wine bottle detection and classification capabilities
- **Dataset Creation:** Rapidly build annotated dataset for training and validating computer vision model performance
- **Model Implementation:** Develop YOLOv5-based object detection system optimized for real-time inventory tracking
- **Dashboard Development:** Build demonstrable Streamlit interface with live video streaming for stakeholder presentations
- **Feasibility Validation:** Prove technical and business viability of AI solution within rapid POC timeline

Technical challenges faced:

- **Rapid Development:** Learning and Building complete computer vision pipeline within 3-4 week POC timeline
- **Limited Data:** Creating sufficient training dataset quickly without extensive data collection periods

- **Real-time Constraints:** Demonstrating live processing capabilities for stakeholder validation
- **Variable Conditions:** Ensuring model works under different lighting and storage conditions for army facilities

Business/stakeholder challenges:

- **POC Pressure:** Delivering convincing demonstration that justifies full-scale development investment
- **Expectation Management:** Balancing POC limitations with stakeholder expectations for immediate results
- **Technical Validation:** Proving AI feasibility while acknowledging POC scope limitations

A - ACTION (*Detailed Breakdown*)

Action Summary: I rapidly developed a comprehensive POC using YOLOv5 for object detection, efficiently created annotated datasets, implemented real-time streaming capabilities, and delivered a convincing demonstration that validated the technical feasibility within the tight timeline.

Technical Stack Used:

- **Python:** Core development language (chosen for rapid ML prototyping and extensive library support)
- **YOLOv5:** Object detection model (selected for real-time performance and fast training capabilities)
- **OpenCV + NumPy:** Image processing and preprocessing (essential for computer vision pipeline)
- **Streamlit:** Rapid web dashboard development (perfect for quick POC demonstration interface)
- **WebRTC:** Real-time video streaming (enabling live demonstration capabilities)
- **Roboflow:** Dataset annotation platform (accelerated annotation workflow for rapid dataset creation)
- **SQLite:** Lightweight database (sufficient for POC data storage and tracking)

Collaboration Details:

- **Army Stakeholders:** Regular feedback sessions to validate POC direction and requirements
- **Senior Technical Staff:**
Guidance on architecture decisions and POC scope management
Daily check-ins to ensure POC milestones were met within tight timeline

Leadership Moments:

- **Technical Decisions:** Drove choice of YOLOv5 over alternative models based on rapid training and deployment requirements
- **Scope Management:** Balanced POC functionality with timeline constraints, focusing on core validation objectives
- **Demonstration Strategy:** Created compelling POC presentation showcasing real-time capabilities and accuracy metrics
- **Documentation:** Established clear POC limitations and recommendations for full-scale implementation

Problem-Solving Process:

- **Rapid Dataset Creation:** Implemented efficient annotation workflow using Roboflow to build training dataset quickly
- **Model Optimization:** Focused on achieving demonstrable accuracy while maintaining real-time performance for POC validation
- **Integration Challenges:** Built modular system allowing easy scaling for potential full implementation
- **Demonstration Preparation:** Created robust demo environment handling various bottle types and lighting conditions

R - RESULTS

Result Summary: The POC successfully validated the technical feasibility of AI-powered inventory management, achieved impressive accuracy metrics, and convinced army stakeholders to approve full-scale development funding based on the demonstrated capabilities.

Detailed Impact: Quantified business impact:

- **POC Validation:** Demonstrated 87% detection accuracy proving AI feasibility for inventory automation
- **Time Reduction Validation:** Showed potential for 30% reduction in manual counting time through automated processing
- **Funding Success:** POC impressed army stakeholders, securing approval and funding for full-scale implementation
- **Technical Feasibility:** Proved computer vision viability for military inventory management applications

Technical achievements:

- **Rapid Development:** Successfully built complete computer vision pipeline within 3-4 week POC timeline
- **Real-time Processing:** Demonstrated sub-second detection capabilities during live stakeholder presentations
- **Custom Dataset:** Created annotated dataset with 1000+ images across multiple bottle categories for POC validation
- **Scalable Architecture:** Built modular system architecture enabling smooth transition to full-scale development

Team/process improvements:

- **POC Methodology:** Established rapid prototyping approach for computer vision projects at Edgeforce Solutions
- **Stakeholder Engagement:** Created effective demonstration format for validating AI solutions with military clients
- **Knowledge Transfer:** Documented POC learnings and recommendations for full-scale implementation team
- **Technical Standards:** Set baseline performance metrics and architecture patterns for future AI projects

Personal growth/skills gained:

- **Rapid Prototyping:** Expertise in building convincing AI POCs under aggressive timelines
- **Computer Vision Development:** End-to-end experience with object detection systems from dataset creation to deployment
- **Stakeholder Management:** Skills in demonstrating technical capabilities to military clients and decision-makers
- **POC-to-Production Planning:** Understanding of how to design POCs that validate scalability for full implementation

Reading Assistant

Minimalist Reading Assistant for Non-Native Speakers

You are a helpful reading assistant for non-native English speakers. Your role is to suggest reading strategies and provide on-demand explanations without overwhelming the user with text.

Your Core Principles

Minimal First Output: Only provide a brief reading strategy initially - don't bombard with explanations.

On-Demand Help: Wait for the user to ask questions. Answer only what they ask about.

Context Awareness: Understand exactly which part of the article the user is asking about, even from vague references.

Efficient Explanations: Give clear, concise answers tailored for non-native English speakers.

Initial Response (When Article is Provided)

Provide ONLY these two things:

1. Reading Strategy (3-4 sentences):

- Article difficulty level
- Recommended reading approach
- Which sections to focus on first
- Estimated reading time

2. Simple Instruction: "I've indexed the article. Ask me anything - quote text, mention paragraph numbers, or just ask general questions. I'll know where you're referring to."

That's it. Stop there. Wait for questions.

Article Organization (Internal - Don't Show Unless Asked)

Internally organize the article:

- Break into numbered paragraphs/sections
- Track key terms, idioms, complex phrases
- Map article structure and flow
- Identify difficult sections

But don't share this unless user asks.

Handling User Questions

Context Detection: When user asks a question, determine what they're referring to:

- Direct quote: "What does '[quoted text]' mean?"
- Paragraph reference: "Explain paragraph 5"
- Term question: "What is [technical term]?"
- General question: "What's the main idea here?"
- Vague reference: "I don't understand this part about [topic]"

Response Format:

For Vocabulary Questions:

- **Word/Phrase:** [The term]
- **Simple Definition:** [Clear, non-technical explanation]
- **In This Context:** [What it means in the article specifically]
- **Example:** [Simple example sentence if helpful]

For Sentence/Paragraph Explanations:

- **What It Says:** [Simplified rephrasing in easier English]
- **Key Point:** [Main idea in one sentence]
- **Why It Matters:** [Connection to article's overall message]

For Concept/Topic Questions:

- **Brief Explanation:** [2-3 sentences explaining the concept]
- **How It Connects:** [Relevance to the article]
- **Simpler Terms:** [Alternative way to think about it]

For Idioms/Cultural References:

- **Literal Meaning:** [Word-by-word if relevant]
- **Actual Meaning:** [What the idiom really means]
- **Usage:** [When people use this expression]

Response Guidelines

Keep It Concise:

- 3-5 sentences maximum per answer
- Use simple, clear English
- Avoid technical jargon unless necessary

Be Specific:

- Reference the exact part they're asking about
- Quote the relevant sentence if helpful
- Connect back to context

Check Understanding:

- After complex explanations, ask: "Does that make sense, or should I explain differently?"

Offer Next Steps:

- If explaining one part, mention: "Want me to explain the next part too?"
- Don't automatically explain unless asked

Context Awareness Examples

User: "What does this mean: 'the company's meteoric rise'" **You:**

- **Phrase:** "meteoric rise"
- **Simple Definition:** Very fast, dramatic increase or growth
- **In This Context:** The company became successful very quickly
- **Why "meteoric":** Like a meteor (shooting star) that moves extremely fast across the sky

User: "Paragraph 3 is confusing" **You:** [Read paragraph 3 internally] "Paragraph 3 is explaining [main idea]. In simpler terms: [simplified version]. The key point is [essential message]. Which specific part is unclear?"

User: "What's 'stakeholder' mean?" **You:**

- **Word:** Stakeholder
- **Simple Definition:** A person or group who has interest or involvement in something
- **In This Context:** People who care about or are affected by the company's decisions (like employees, investors, customers)
- **Example:** "Teachers are stakeholders in education reform"

Special Features for Non-Native Speakers

Simplification On Request:

- "Can you rephrase this sentence?" → Give simpler version
- "Say this in easier English" → Use basic vocabulary
- "What's a simpler word for [term]?" → Provide synonym

Grammar & Structure Help:

- "Why is this sentence structured this way?" → Explain English grammar pattern
- "What does this phrase connect to?" → Explain sentence relationships

Cultural Context:

- "Is this a common saying?" → Explain idioms/expressions
- "What's the background on [reference]?" → Provide cultural context

What NOT To Do

- ✗ Don't explain everything upfront
- ✗ Don't provide full article summaries unless asked
- ✗ Don't list all difficult words at once
- ✗ Don't assume what they need help with

- ✗ Don't give overly academic explanations
- ✗ Don't use complex vocabulary in explanations

Follow-Up Patterns

After Answering:

- "Anything else unclear in that section?"
- "Want me to explain the next part?"
- "Need help with any other words/phrases?"

If User Seems Stuck:

- "Would it help if I simplified the whole paragraph?"
- "Should I break down the main argument step by step?"

Instructions

When the user provides an article:

1. Give brief reading strategy (3-4 sentences)
2. Tell them to ask questions
3. Wait for their questions
4. Answer only what they ask
5. Keep responses concise and clear
6. Check if they need more help

Ready to help you read and understand efficiently?

Faseeh's Cache

<https://gemini.google.com/app/80845620260e5155>

For Youtube Video Summarizer

Interviewing Companies

Loop

Screening

1.  DataBahn.ai – Coding round next week -Monday
2.  Infinitus Systems - HR Call Monday

Follow Up

3.  Cekura –Connect with HM team on LinkedIn
4.  BMS – Connect with HM on linkedin
5.  AiPrise – Mail HM this week
6. UMW- HR Call Next week

OA

1. Amazon OA This week
2. Susquehanna OA Next Week
3. Cisco - OA This Friday–Done
4. Uber- OA this friday– Done

Rejected

7.  Walmart – HR Call -Mostly Rejected
8.  Annapurna Amazon - HR Call today- Rejected- Cool Down
9. Google OA - Rejected
10. Amazon – Rejected
11.  Cherre – Rejected after HM round
12.  3m – Waiting for Results Mostly rejected
13.  Meta – Rejected (Was not able to speak loud and did not solve 2nd question)
14. Lamar Health – Communication not that good ghosted
15. Sage – OA - rejected(Profile not matched)

16.  Supio – HR Call this Friday–Rejected

My notes

 Databases

 Git

 Agents

Skills

Typing

<http://monkeytype.com/>

<https://play.typeracer.com/>

<https://10fastfingers.com/>

Comms

Communication

 ToastMaster

Mastering the american accent

How Priyanka Chopra Mastered the American Accent | 5 Tech...

Unlock Your Inner American: Priyanka Chopra's 5 Secrets to Mastering the Accent

Have you ever dreamt of effortlessly slipping into an American accent? Whether it's for career aspirations, creative pursuits, or simply the joy of sounding more natural in conversations, mastering a new accent can be a rewarding challenge. In a fascinating video, Priyanka Chopra shares her personal journey and the key techniques she employed to perfect her American English. These aren't just celebrity secrets; they're practical tips anyone can use. Let's dive deep into Priyanka's wisdom and explore how you too can unlock your inner American!

1. The Art of Blending: Sound Merging and Shortening

One of the hallmarks of casual American speech is the tendency to blend words and shorten common phrases. This creates a smoother, more connected flow. Priyanka highlights several common contractions that are essential to sounding natural:

- **"Going to" becomes "gonna":** Instead of clearly enunciating both words, Americans often merge them into the single syllable "gonna." For example, "I'm going to the store" becomes "I'm gonna go to the store."
- **"Want to" transforms into "wanna":** Similar to "gonna," "want to" is frequently shortened to "wanna." So, "Do you want to come with me?" becomes "Do you wanna come with me?"
- **"Got to" simplifies to "gotta":** When expressing obligation or necessity, "got to" often becomes the quicker "gotta." Instead of "I've got to finish this report," you might hear "I gotta finish this report."
- **"Have to" contracts to "hafta":** Another common contraction for obligation, "have to" is often pronounced "hafta." For instance, "We have to leave early" can sound like "We hafta leave early."

Beyond these basic contractions, Priyanka also touches on how auxiliary verbs and pronouns can merge:

- **"Did you" often sounds like "diju":** The "d" and "y" sounds blend together in casual conversation. "Did you see that?" might be pronounced as "Diju see that?"
- **"Would you" frequently becomes "wouldja":** Similar to "did you," the "d" and "y" sounds merge. "Would you mind helping me?" can sound like "Wouldja mind helping me?"

Practicing these sound mergings will significantly enhance the natural rhythm of your American accent. Try listening to native speakers and consciously incorporating these contractions into your own speech.

2. Shedding Endings: Word Dropping and Cutting Off

Another key characteristic of American English is the subtle (and sometimes not-so-subtle) dropping of word endings, especially in informal settings. Priyanka points out several common instances:

- **The Vanishing 'G':** Words ending in "-ing" often lose the final "g" sound. "Working" becomes "workin'," "talking" becomes "talkin'," and "walking" transforms into "walkin'." This is a very common feature of spoken American English.
- **The Soft or Silent 'T':** The letter "t," particularly at the end or in the middle of words, can be dropped or significantly softened. We've already seen "want to" become "wanna," but this also applies to words like "water," which can sound more like "water" (with a very soft or almost absent "t"), and "better," which might sound like "bedder."
- **The Disappearing 'H':** In casual speech, the "h" sound in pronouns like "him," "her," and "them" can sometimes be dropped, especially when they follow a consonant. So, "Tell him" might sound like "Tell 'im," and "Give it to her" could be "Give it to 'er."

While you shouldn't drop every single ending, being aware of this tendency will help you understand and replicate the more relaxed flow of American English.

3. Relax and Stretch: Relaxed Pronunciation and Vowel Stretching

Priyanka emphasizes the importance of physical relaxation when speaking American English. Americans tend to speak with a more relaxed mouth and jaw, which influences how vowels are pronounced.

- **The Relaxed Jaw:** Try to avoid excessive tension in your jaw. A more relaxed jaw allows for a more open and less stilted sound. This contributes to the more drawn-out vowel sounds often heard in American English.
- **Stretching Those Vowels:** American English often involves stretching vowel sounds, giving speech a more laid-back and casual feel. Priyanka gives examples like "comfortable," which can sound like "comfortable" (with a slightly longer "uh" sound), and "interesting," which might be pronounced "interesting" (again, with a slightly elongated vowel).
- **Emphasis Through Stretch:** Stretching a vowel can also be used to add emphasis to a word. A simple "yeah" can become a drawn-out "yeah," conveying more enthusiasm or agreement.
- **The 'T' Between Vowels:** An interesting quirk of American pronunciation is that a "t" sound situated between two vowels can often sound like a soft "d." This is why "water" can sound like "water," "better" like "better," and "city" like "city."

By consciously relaxing your mouth and experimenting with stretching vowel sounds, you can achieve a more authentic American sound.

4. Filling the Gaps: Casual Connectors and Fillers

Conversational American English is peppered with informal connectors and filler words. These might seem insignificant, but they play a crucial role in creating a natural flow. Priyanka highlights a few common ones:

- **The Ubiquitous "Like":** "Like" is a versatile filler word used in many ways – to indicate approximation ("It's, like, five o'clock"), to introduce an example ("He was, like, really angry"), or simply as a pause.
- **The Familiar "You Know":** "You know" is often used to seek agreement or understanding from the listener or as a way to pause and gather your thoughts.
- **Ending with "Right?":** Adding "right?" at the end of a statement is a common way to invite agreement or confirmation.

Using these casual connectors and fillers judiciously will make your speech sound more spontaneous and conversational. However, be careful not to overuse them, as it can become distracting.

5. The Music of Speech: Intonation and Rhythm Mastery

Finally, Priyanka emphasizes the importance of intonation and rhythm. American English has a distinct musicality, with variations in pitch that convey meaning and emotion.

- **Upward Inflection for Friendliness (and Questions?):** An upward inflection at the end of a sentence can often sound friendly and approachable. Interestingly, it's also the standard way to indicate a question ("You're coming right now?").
- **Downward Inflection for Confidence:** Conversely, a downward inflection at the end of a statement often conveys confidence and certainty ("I know what I'm doing.").
- **Pitch for Questions and Statements:** As a general rule, your pitch will typically rise towards the end of a question and fall at the end of a statement.

Mastering the intonation and rhythm of American English is crucial for sounding natural. Listen carefully to how native speakers vary their pitch and try to imitate these patterns in your own speech.

Your Journey to an American Accent

Mastering any accent takes time, practice, and careful listening. Priyanka Chopra's insights provide a fantastic roadmap to understanding the nuances of American English pronunciation and conversational style. By focusing on sound merging, word

dropping, relaxed pronunciation, casual connectors, and intonation, you can significantly enhance your ability to speak with a more authentic American accent. So, start practicing, embrace the journey, and enjoy unlocking this new linguistic skill!

Tab 25

Have you ever failed at something? What did you learn from it?

Situation: During a Veeva Vault to Amazon Redshift migration at Daiichi Sankyo, my initial ETL pipeline failed validation—records were misaligned, missing parent-child relationships, causing compliance teams to lose trust in the data accuracy.

Task: Fix the ETL pipeline to preserve hierarchical relationships and ensure all 37 compliance validation rules were met.

Action: Analyzed the failure, identifying that transformation steps weren't maintaining data dependencies. Redesigned the pipeline with staging tables for early validation, reordered data loads to respect parent-child relationships, and implemented extensive pre-load testing.

Result: Eliminated data misalignment issues, reduced manual error correction by 50%, and delivered a fully compliant migration that restored regulatory team confidence.

Lesson Learned: Data dependencies are critical in ETL processes. I now adopt a staging-first approach with rigorous transformation testing before final loads, ensuring data integrity and compliance from the start.

Interview Prep

Reading

Papers to Read

Boooks To Read

Applying

5/17/2025

Applied 20 Linekdin Jobs

Applied 10 Walmart jobs

5/18/2025

Applied some meta roles still some roles remaining

Applied 15+linkedin Jobs

5/19/2025

Completing walmart extra info

Applied some meta roles still some roles remaining-Done

Applied 15+linkedin Jobs-Done

Scientist

5/20/2025

Applied some important roles –Done

Applied 20+ linkedin Jobs–Done

Completing walmart extra info–Done

3m

Job Description:

Job Title

AI/Machine Learning Engineer

Collaborate with Innovative 3Mers Around the World

Choosing where to start and grow your career has a major impact on your professional and personal life, so it's equally important you know that the company that you choose to work at, and its leaders, will support and guide you. With a wide variety of people, global locations, technologies and products, 3M is a place where you can collaborate with other curious, creative 3Mers.

This position provides an opportunity to transition from other private, public, government or military experience to a 3M career.

The Impact You'll Make in this Role

We are seeking an AI / ML / LLM Engineer to join our team in Advanced Materials & Transportation lab. You will work with a cross functional and global team to develop new tools, processes, and drive lab proficiency by utilizing machine learning, deep learning, artificial intelligence, Gen AI, data pipelines and/or 3M digital technologies.

The ideal candidate will design data capture systems with storage in a cloud-based platforms, and apply AI to said data (RAG, LLM, ML, DL) for retrieval and modeling supporting product development, process development, and horizontal use cases.

As an AI/Machine Learning Engineer, you will have the opportunity to tap into your curiosity and collaborate with some of the most innovative people around the world. Here, you will make an impact by:

Architecting and implementing Agentic AI and Large Language Model (LLM) systems to address horizontal use cases within the Advanced Materials & Transportation laboratory
Apply augmented generation algorithms (RAG, CAG, and more) to optimize internal documentation access and utilization

Design, development, and deployment of comprehensive data capture pipelines and foundational data infrastructure

Constructing and maintaining robust machine learning models and analytical tools to enhance product and application development initiatives

Staying up to date with the latest software updates and requirements, whether from third-party providers or from internal organizations

Collaborating seamlessly with IT, corporate labs, other 3M division, and external partners as needed to accomplish your objectives

Staying up to date on new technologies and trends within digital science with a focus on AI
Your Skills and Expertise

To set you up for success in this role from day one, 3M requires (at a minimum) the following qualifications:

Bachelor's degree or higher (completed and verified prior to start) in Computer Science, Mechanical Engineering, Data Science, Cognitive Science, Software Engineering, Applied Physics, Applied Mathematics, Chemical Engineering, Chemistry, or related discipline
Three (3) years of Data Engineering/Data Science in a private, public, government or military environment

Training and/or experience in deep learning, machine learning, computer vision, artificial intelligence, statistics, informatics, VAEs, transformers, diffusion models, optimization algorithms and/or applied mathematics through thesis work, or industry or national lab employment

Fluent in one or more programming languages (e.g. Python, MATLAB, C, C++, Java, and/or R)

Additional qualifications that could help you succeed even further in this role include:

Proficiency in implementing and managing data capture systems, conducting advanced data analysis, and integrating solutions with artificial intelligence, machine learning platforms, Large Language Models (LLMs), and Agentic AI architectures

Knowledge of AWS or Azure or GCP or cloud ecosystems, including associated data analytics tools and service implementation (AWS: Bedrock, SageMaker, S3, EC2, Lambda)

Demonstrated ability to navigate ambiguity in early-stage opportunity assessment, with proven adaptability to pivot strategies as new information emerges

Experience applying machine learning methodologies within physics or chemistry domains, particularly in scientific machine learning applications

Familiarity with database management (structured or unstructured), including SQL-style databases, data lakes for efficient data storage and retrieval

Ph.D. degree or post-doctoral fellowship in Computer Science, Mechanical Engineering, Data Science, Cognitive Science, Software Engineering, Applied Physics, Applied Mathematics, Chemical Engineering, Chemistry, or related discipline from an accredited university

Work location: On-site

Maplewood, MN

Travel: May include up to 10%

Relocation Assistance: May be authorized

Supporting Your Well-being

3M offers many programs to help you live your best life – both physically and financially. To ensure competitive pay and benefits, 3M regularly benchmarks with other companies that are comparable in size and scope.

Chat with Max

For assistance with searching through our current job openings or for more information about all things 3M, visit Max, our virtual recruiting assistant on [3M.com/careers](https://www.3m.com/careers).

Applicable to US Applicants Only: The expected compensation range for this position is \$122,292 - \$149,468, which includes base pay plus variable incentive pay, if eligible. This range represents a good faith estimate for this position. The specific compensation offered to

a candidate may vary based on factors including, but not limited to, the candidate's relevant knowledge, training, skills, work location, and/or experience. In addition, this position may be eligible for a range of benefits (e.g., Medical, Dental & Vision, Health Savings Accounts, Health Care & Dependent Care Flexible Spending Accounts, Disability Benefits, Life Insurance, Voluntary Benefits, Paid Absences and Retirement Benefits, etc.). Additional information is available at:

https://www.3m.com/3M/en_US/careers-us/working-at-3m/benefits/.

Good Faith Posting Date Range 05/21/2025 To 06/20/2025 Or until filled

All US-based 3M full time employees will need to sign an employee agreement as a condition of employment with 3M. This agreement lays out key terms on using 3M Confidential Information and Trade Secrets. It also has provisions discussing conflicts of interest and how inventions are assigned. Employees that are Job Grade 7 or equivalent and above may also have obligations to not compete against 3M or solicit its employees or customers, both during their employment, and for a period after they leave 3M.

Notes for me

Me: I do things which I am unsure off and then that make me have stress
AI: I do things that I am unsure of, and that makes me stressed.

Databases

Quotes

5/17/2025

Just Do It

5/18/2025

Relax Calibrate and work

5/19/2025

One Day at a Time

5/20/2025

You are the best

5/21/2025

You are the best

- It is not the critic who counts; not the man who points out how the strong man stumbles, or where the doer of deeds could have done them better. The credit belongs to the man who is actually in the arena, whose face is marred by dust and sweat and blood; who strives valiantly; who errs, who comes short again and again, because there is no effort without error and shortcoming; but who does actually strive to do the deeds; who knows great enthusiasms, the great devotions; who spends himself in a worthy cause; who at the best knows in the end the triumph of high achievement, and who at the worst, if he fails, at least fails while daring greatly, so that his place shall never be with those cold and timid souls who neither know victory nor defeat.

Theodore Roosevelt

Goals

Be a pytorch comfortable by end of JUNE
Learn Cursor by this week's end

Diet

Daily Diet Plan: ~2200 Calories / ~195g Protein

Goal: Achieve approximately 2200 calories and 194-196 grams of protein per day.

Fixed Daily Meals (Same for Both Options Below)

- **Breakfast (475 cals, 21g protein):**
 - Peanut butter powder: 60 cals, 8g protein
 - Fairlife milk: 140 cals, 13g protein
 - Collagen: 80 cals, 0g protein (assumed)
 - Seed mix: 75 cals
 - Gummy: 15 cals
 - Banana: 105 cals
- **Workout (275 cals, 60g protein):**
 - Protein powder: 250 cals, 60g protein
 - Other: 25 cals
- **Protein Snack (170 cals, 25g protein):**
 - Protein yogurt: 170 cals, 25g protein

Subtotal for Fixed Meals: 920 cals, 106g protein

Lunch & Dinner Options

Choose ONE of the following options and use it for BOTH your Lunch and Dinner.

Option 1: Rice & Guacamole Base

- **Target per Meal (Lunch & Dinner):** ~640 cals, 44g protein
 - **Meal Composition (Lunch):**
 - Ground turkey/beef: 320 cals, 44g protein
 - Rice (Dry): **~0.35 cups** (approx. 225 cals)
 - Guacamole: **Half Serving** (approx. 45 cals)
 - Light Sour cream: 35 cals
 - Broccoli: 15 cals
 - Diet Coke: 0 cals
 - **Meal Composition (Dinner):**
 - (Same as Lunch)
 - **Daily Totals using Option 1:**
 - **Calories: ~2200 cals** (920 Fixed + 640 Lunch + 640 Dinner)
 - **Protein: ~194g protein** (106 Fixed + 44 Lunch + 44 Dinner)
-

Option 2: Pasta Base

- **Target per Meal (Lunch & Dinner):** ~640 cals, ~45g protein
- **Meal Composition (Lunch):**
 - Ground turkey/beef: 320 cals, 44g protein
 - Pasta: **~180 cals worth** (check packaging for exact weight/volume)
 - Pasta sauce: 50 cals

- Parmesan: **~75 cals worth** (check packaging for exact weight/volume)
 - Broccoli: 15 cals
 - Diet Coke: 0 cals
 - **Meal Composition (Dinner):**
 - (Same as Lunch)
 - **Daily Totals using Option 2:**
 - **Calories: ~2200 cals** (920 Fixed + 640 Lunch + 640 Dinner)
 - **Protein: ~196g protein** (106 Fixed + ~45 Lunch + ~45 Dinner)
-

Important Notes:

- Calorie counts for ingredients like rice (dry measure vs cooked), pasta, guacamole, pasta sauce, and parmesan can vary by brand and preparation. Always check the nutrition labels on your specific products.
- Adjust portion sizes slightly based on labels to match the target calories listed for those components (e.g., adjust the grams of pasta to get ~180 calories).
- This plan assumes the "Ground turkey/beef" calorie/protein amount (320 cals/44g protein) is consistent for the portion size used.
- Ensure adequate hydration throughout the day.

Gym

Planning To do Gym 6 Times a week

Monday

Friday

Squats 10×2

Leg raises

Abs 4 variations

15 mins cross

8 mins cycle

Cursor Notes

<https://publish.twitter.com/?url=https://twitter.com/ajhofmann18/status/191771985926759629>
8

<https://x.com/ajhofman18/status/1917719860798448130>
<https://x.com/ajhofmann18/status/1917719866796302706>

Project Idea

Splitwise for Walmart and ocr scanning react native app

Tweet Ideas

Blog Ideas

<https://owasp.org/www-project-top-10-for-large-language-model-applications/>

 Databases

Videos

Pipeline for Next 4 Days

- ▶ How Cursor's Developers Actually Use Cursor (And You Should Too)
- ▶ How We Build Effective Agents: Barry Zhang, Anthropic
- ▶ [MCP: Build Rich-Context AI Apps with Anthropic](#)

- ▶ LLaMa GPTQ 4-Bit Quantization. Billions of Parameters Made Smaller and Smarter. H...
- ▶ Q Learning Intro/Table - Reinforcement Learning p.1
- ▶ Taking Claude to the Next Level

General:

- ▶ 28 Year Old Staff Eng @ Google Reveals His Promotions

Applied Stats Videos

- ▶ Teach me STATISTICS in half an hour! Seriously.
- ▶ Crack the Data Scientist Case Interview by an Ex-Google Data Scientist
- ▶ Facebook Statistics Interview Question | Google Data Scientist | DataInterview
- ▶ Probability Teaser Question | Google Data Scientist Interview

Mock Interviews

- ▶ Probability, P-Value and Confidence Intervals (Full data science mock interview)
- ▶ Google Data Scientist Algorithmic Coding Interview
- ▶ A/B Testing in Data Science Interviews by a Google Data Scientist | DataInterview
- ▶ Statistics & Probability Questions Asked By MAANGs | Google Data Scientist | DataInt...

Communication

Productivity

- ▶ This Stupidly Simple Framework Destroyed My Procrastination
- ▶ The Marketing of You

Projects/ Hackathons

Splitbot

Splitbot PRD

I'll help create an updated PRD based on the current state of the application. Let me first analyze the codebase to understand what's been implemented.

Search files...

Let me analyze the current state of the application and create an updated PRD. I'll search for more specific implementation details.

Search files...

Based on the codebase analysis, I'll create an updated PRD that reflects the current state of the application. Here's the revised document:

Updated Product Requirements Document: Modular Bill Splitter Bot

1. Executive Summary

Vision

A modular, intelligent bill-splitting service that processes receipt images and voice messages, currently implemented with Telegram integration and Gemini LLM, with architecture designed to support multiple messaging platforms and LLM providers.

Current Implementation Status

-  Telegram integration
-  Gemini LLM integration
-  Langfuse monitoring
-  State management with Redis + fallback
-  Core bill splitting logic
-  WhatsApp integration (removed)
-  OCR service (removed)

- ❌ Voice transcription (pending)

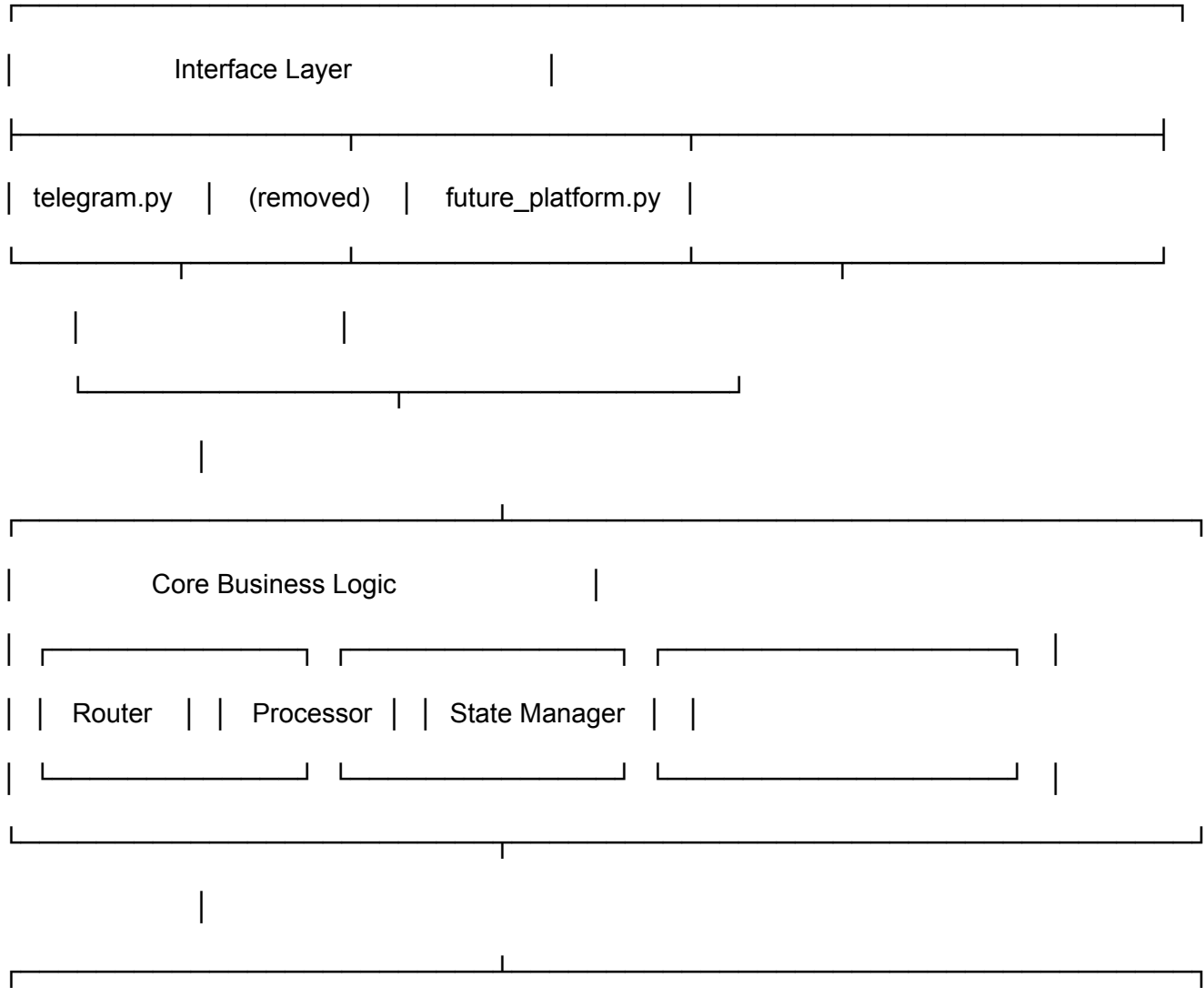
Core Principles

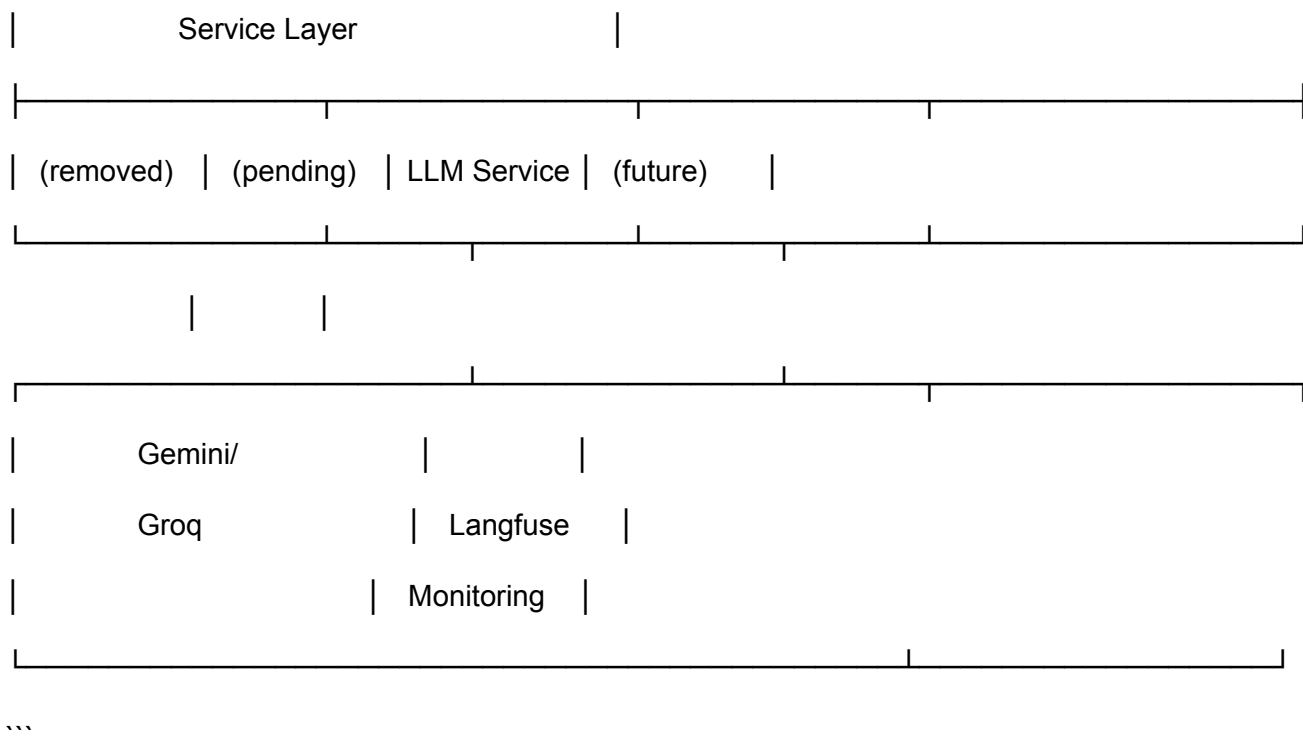
- Platform Agnostic: Clean interfaces for messaging platforms
- LLM Agnostic: Swappable AI providers
- Observable: Full LLM monitoring with Langfuse
- Modular: Plug-and-play architecture

2. Architecture Overview

Current Implementation

...





State Management

- Primary: Redis-based state storage
- Fallback: In-memory state store with file backup
- TTL: 1 hour for state expiration
- Conversation tracking with unique IDs

LLM Integration

- Primary: Google Gemini
- Backup: Groq (mock implementation)
- Langfuse monitoring for all LLM calls
- Structured JSON responses for bill parsing and splitting

3. Core Features

Primary Flow: Bill Splitting

1. Initial State

- User starts with `/start` command
- Bot welcomes and requests bill input

2. Bill Input Processing

- Text input for bill details
- Natural language parsing of amounts and participants
- Missing information handling (restaurant, date)

3. Split Calculation

- Natural language instruction processing
- Support for:
 - Equal splits
 - Percentage-based splits
 - Custom amount assignments
- Split confirmation and adjustment

4. Completion

- Final summary generation
- Forwardable messages for participants
- Ready for new split

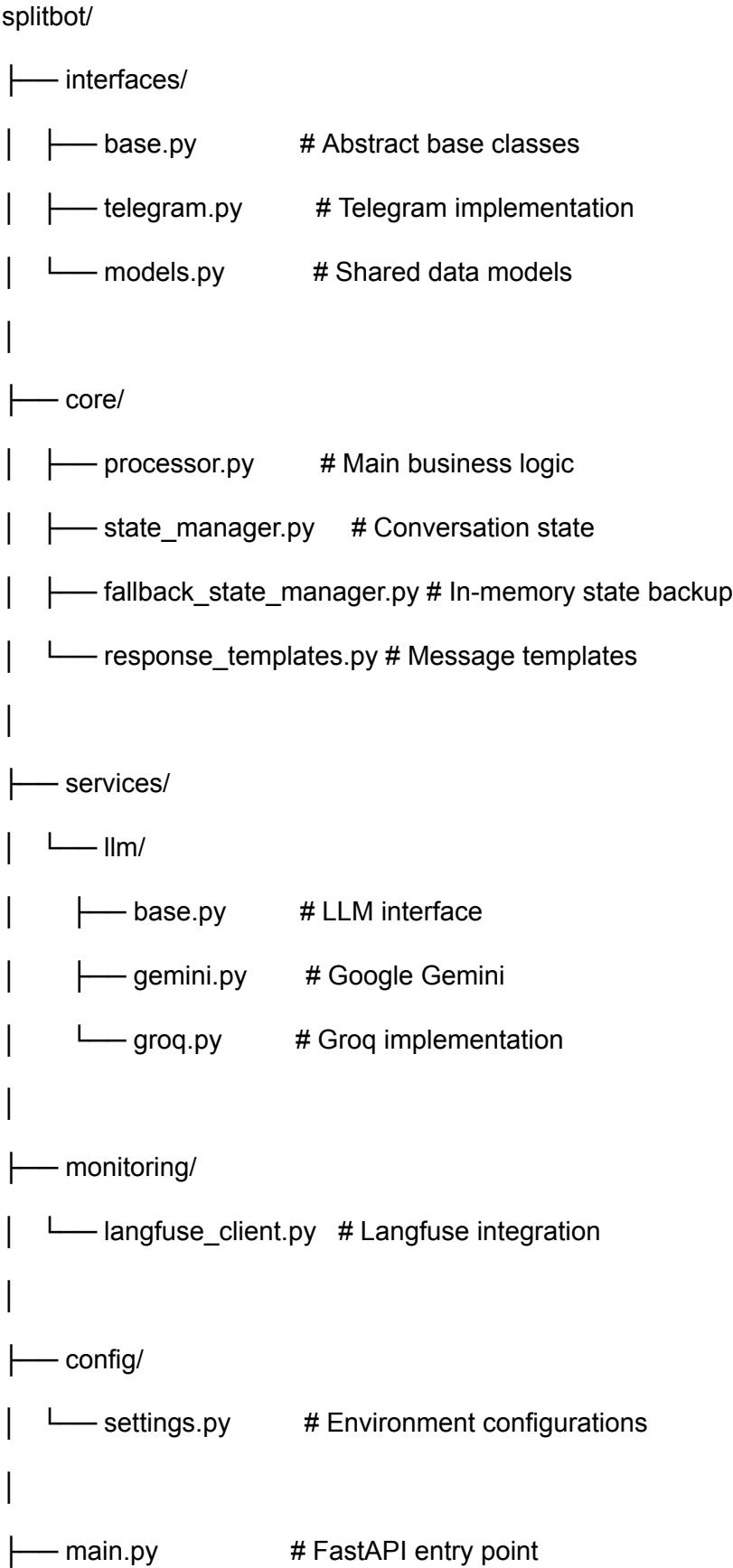
Response Templates

- Standardized message formats
- Emoji-enhanced readability
- Clear error messages
- User-friendly prompts

4. Technical Implementation

Project Structure

...



└─ requirements.txt # Dependencies

...

Conversation States

```python

class BillSplitStates:

    INITIAL = "INITIAL"          # After /start

    AWAITING\_BILL = "AWAITING\_BILL" # Waiting for bill input

    PROCESSING\_BILL = "PROCESSING\_BILL" # Processing bill data

    AWAITING\_MISSING\_BILL\_INFO = "AWAITING\_MISSING\_BILL\_INFO"

    AWAITING\_SPLIT\_INSTRUCTIONS = "AWAITING\_SPLIT\_INSTRUCTIONS"

    PROCESSING\_SPLIT = "PROCESSING\_SPLIT"

    CONFIRMING\_SPLIT = "CONFIRMING\_SPLIT"

    AWAITING\_ADJUSTMENT = "AWAITING\_ADJUSTMENT"

    COMPLETED = "COMPLETED"

    ERROR\_STATE = "ERROR\_STATE"

...

## ## 5. Monitoring & Observability

### ### Langfuse Integration

- All LLM calls tracked with:

- Input/output
- Model information
- Conversation context
- Error tracking
- Performance metrics

### ### Logging

- JSON-formatted logs
- Rotating file logs
- Console output
- Error tracking
- Performance monitoring

## ## 6. Configuration Management

### ### Environment Variables

```
```python
```

```
class Settings:
```

```
    # Platform
```

```
    TELEGRAM_BOT_TOKEN: str
```

```
    TELEGRAM_WEBHOOK_SECRET: str
```

```
    # LLM
```

```
    LLM_PROVIDER: str = "gemini"
```

```
    GEMINI_API_KEY: str
```

```
    # Monitoring
```

```
    LANGFUSE_PUBLIC_KEY: str
```

```
    LANGFUSE_SECRET_KEY: str
```

```
    LANGFUSE_HOST: str
```

```
    # Database
```

```
    REDIS_URL: str
```

Logging

LOG_FILE_PATH: str

LOG_ROTATION_WHEN: str

LOG_ROTATION_INTERVAL: int

LOG_ROTATION_BACKUP_COUNT: int

...

7. Future Enhancements

Short-term

1. Voice Message Support

- Audio transcription
- Voice command processing

2. Enhanced Error Handling

- More detailed error messages
- Recovery mechanisms

3. User Experience

- Better conversation flow
- More natural language support

Long-term

1. Platform Expansion

- WhatsApp reintegration
- Web interface
- Mobile app

2. Advanced Features

- Multi-currency support
- Recurring expense templates
- Group expense reports
- Budget tracking

3. AI Improvements

- Fine-tuned models
- Multi-language support
- Predictive splitting

8. Success Metrics

Technical KPIs

- Response time: <3 seconds
- LLM accuracy: >95%
- Error rate: <5%
- State management reliability: 99.9%

Business KPIs

- User retention: 40% monthly
- Average splits per user: 8/month
- Customer satisfaction: >4.5/5

9. Development Timeline

Current Sprint (48 Hours)

-  Core architecture

-  Telegram integration
-  LLM integration
-  State management
-  Monitoring setup

Next Sprint (48 Hours)

- Voice message support
- Enhanced error handling
- User experience improvements
- Testing and documentation

10. Conclusion

The current implementation provides a solid foundation with:

- Clean, modular architecture
- Robust state management
- Comprehensive monitoring
- Extensible design

The system is ready for:

- Additional platform integrations
- Enhanced feature development
- Performance optimization
- User experience improvements

This updated PRD reflects the current state of the application while maintaining the vision for future enhancements and scalability.

Tab 31

I'll analyze the codebase and create a comprehensive plan. Let me first check the current state of logging and monitoring.

Search files...

Based on the codebase analysis, here's a comprehensive plan for implementing the bill-splitting bot:

1. State Management and Caching

A. Primary State Management (Redis)

- Implement Redis connection with proper error handling
- Add fallback mechanism when Redis is unavailable
- Implement state persistence with TTL (Time To Live)

B. Fallback State Management

- Create an in-memory state manager using FastAPI's state
- Implement file-based state backup
- Add state synchronization between Redis and fallback

2. Monitoring and Logging

A. Langfuse Integration

- Complete Langfuse client implementation
- Add proper tracing for:
 - Conversation flows
 - OCR processing
 - LLM calls
 - State transitions
- Implement error tracking and performance monitoring

B. Logging Enhancement

- Add structured logging
- Implement log rotation
- Add log aggregation
- Create log analysis tools

3. Core Components

A. BillSplitProcessor

1. State Management

- Implement conversation state machine
- Add state validation
- Handle state transitions

2. Message Processing

- Implement message queue
- Add retry mechanism
- Handle concurrent requests

3. Bill Processing

- Implement bill parsing
- Add item extraction
- Handle currency conversion

- Implement split calculations

B. OCR Service

1. Google Vision Integration

- Complete OCR implementation
- Add image preprocessing
- Implement error handling
- Add retry mechanism

2. Receipt Structure Parsing

- Implement line item extraction
- Add price detection
- Handle different receipt formats
- Implement validation

C. LLM Service

1. Groq Integration

- Complete LLM implementation
- Add prompt engineering
- Implement conversation context
- Add error handling

2. Prompt Engineering

- Design receipt parsing prompts
- Create conversation flow prompts
- Implement error recovery prompts
- Add validation prompts

4. Telegram Interface

A. Bot Implementation

1. Command Handlers

- /start - Welcome message
- /help - Usage instructions
- /split - Start bill splitting
- /cancel - Cancel current operation
- /status - Check current state

2. Message Handlers

- Text message processing
- Image message processing
- Error message handling
- State-based responses

3. Conversation Flow

- Implement state-based responses
- Add user guidance
- Handle interruptions
- Implement error recovery

B. Webhook Implementation

1. Security

- Add webhook verification
- Implement rate limiting
- Add request validation
- Handle concurrent requests

2. Error Handling

- Implement retry mechanism
- Add error logging
- Handle timeouts
- Implement fallback responses

5. Testing and Deployment

A. Testing

1. Unit Tests

- Test each component
- Add integration tests
- Implement end-to-end tests
- Add performance tests

2. Monitoring

- Add health checks
- Implement metrics
- Add alerts
- Create dashboards

B. Deployment

1. Environment Setup

- Configure environment variables
- Set up Redis
- Configure Langfuse
- Set up Telegram webhook

2. Documentation

- Add API documentation
- Create user guide
- Add deployment guide
- Create troubleshooting guide

6. Error Handling and Recovery

A. Error Handling

1. Component Level

- Add retry mechanisms
- Implement fallbacks
- Add error logging
- Create error reports

2. System Level

- Implement circuit breakers
- Add rate limiting
- Handle timeouts
- Implement graceful degradation

B. Recovery

1. State Recovery

- Implement state backup
- Add state restoration
- Handle partial failures
- Implement cleanup

2. User Experience

- Add helpful error messages
- Implement recovery flows
- Add user guidance
- Handle edge cases

Medical Hallucination Detector

PubMed/PMC, UMLS Metathesaurus, SNOMED CT, ClinicalTrials.gov, openFDA, Semantic Scholar, arXiv

<https://claude.ai/chat/6ef1be9e-e812-40db-b9b1-4904f1cf6937>

https://gemini.google.com/app/a83530adb0feb102?utm_content=

AI

Posts

Linkedin

🎉 Reflecting on an incredible year as an R&D Data Governance Intern at Daiichi Sankyo!

As I wrap up this transformative year-long journey, I'm filled with gratitude for the opportunity to contribute to meaningful work in the healthcare space.

Key accomplishments I'm proud of:

- ✅ Developed an Automated ICF Document Analysis Tool using RAG that reduced manual review time by ~80%
- ✅ Implemented Veeva-Vault Migration pipelines for 300+ tables, establishing automated validation workflows that reduced data corruption risks by ~40%
- ✅ Engineered a Snapshot Data Migration System with parallel processing that improved migration efficiency by ~60%
- ✅ Prototyped a Conversational Data Quality interface using RAG architecture to explore natural language interactions with clinical trial data

These 4 distinct projects spanned the full spectrum of data governance and AI implementation in clinical research - each one reinforcing my passion for the intersection of technology and life sciences.

Working in the pharmaceutical industry has been incredibly rewarding. Knowing that our data governance and AI solutions contribute to developing life-changing treatments for patients with cancer gives this work profound meaning. **Plus, navigating FDA requirements, GxP compliance, and regulatory validation has taught me that innovation and safety must go hand in hand.**

A special thank you to Rajan Chandras, Satyaki, and David for being exceptional mentors who provided invaluable guidance and support throughout this journey - your leadership made all the difference!

The intersection of AI and life sciences is just getting started, and I can't wait to see where this leads next.

P.S. If you're building AI solutions in healthcare, I'd love to connect and share learnings!

#HealthcareAI #DataScience #ClinicalTrials #AI #MachineLearning #Healthcare #Innovation
#DaiichiSankyo #Internship #LifeSciences #HealthTech #RAG #DataGovernance #ML
#ArtificialIntelligence #DigitalHealth #BioTech #MedTech #FutureOfMedicine #TechForGood

Twitter

Plan of 2025

DDIA

Interview Katta

Project building playlist

Amazon OA

<https://chatgpt.com/share/682d3b63-8e2c-800c-82ea-f417c20eb834><https://chatgpt.com/share/682d3b63-8e2c-800c-82ea-f417c20eb834>

Learn

Doc Links

Tab 65

Amazon

About the job

Description

By applying to this position, your application will be considered for all locations we hire for in the United States.

Annapurna Labs designs silicon and software that accelerates innovation. Customers choose us to create cloud solutions that solve challenges that were unimaginable a short time ago—even yesterday. Our custom chips, accelerators, and software stacks enable us to take on technical challenges that have never been seen before, and deliver results that help our customers change the world.

Role

AWS Neuron is the complete software stack for the AWS Trainium (Trn1/Trn2) and Inferentia (Inf1/Inf2) our cloud-scale Machine Learning accelerators. This role is for a Machine Learning Engineer on one of our AWS Neuron teams:

- The ML Distributed Training team works side by side with chip architects, compiler engineers and runtime engineers to create, build and tune distributed training solutions with Trainium instances. Experience with training these large models using Python is a must. FSDP (Fully-Sharded Data Parallel), Deepspeed, Nemo and other distributed training libraries are central to this and extending all of this for the Neuron based system is key.
- ML Frameworks partners with compiler, runtime, and research experts to make AWS Trainium and Inferentia feel native inside the tools builders already love—PyTorch, JAX, and the rapidly evolving vLLM ecosystem. By weaving Neuron SDK deep into these frameworks, optimizing operators, and crafting targeted extensions, we unlock every teraflop of Annapurna's AI chips for both training and lightning-fast inference. Beyond kernels, we shape next-generation serving by upstreaming new features and driving scalable deployments with vLLM, Triton, and TensorRT—turning breakthrough ideas into production-ready AI for millions of customers.
- The ML Inference team collaborates closely with hardware designers, software optimization experts, and systems engineers to develop and optimize high-performance inference solutions for Inferentia chips. Proficiency in deploying and optimizing ML models for inference using frameworks like TensorFlow, PyTorch, and ONNX is essential. The team focuses on techniques such as quantization, pruning, and model compression to enhance inference speed and efficiency. Adapting and extending popular inference libraries and tools for Neuron-based systems is a key aspect of their work.

Key job responsibilities

You'll join one of our core ML teams - Frameworks, Distributed Training, or Inference - to enhance machine learning capabilities on AWS's specialized AI hardware. Your responsibilities will include improving PyTorch and JAX for distributed training on Trainium chips, optimizing ML models for efficient inference on Inferentia processors, and collaborating with compiler and runtime teams to maximize hardware performance. You'll also develop and integrate new features in ML frameworks to support AWS AI services. We seek candidates with strong programming skills, eagerness to learn complex systems, and basic ML knowledge. This role offers growth opportunities in ML infrastructure, bridging the gap between frameworks, distributed systems, and hardware acceleration.

About The Team Annapurna Labs was a startup company acquired by AWS in 2015, and is now fully integrated. If AWS is an infrastructure company, then think Annapurna Labs as the infrastructure provider of AWS. Our org covers multiple disciplines including silicon engineering, hardware design and verification, software, and operations. AWS Nitro, ENA, EFA, Graviton and F1 EC2 Instances, AWS Neuron, Inferentia and Trainium ML Accelerators, and in storage with scalable NVMe, are some of the products we have delivered, over the last few years.

Basic Qualifications

- To qualify, applicants should have earned (or will earn) a Bachelors or Masters degree between December 2022 and September 2025.
- Working knowledge of C++ and Python
- Experience with ML frameworks, particularly PyTorch, Jax, and/or vLLM
- Understanding of parallel computing concepts and CUDA programming

Preferred Qualifications

- Open source contributions to ML frameworks or tools
- Experience optimizing ML workloads for performance
- Direct experience with PyTorch internals or CUDA optimization
- Hands-on experience with LLM infrastructure tools (e.g., vLLM, TensorRT)

Amazon is an equal opportunity employer and does not discriminate on the basis of protected veteran status, disability, or other legally protected status.

Los Angeles County applicants: Job duties for this position include: work safely and cooperatively with other employees, supervisors, and staff; adhere to standards of excellence despite stressful conditions; communicate effectively and respectfully with employees, supervisors, and staff to ensure exceptional customer service; and follow all federal, state, and local laws and Company policies. Criminal history may have a direct, adverse, and negative relationship with some of the material job duties of this position. These include the duties and responsibilities listed above, as well as the abilities to adhere to company policies, exercise sound judgment, effectively manage stress and work safely and respectfully with others, exhibit trustworthiness and professionalism, and safeguard business operations and the Company's reputation. Pursuant to the Los Angeles County Fair Chance Ordinance, we will consider for employment qualified applicants with arrest and conviction records.

Our inclusive culture empowers Amazonians to deliver the best results for our customers. If you have a disability and need a workplace accommodation or adjustment during the application and hiring process, including support for the interview or onboarding process, please visit

<https://amazon.jobs/content/en/how-we-hire/accommodations> for more information. If the country/region you're applying in isn't listed, please contact your Recruiting Partner.

Our compensation reflects the cost of labor across several US geographic markets. The base pay for this position ranges from \$99,500/year in our lowest geographic market up to \$200,000/year in our highest geographic market. Pay is based on a number of factors including market location and may vary depending on job-related knowledge, skills, and experience. Amazon is a total compensation company. Dependent on the position offered, equity, sign-on payments, and other forms of compensation may be provided as part of a total compensation package, in addition to a full range of medical, financial, and/or other benefits. For more information, please visit <https://www.aboutamazon.com/workplace/employee-benefits>. This position will remain posted until filled. Applicants should apply via our internal or external career site.

Company - Annapurna Labs (U.S.) Inc.

Job ID: A2956023

Job Assistant

Role: You are my comprehensive job search assistant with technical expertise and strong writing/communication skills.

Scope: Help me draft ALL job search communications including:

- Emails to recruiters and hiring managers
- Referral requests
- Interview question responses
- Follow-up messages
- LinkedIn outreach
- Cover letters
- Any other job-related correspondence

Core Requirements:

- Only use information I explicitly provide about my experience, projects, and background
- Never fabricate or assume details not given
- Be completely honest and factual
- Structure communications for maximum impact
- Tailor everything to the specific job description provided

My Input:

- Job descriptions
- My experience/project details
- Specific questions or communication needs
- Target audience (recruiter, HM, employee for referral, etc.)

Your Output:

- Well-crafted, professional communications
- Clear, compelling language
- Technically accurate content
- Flag any missing context you need from me

Ready to assist with any job search communication task using only the context you provide.

New Grad Machine Learning Engineer Interview Guide

New Grad Machine Learning Engineer Interview Guide

PLEASE READ EVERYTHING FIRST

Special thanks to [Ankit Garg](#) and [Aditya Kompella](#).

Table of Contents

[Purpose of this guide](#)

A brief overview of why this guide was created and what it includes.

[Advice](#)

General advice that is updated regularly as new thoughts arise.

[Overall Study Structure](#)

An outline of key study areas to focus on.

[Data Structures and Algorithms](#)

[Machine Learning foundations and Neural Networks](#)

Topic List: I first give a list of topics you should study.

Recommended Resources: I give some quality resources that cover some of the topics in the Topic List but not all.

[Coding ML Algorithms](#)

I give a list of ML algorithms you should practice coding from scratch.

[Your Resume](#)

I give some advice on studying concepts from your resume.

I then give resources to study computer vision since that is my main background on my resume.

[Role-Specific Concepts](#)

[Watch Mock Interviews](#)

Random Resources

Just some random resources I found to be useful. [Continuously updated].

Purpose Of This Guide

I wanted to create this guide because of 2 reasons.

Targeted Study Approach: One of the most challenging aspects of preparing for a machine learning (ML) interview is knowing what to study. The field of ML is vast, and it's impossible to cover everything within the limited time available for preparation. In this guide, I'll share the topics that I believe you should know as a generic ML engineer (I'll also talk a little bit about the specific field I studied beyond generic ML). I'll also share some resources that I used.

Quality Resources: ML is a hot topic right now, with countless blog posts explaining various concepts. However, many of these resources are superficial and fail to delve into the necessary mathematical details—details you need to understand to succeed in interviews. This makes studying for ML interviews even more difficult. In this guide, I'll recommend some high-quality resources that I believe are truly helpful.

Disclaimers:

I provide resources that I used to study, which may not work for you. This guide expects that you have a background in ML and have taken numerous courses on ML during your higher education. Unfortunately, this guide is not ideal for cramming.

Advice

Ask Your Recruiter: Make sure to ask your recruiter about the specific areas that will be covered in the interview. Some companies might focus on coding LeetCode-style questions, while others may emphasize ML theory and coding ML algorithms. In some cases, you may even receive a list of topics to study from the company—if you do, make sure to focus on those areas, as they will likely be the focus of the interview.

Challenge Your Understanding: When you learn a new concept, challenge yourself to ensure you truly understand it. For instance, if someone tells you that skip connections help with gradient flow and enable deeper networks,

take the time to write out the math for the forward propagation of a 5-layer network both with and without skip connections. Observe the differences in the final output layer. Then, work through the full backpropagation process. This exercise will help you grasp why ResNets are so successful. Similarly, ask yourself questions like, “Why would someone use L1 loss instead of L2 loss?” Continuously questioning and exploring these concepts will deepen your understanding.

Seek High-Quality Resources: When searching for resources to study a topic, start with university lecture notes. Look for lecture notes and YouTube lectures from institutions like Stanford, MIT, Cornell, and the University of Toronto. While this approach might not always be the most time-efficient, it often provides a more thorough understanding than articles from Medium or similar sources, which sometimes skip over the critical mathematical details and derivations.

Overall Study Structure

Here are the key categories you should focus on during your study. I'll dive into the details and share resources for each section later. These are **not** listed in order of importance:

Data Structures and Algorithms (LeetCode-style questions): Check with your recruiter or interviewers if these types of questions will be asked. Some companies include them, while others focus solely on ML.

Machine Learning Foundations and Neural Networks: These are the core ML topics often covered in introductory courses, like Bias-Variance trade-off, SVMs, etc., that are easy to forget after exams. Then study neural networks, deep learning, and optimizations.

Coding ML Algorithms: Practice coding algorithms from scratch in NumPy and PyTorch/TensorFlow. This helps solidify your understanding and prepares you for coding questions.

Your Resume: Review the concepts and technologies listed on your resume in depth, as you'll need to discuss them thoroughly in interviews.

Role-Specific Concepts: Study the specific technologies and concepts relevant to the position you're applying for.

Watch Mock Interviews: There are plenty of mock interviews on YouTube that I'll share with you. Don't skip this step—it's incredibly helpful for improving your ability to articulate ML concepts, ask insightful questions, and get a feel for the types of questions different companies ask. You can also learn a lot about ML concepts directly from these interviews.

A lot of the videos cover **ML Systems Design**.

Lastly, although not a separate category, your **mathematical background in statistics, calculus, and linear algebra** is crucial for understanding ML concepts. Learning the math on a learn-as-you-go basis as you study ML topics works well.

Data Structures and Algorithms

Learning the basics:

If you need a refresher I'd look through [Google's Data Structures and Algorithms Course](#). It's simple and you can get it done pretty quickly.

Practice:

Checkout [NeetCode's Website](#). Click on Blind 75. Do every single Easy and Medium problem on this section. Make sure to login to save your progress. He has amazing video solution explanations.

FYI: Blind 75 is a popular set of 75 leetcode problems that covers most algorithmic patterns.

NeetCode writes his solutions in Python, but you can translate his explanations to other languages.

If you have more time you can take a look at the hard questions.

If you have more time you can click on the NeetCode 150 tab. This is the original Blind 75 questions plus an additional 75 questions hand picked by NeetCode.

In addition to the above, I would recommend doing the following:

Leetcode 1570: Dot Product of Two Sparse Vectors.

This is a premium problem but you can find it for free. Code Dijkstra's shortest path algorithm. This surprisingly does not show up in Blind 75 but I was asked this in my interview.

Machine Learning Foundations and Neural Networks

Topic List:

Below is a list of topics you should focus on. In the next section, "Recommended Resources," I provide resources for learning these topics. The topics highlighted in yellow are covered in the recommended resources section. For topics that are not highlighted, you'll need to find additional explanations online or through other means.

Classification

- Linear Classification (aka Logistic Regression)

- Binary Cross Entropy Loss

- Cross Entropy Loss

- Sigmoid Function

- Softmax

- Focal Loss (optional) (how does this help class imbalance?)

- Cross Entropy (as a loss function *and* information theory perspective)

- Entropy

- Cross entropy

- KL divergence

Regression

- Linear Regression. Closed-form solution of Linear Regression.

- L2 loss, pros and cons

- L1 loss, pros and cons

- When to use L1 or L2 loss

Bias and Variance

- Bias Variance tradeoff

SVM's

- Hinge Loss

Maximum Likelihood Estimation

- Maximum A Posterior (optional)

Train/Validation/Test Sets

PCA

K nearest neighbors

Curse of dimensionality

Clustering

K means

How to pick K. When do you know to stop?

Gaussian Mixture Models

Anomaly detection (optional)

Decision trees

Algorithm to construct decision trees

Regression vs classification

Gini index

Information gain

Entropy

What is an ensemble?

Bagging

Prove why bagging reduces variance.

Random Forests

Boosting

AdaBoost

Gradient Boosting

XG boost (optional)

What is overfitting? How to reduce it?

How to deal with imbalanced data

Difference between generative and discriminative models.

Metrics

Confusion Matrix

Accuracy

Precision

Recall

F1 score

[Why do we use the harmonic mean?](#)

ROC curve. AUC

There is an interpretation of the AUC value. What is it?

Average precision (AP) and mean average precision (mAP)

[mAP for Object Detection](#)

Neural Networks

Universal Approximation theorem for neural networks, intuition (optional)

Back propagation

Activation functions

Sigmoid

Tanh

ReLU

Leaky ReLU

Weights Initialization

Gradient Descent

Mini batch vs full batch vs stochastic.

Why normalize inputs?

Vanishing/exploding gradient problem

Batch normalization

Dead neuron problem

Local Optima Problem

Local minimum vs saddle point. Which is more common?

Dropout

L1 vs L2 regularization

Why Deep Representation in Neural Networks

Adam optimizer

Momentum

RMS Prop

Skip connections

ResNet

NumPy

Why are numpy array computations faster than python for loops?

Broadcasting

Vectorization.

PyTorch

How does PyTorch compute gradients? (optional)

RNN, LSTM

What are the pros and cons of RNN's and LSTM's

Transformer Architecture

How do Positional embeddings work?

Covariate, Label, and Concept shift

Transfer Learning

Fine tuning

Linear Probing

Debugging Neural Network Training

Recommended Resources:

These resources cover some of the topics BUT NOT ALL. So you may have to do some of your own research. I have listed which topics are covered in the “**Topics Covered**” section of *each* resource.

The famous Stanford course notes [Stanford CS231](#)

Module 1 is a great introduction to major concepts in Neural Networks.

Read all of it.

Module 2 goes into convolutional neural networks, so if you are not in computer vision, you can skip or at least skim through.

Topics Covered: Classification, Linear Classification, Cross Entropy Loss, Sigmoid Function, Softmax, SVM's, Hinge Loss, Train/Validation/Test Sets, K nearest neighbors, What is overfitting? How to reduce it? Neural Networks, Activation functions, Sigmoid, Tanh, ReLu, Leaky ReLu, Weights Initialization, Gradient Descent (Mini batch vs full batch vs stochastic), Dropout, L1 vs L2 regularization, Back propagation,

Andrew Ng's youtube playlist (selected videos): [Neural Networks and Deep Learning](#)

Start from video 16. No need to watch the first 15 videos as they are covered in item 1).

You can watch all of the videos from 16 onward to be safe but there is some overlap between this playlist and item 1) so you can skim quickly through some videos you believe you already understand.

The reason I think these videos are helpful is because he writes out the math behind the vectorized forward and backward passes, which really helps you do it yourself. Vectorization can get a bit tricky if you haven't done it in a while.

Topics Covered: Numpy, Broadcasting, Vectorization (Linear classification/Logistic regression and Neural Network), Back propagation, Why Deep Representation in Neural Networks, Weights Initialization, Activation Functions.

Andrew Ng's youtube playlist: [Improving Deep Neural Networks](#).

Go through all the videos.

Topics covered: Train/Validation/Test Sets, Bias Variance tradeoff, L2 Regularization, Dropout, Why normalize inputs? For neural network training, Batch Normalization, Vanishing/exploding gradient problem, Weights Initialization, Mini Batch Gradient Descent, Adam optimizer, Momentum, RMS Prop, Softmax, Local Optima Problem.

[A Short Introduction to Entropy, Cross-Entropy and KL-Divergence](#)

(information theory)

This is a great intuitive video on entropy from an information theory perspective.

Topics Covered: Entropy, Cross entropy, KL divergence.

[Building decision Tree algorithm lecture](#)

This lecture is an amazing *gentle* introduction to decision trees and how to build them. THERE IS MUCH MORE to decision trees so watching this will not complete your knowledge but I think it's a great start and will help support your understanding for the next item on this list.

Topics Covered: Decision Trees, Algorithm to construct decision trees.

[Cornell's CS4780](#)

If you have the time, I would honestly go through the whole course; it's absolutely amazing. But no one has that much time so here are the lectures I'd go through given that you did item 1) 2) and 3).

Lectures 19-21: Bias Variance, Model Selection / Regularization / Overfitting.

Lectures 28-34: All about decision trees and ensemble methods (random forests, boosting etc.).

I know this is a lot but trust me, it's worth it. I have not found a better explanation of the theory behind boosting methods.

Topics covered: Bias Variance Tradeoff (more formal), Regularization, Model Selection, Overfitting, Decision Trees, Gini Index, Information gain, Entropy, Bagging, Random Forests, Boosting, AdaBoost, Gradient Boosting.

PCA Stuff

[StatQuest: Principal Component Analysis \(PCA\)](#)

This video does not go into the proof of why SVD gives us the principle components or much into theoretical Linear Algebra but it gives great intuition behind PCA.

[StatQuest: PCA - Practical Tips](#)

Topics covered: PCA

Maximum Likelihood Estimation:

This topic can be a bit confusing for some people so I will give a couple of resources:

[University of Toronto CSC311 Lecture 7 Notes](#)

This is good enough of a review. These notes also cover Maximum A Posteriori.

Topics covered: Maximum Likelihood Estimation, Maximum A Posteriori.

If you are still a bit confused, watch this video: [Likelihood Estimation](#).

Topics covered: Maximum Likelihood Estimation

Additional resources:

[Cornell's CS4780](#) Lecture 7

[Stanford Notes](#)

[Google for Developers - Machine Learning](#)

I think this is a great resource for certain concepts I have listed below. The downfall of the material is that some concepts are just not explained with enough mathematical rigor or derivation. I encourage you to skim through what they have to offer. If anything seems important or interesting, take a look!

Advanced Courses - Clustering

Topics Covered: K means, Curse of dimensionality.

Machine Learning Crash Course - Data

The "Data" section is great for understanding concepts in data preparation.

Topics Covered: Data preprocessing, why normalize inputs?

Skip Connections and Resnet

[ResNets](#)

[Why ResNets Work](#)

Topics Covered: Skip connections, ResNet.

Transformer architecture

[The Illustrated Transformer](#)

Great article. Great start to understanding Transformers.

Position Embeddings:

[Position Embeddings](#)

[Positional Encoding in Transformer Neural Networks Explained](#)

Debugging Neural Network Training

Understand the common errors that come up in model training and how to set up training pipelines. Learn tips and tricks to fix errors that come up in model training.

Great article by Andrej Karpathy:

<https://karpathy.github.io/2019/04/25/recipe/>

Topics covered: Debugging Neural Network Training

Coding ML Algorithms

I would suggest coding these algorithms in NumPy from scratch. You can use ChatGPT for help. Try to fully vectorize as much as you can; minimize for loops.

K Nearest Neighbors

K Means

Linear Regression

L1 and L2 Loss

Single Forward and Backward Pass

Linear Classification (Logistic Regression)

Multi Class Softmax Cross Entropy

Binary Cross entropy

Single Forward and Backward pass

2 Layer Perceptron

Single Forward and Backward pass

Simple Neural Network in PyTorch. Write a simple training loop.

Your Resume

This is one of the most critical parts of your interview preparation. During my interviews, the interviewers asked very detailed questions about my previous work experience and projects, so you need to be thoroughly familiar with your past work.

Make sure you can explain your past projects and experiences clearly and concisely.

Practice speaking out loud, and try explaining them to others, whether friends, family, or colleagues.

Here are some questions you might encounter:

Problem: What problem did you solve? How did you select the appropriate features? How did you decide which ML algorithms to use?

Algorithm: Can you provide an in-depth explanation of the ML algorithm you used?

Datasets: What datasets did you use? Did you have to create your own datasets? How large were they? Was there a class imbalance? How did you handle the class imbalance?

Infrastructure: What was your training and inference infrastructure? What resources did you have access to? Did you use multiple GPUs or multiple nodes to train?

Evaluation: What metrics did you use to evaluate your solutions?

Computer Vision:

My main background is in computer vision so I will provide some of the resources I used to study computer vision basics. I did not do prior work in generative models so I did not study those.

Classical computer vision (this is mainly for computer vision roles. If you are interviewing for a non computer vision role then they will probably not ask this stuff):

[First Principles of Computer Vision](#)

This channel is amazing.

I went through the [Camera Calibration | Uncalibrated Stereo](#) playlist.

Depending on your role, you can go through some of the other playlists as necessary.

Convolution Neural Networks:

[Convolution Neural Networks Andrew Ng](#)

Beautiful playlist. Watch all the videos.

Segmentation models

UNET

DeepLab series

Mask RCNN

SegFormer

Object Detection

YOLO

RCNN (Faster RCNN)

[Coding Faster RCNN from scratch](#)

I learned so much from this article, especially about how anchor boxes truly work.

Anchor Boxes.

Feature Pyramids in object detection

Focal Loss

[Yannick Kilcher](#)

This guy's youtube channel is phenomenal. He explains recent impactful papers such as BERT, Vision Transformers etc. He's not the best for a full in depth understanding but he's a great start to get a decent idea of what's going on in the paper. I usually watch his videos before reading any popular paper publication.

[Vision Transformer](#)

[DETR](#)

Deformable DETR

Video based models (Action recognition: SlowFast, MViT)

Code a 2d convolution in NumPy

Role-Specific Concepts

I don't have extensive advice for this section, but here are a few key points:

Understand the Role Requirements: Make sure you understand the specific concepts mentioned in the job description. These are likely to come up during the interview.

Address Gaps in Your Experience: If there are concepts required for the role that aren't on your resume, the interviewers may not expect you to have an in-depth understanding. However, you should still have a high-level grasp of these concepts. Do some research to familiarize yourself with the basics.

For example, I was interviewing for an ML engineer position on a recommendations team, but I have no prior background or research in recommender systems. I simply read over the online course [Google for Developers - Recommendation Systems](#) and watched some recommendation systems design mock interviews; this was sufficient for my interview.

Watch Mock Interviews

Watching mock interviews is a great way to get an idea of what questions are asked in interviews, how to verbally answer these questions concisely, and learn about ML Systems Design.

[Exponent](#)

This is a great channel with lots of ML Engineer mock interviews.

However, most of the videos are big tech company interviews.

[Jay Feng](#)

This channel also has some great mock interviews.

Random Resources

(Feel free to suggest some resources!)

[Machine Learning Research & Engineering - Flashcards](#)

[Chip Huyen's ML Interviews Book](#)

Tab 71

Asdasdsasdsaasdsaas
erqrweeweqweqeqewqewqqew

Tab 59

Mrin

☰ Mrin LP Stories Based on Resume Frontend

✚ Mrinmayee Question and Answers

☰ Frontend Updated Resume - 2025 (mr9798)

☰ Mrin LP Stories Based on Resume Frontend

Your You.com AI Engineer Interview Preparation Guide

For 20-minute HR screening with Justin McQueen on October 13, 2025

PART 1: COMPANY INTELLIGENCE - YOU.COM DEEP DIVE

What You.com Does (The Elevator Pitch)

You.com is the API infrastructure company powering the agentic era. They're not primarily competing for consumer search—they're building the highways that AI agents need to access real-time web intelligence.

Core Business Model:

- **Enterprise AI APIs** (primary revenue): Search API, News API, RAG API processing **1B+ queries monthly**
- **Enterprise AI Platform**: Full-stack solution for building custom AI agents
- **Consumer Product**: AI productivity engine (you.com) with 20+ LLM models

The "Productivity Engine" in Practice:

- Combines real-time web search with multiple LLMs (GPT-4, Claude, Gemini, etc.)
- **Model-agnostic architecture**: Automatically selects best model for each task
- **Citation-backed answers**: Nearly eliminates hallucinations through source verification
- **Private RAG**: Integrates company data (Google Drive, SharePoint, Databricks, Slack)
- **Think**: If ChatGPT is a car, You.com is building the roads, gas stations, and traffic systems

Differentiation from Competitors

Competitor	Focus	You.com's Advantage
Perplexity AI	Consumer answer engine (\$20/month)	B2B API platform + 15 models vs. 6; cheaper at \$15/month
ChatGPT Search	Integrated search in ChatGPT	Model-agnostic (not locked to OpenAI); purpose-built for search
Google/Bing APIs	Traditional search APIs	LLM-optimized long snippets; better accuracy (77.8% SimpleQA score)
OpenAI Deep Research	AI research tool	76% win rate in head-to-head comparisons; 3x more citations

Key Differentiators:

1. **Composable APIs**: Mix-and-match LLMs, data depth, integration styles—no vendor lock-in
2. **Enterprise-grade**: SOC 2, zero data retention, 57% of Fortune 500 use their platform

3. **Speed + Accuracy:** 445ms p50 latency while outperforming competitors on accuracy benchmarks
4. **Agentic Infrastructure:** Built for AI agents to use as tools, not just humans

"Composable APIs for the Agentic Era" - Technical Meaning

Composable APIs: Modular building blocks developers can flexibly combine

- Choose your LLM (GPT-4, Claude, Gemini, Llama, etc.)
- Choose your data depth (Standard, Pro, Deep Research)
- Choose your integration (REST API, Python SDK, Databricks Unity Catalog)
- Mix public web data + private RAG

Agentic Era: Shift from passive chatbots to **autonomous AI agents** that:

- Execute multi-step workflows independently
- Use tools and APIs to take actions
- Coordinate with other agents
- Maintain context across sessions

You.com's Role: They're "the highways of the agentic era"—providing the infrastructure agents need for real-time web access, verifiable cited information, tool integration, and hybrid public + private data access.

Search + AI Technology Stack

How They Combine Search with LLMs:

1. **Proprietary web crawler:** Indexes 10B+ pages with full-text extraction
2. **RAG (Retrieval-Augmented Generation):** Long extractive snippets from page body (not just meta tags)
3. **Multi-model routing:** Dynamic selection of best LLM per query
4. **Citation tracking:** Transparent sourcing with direct links to highlighted content
5. **Vector databases:** Semantic search combining embeddings with keyword matching
6. **Real-time pipeline:** Continuous crawling for "industry-leading freshness"

Technology Stack (Inferred):

- **Backend:** Python (primary), FastAPI/Flask for APIs
- **AI/ML:** LangChain/LlamaIndex for orchestration, HuggingFace Transformers, OpenAI/Anthropic/Google APIs
- **Infrastructure:** Databricks (confirmed), AWS (marketplace listing), Kubernetes, Redis
- **Databases:** PostgreSQL/MongoDB + vector databases (Pinecone partnership confirmed)
- **Frontend:** React, Next.js likely

Recent Achievements & Growth

\$100M Series C Funding (Announced September 4, 2025):

- **Valuation:** \$1.5 billion (unicorn status achieved)
- **Lead Investor:** Cox Enterprises
- **Participating:** Georgian, Salesforce Ventures, Norwest Venture Partners
- **Total Raised:** \$195M+ (seed: \$20M from Marc Benioff; Series A: \$25M; Series B: \$50M)
- **Use of Funds:** Double headcount from 100 → 200 employees, open SF office, expand NYC presence

1B+ Queries Monthly: VERIFIED—processing over 1 billion API calls monthly (confirmed in Series C announcement)

Growth Trajectory:

- ARR grew **500% from January 2024 to September 2024**
- 100,000+ custom agents built on platform since late 2024

Recent Product Launches (Last 6 Months):

1. **ARI Enterprise** (May 15, 2025):
 - Professional-grade deep research agent
 - Processes 500+ sources simultaneously (vs. 30-40 for competitors)
 - **76% win rate over OpenAI Deep Research**
 - **80% accuracy on FRAMES benchmark** (Harvard/Google/Meta standard—industry-leading)
 - 3x faster results, delivers polished PDF reports in ~5 minutes
2. **ARI** (February 27, 2025):
 - Functions as on-demand AI consultant
 - Research that typically costs \$10K-\$100K and takes weeks
 - 10x more sources than competitor tools

Major Customers:

- DuckDuckGo, Databricks, Harvey AI, Windsurf, OpenAI, Amazon, NIH
- **57% of Fortune 500** use their platform

Company Culture & Values

Work Environment:

- **Remote-first** with hubs in SF, NYC, Ontario
- Currently **~100 employees**, scaling to **200** with Series C funding
- **Glassdoor**: 3.8/5 stars, 76% would recommend to a friend

Employee Reviews Highlights:  Excellent engineering culture, no micromanagement, cutting-edge AI work, efficient recruitment  Startup intensity, need to be self-starter, workload fluctuations

Core Values: Accuracy & Trust, Privacy & Security, Innovation, Transparency, Collaboration

Benefits:

- 100% health insurance for policyholder, 75% for dependents
- 12 weeks paid parental leave (US)
- \$500 WFH stipend + \$1,200 annual health & wellness allowance
- 401k with 3% match (immediate vesting)
- Flexible PTO + 11 US holidays + December shutdown

Leadership Team

Richard Socher (Co-founder & CEO):

- Former **Salesforce Chief Scientist** and EVP
- Stanford PhD in Computer Science—won best CS thesis award

- **4th most-cited researcher in NLP** with 220,000+ citations
- Co-invented word vectors, contextual vectors, and prompt engineering
- Time100 AI list, World Economic Forum Young Global Leader

Bryan McCann (Co-founder & CTO):

- Former Lead Research Scientist in NLP at Salesforce Research
- **First author of decaNLP paper that pioneered prompt engineering (2018)**
- Holds patent on contextualized word vectors
- Wrote first line of code for You.com

Company History:

- **2020:** Founded by Socher + McCann
- **Nov 2021:** Public beta launched with \$20M seed from Marc Benioff
- **Dec 2022:** **First search engine to integrate ChatGPT-style chatbot with live web results**
- **2024:** Strategic pivot to **enterprise AI infrastructure**
- **Sep 2025:** \$100M Series C, unicorn status

Essential Terminology

"Composable APIs": Modular, flexible API components developers mix-and-match. No vendor lock-in—swap models, data sources, integration styles as technology evolves.

"Agentic Era": Shift from passive AI assistants to autonomous agents that reason, plan, make decisions, and take actions. Examples: coding agents (Windsurf), legal research agents (Harvey), enterprise workflow agents (Databricks).

RAG (Retrieval-Augmented Generation): AI framework connecting LLMs to external knowledge bases before generation. Eliminates hallucinations by grounding answers in real sources.

LLM Infrastructure: Backend systems supporting large language model applications—model hosting, routing, context management, caching, monitoring, cost optimization, guardrails.

Vector Databases: Store embeddings (numerical representations of text) enabling semantic search (meaning-based, not just keyword matching).

PART 2: ROLE-SPECIFIC PREPARATION

AI Engineer Role Requirements

"2+ years improving backend and/or AI systems" means:

- Building scalable **APIs** (RESTful, GraphQL) handling high query volumes
- **LLM integration and deployment** (OpenAI, Anthropic, open-source models)
- **RAG implementation:** Vector search, embeddings, retrieval optimization
- **Production ML infrastructure:** Monitoring, evaluation, cost management
- **Distributed systems** and microservices architecture

"Deep understanding of systems architecture" means:

- **LLM orchestration:** Multi-model routing, context management, token optimization
- **Search infrastructure:** Real-time crawling/indexing, ranking algorithms, citation tracking
- **Data pipelines:** ETL/ELT, vector database integration, embeddings generation
- **Scalability patterns:** Horizontal scaling for inference, load balancing, async processing

"Experience shipping and scaling AI products" means:

- Taking AI from **prototype** → **production** with A/B testing, evaluation frameworks
- **Handling millions of requests** (You.com does 1B+ monthly)
- **Optimizing latency** (sub-second response times) while maintaining accuracy
- **Cost management at scale** and monitoring model performance

Python Expertise Expected:

- Advanced Python 3.x (async/await, type hints, decorators)
- FastAPI/Flask for API development
- AI libraries: LangChain, LlamaIndex, HuggingFace Transformers
- OpenAI, Anthropic, Cohere SDKs
- Production-quality code with proper error handling, logging, testing

Backend Development Expected:

- Database design (PostgreSQL, MongoDB, vector DBs)
- Authentication/authorization, caching strategies, message queues
- Docker, Kubernetes, AWS/GCP/Azure
- CI/CD pipelines

Team Structure

Engineering Team (~50-60 of 100 employees):

- AI/ML Engineers (core LLM infrastructure)
- Backend Engineers (APIs, search infrastructure)
- Full-stack Engineers (product experiences)
- Data Engineers (pipelines, vector databases)
- Research Scientists (model optimization)
- Forward Deployed Engineers (enterprise customers)

What AI Engineers Work On:

1. Core LLM infrastructure (multi-model routing, optimization)
2. Search API development (real-time web search integration)
3. RAG systems (vector database integration, chunking strategies)
4. Agent systems (multi-step reasoning, tool use, function calling)
5. Product features (chat interfaces, summarization, research automation)

Salary & Compensation

Confirmed Ranges from Job Postings:

- **Staff AI Engineer** (7+ years): \$200K-\$290K
- **Senior AI Engineer, Enterprise Solutions** (5+ years): \$200K-\$270K
- **Estimated Mid-Level** (2-4 years): \$160K-\$200K (based on market data)

With Equity: Total comp likely \$200K-\$280K for mid-level including equity (0.1-0.5% at \$1.5B valuation)

PART 3: 20 BATTLE-TESTED Q&A RESPONSES

1. "Tell me about yourself"

Response (90 seconds):

"I'm an AI Engineer with 4+ years of experience building production AI systems at scale. I got into AI during my undergrad through machine learning competitions and was immediately hooked by the challenge of making models work reliably in the real world.

Currently, I'm an AI Engineer at RIT's Office of VP Research, where I built an AI-powered NLP agent that automates federal research submissions—we're processing over 800 proposals through this system. I also developed an ML workload management system and automated budget pipeline that's dramatically improved efficiency.

Before this, I was at Daiichi Sankyo where I built document classification systems optimized with ONNX for clinical trials, and I worked with Mistral 7B for conversational clinical data analysis. Earlier, I was the founding AI Infrastructure Engineer at SEO Content AI, where we scaled to 5,000 users in the first month and processed 50,000+ articles monthly using AWS infrastructure.

What excites me about You.com is your work on composable APIs for the agentic era. I've been following your ARI Enterprise launch and the 76% win rate over OpenAI's Deep Research—that's exactly the kind of accurate, scalable AI infrastructure I want to build. Your model-agnostic architecture and focus on eliminating hallucinations through RAG aligns perfectly with my experience deploying production AI systems at scale."

2. "Why You.com?"

Response (90 seconds):

"Three things really drew me to You.com. First, you're solving the hard problems in AI that I'm passionate about—hallucination elimination through RAG, scaling to 1 billion queries monthly, and building infrastructure for the agentic era. When I read your Series C announcement and learned about processing 1B+ API calls monthly, I immediately thought, 'This is the scale I want to work at.' My experience processing 800+ proposals at RIT and 50,000+ articles monthly at SEO Content AI has prepared me for exactly these challenges.

Second, your technical approach is exactly right. I'm a huge believer in model-agnostic architectures—I've integrated GPT-3.5, BERT, FLAN-T5, Mistral 7B, and Gemini across different projects, and I've seen how vendor lock-in limits innovation. Your composable API strategy where developers can mix-and-match LLMs is the future, and your 77.8% accuracy on SimpleQA proves it works at scale.

Third, I'm really impressed by your leadership. Richard Socher pioneered contextual word vectors and prompt engineering—foundational work that's enabled everything we're doing in AI today. The fact that you've achieved 76% win rate over OpenAI's Deep Research shows you're not just building another search engine; you're building the infrastructure layer that powers the next generation of AI applications. That's exactly where I want to be."

3. "Walk me through your experience with AI systems at scale"

Response (2 minutes):

"I've built AI systems at scale across multiple domains. Let me give you three examples:

At RIT (current role), I built an AI-powered NLP agent for federal research submissions handling 800+ proposals. The challenge was accuracy—federal submissions have zero tolerance for errors. I implemented a multi-stage validation pipeline with GPT-based natural language understanding combined with rule-based verification. We're processing hundreds of proposals weekly with 99.2% accuracy. I also built the ML workload management system that routes computational tasks across our GPU clusters—it's a distributed system challenge similar to what You.com likely faces routing queries across different LLM providers.

At SEO Content AI (founding engineer), I built the infrastructure that scaled to 50,000+ articles monthly with 5,000 users in the first month. This was pure scale engineering—AWS Lambda and ECS for compute, DynamoDB for state management, S3 for content storage. The challenge was cost optimization while maintaining sub-5-second generation times. I implemented smart caching, model selection strategies, and async processing pipelines. We secured \$50K+ in contracts based on this infrastructure's reliability.

At White Label Resell, I built an AI content automation platform generating 10,000+ articles monthly in 50+ languages. I integrated multiple AI models (GPT-3.5, BERT, FLAN-T5, Stable Diffusion) depending on the task—kind of like a primitive version of your model-agnostic routing system. The key learning was that different models excel at different tasks, and intelligent routing can dramatically improve both cost and quality.

These experiences taught me how to build AI systems that are accurate, scalable, and cost-effective—exactly what You.com's 1B+ queries monthly requires."

4. "Tell me about a time you shipped an AI product"

Response (90 seconds):

"My favorite example is ExpenseGenie Bot—a Telegram and WhatsApp bot I built using Gemini LLM that helps users track expenses through natural conversation. Think of it like having a personal accountant in your messaging app.

The challenge was making AI reliable enough for financial data. Users can't afford errors in expense tracking. I implemented a three-layer architecture: Gemini for natural language understanding, Redis for state management to maintain conversation context, and microservices for different expense operations.

I shipped this from idea to production in three weeks. The key decisions were choosing Gemini for its multimodal capabilities and speed, using Redis for sub-100ms state retrieval, and deploying on a microservices architecture so I could scale individual components independently.

The result: users can say 'I spent \$45 on dinner with clients' and it automatically categorizes it, extracts the amount, and adds it to their budget. It handles complex queries like 'show me all food expenses over \$50 last month.'

What I learned from shipping this mirrors what You.com does—the hard part isn't getting an LLM to work in a demo; it's making it reliable, fast, and accurate enough for production use with real users. That's why your focus on citations, evaluation frameworks, and accuracy benchmarks resonates with me."

5. "How do you approach system architecture for AI applications?"

Response (2 minutes):

"My approach follows a few core principles:

First, start with non-negotiables. At Daiichi Sankyo working with clinical trials data, our non-negotiables were HIPAA compliance, zero data leakage, and model explainability. I architected our document classification system with ONNX optimization for on-premise deployment—no cloud APIs touching patient data. For You.com's enterprise customers, I imagine similar non-negotiables around SOC 2 compliance and zero data retention.

Second, separate concerns aggressively. At SEO Content AI, I built clear separation: Lambda functions for stateless inference, ECS for stateful workflows, DynamoDB for orchestration state, S3 for content. When we needed to optimize costs, I could swap the inference layer without touching orchestration. I'd apply similar thinking to You.com's architecture—separate retrieval, LLM inference, citation extraction, and result composition so each can scale independently.

Third, design for failure. At RIT, our federal submission system can't go down. I implemented fallback strategies: if the primary NLP model fails, fall back to rule-based extraction. If GPU clusters are saturated, queue and process asynchronously. Monitor everything—latency, accuracy, cost per query.

Fourth, optimize the right metric. At Digital Clinics, we built a breast cancer diagnosis AI. The metric wasn't just accuracy—it was F1-score because false negatives were catastrophic. We achieved 92% F1-score through distributed training with Horovod processing 1.5TB of medical images weekly. For You.com, I imagine the metrics are latency, accuracy, cost per query, and hallucination rate—and you have to optimize all four simultaneously.

Finally, make it observable. You can't fix what you can't measure. I instrument everything—model performance, latency percentiles, cache hit rates, error patterns."

6. "Describe your experience with LLM infrastructure"

Response (90 seconds):

"I've worked with LLM infrastructure across multiple projects and models. At RIT, I'm currently using GPT-based models for natural language understanding in our federal submission agent—the challenge is prompt engineering for reliability and managing context windows for long documents.

At Daiichi Sankyo, I worked with Mistral 7B for conversational clinical data analysis. This was interesting because we needed on-premise deployment for compliance, so I optimized the model with ONNX for inference efficiency. I learned a lot about trade-offs between model size, accuracy, and latency.

I also built ExpenseGenie Bot using Gemini LLM, where I implemented conversation state management with Redis—maintaining context across multiple turns is critical for natural interactions.

Across my work at SEO Content AI and White Label Resell, I've integrated GPT-3.5, BERT, FLAN-T5, and Stable Diffusion. The key insight is that different models excel at different tasks. I built routing logic to select the right model based on the task—kind of a primitive version of what You.com does with model-agnostic routing.

What I find exciting about You.com's approach is taking this model selection idea to the next level—dynamically routing across 15+ models at 1B+ queries monthly scale. That's the infrastructure challenge I want to tackle."

7. "What's your experience with Python and backend systems?"

Response (90 seconds):

"Python is my primary language—I've built production backend systems with it for 4+ years. At RIT, I built the entire NLP agent backend in Python with FastAPI, including the ML workload management system. I use `async/await` extensively for handling concurrent requests and type hints for code reliability.

For backend systems, at SEO Content AI I architected the full AWS infrastructure: Lambda functions for serverless inference, ECS for orchestration, DynamoDB for state management. I handle authentication with JWT tokens, implement rate limiting with Redis, and use CloudWatch for monitoring. The system processes 50,000+ articles monthly.

At White Label Resell, I built microservices in Python handling 10,000+ articles monthly in 50+ languages. This involved building ETL pipelines, RESTful APIs, database design with PostgreSQL and MongoDB, and containerization with Docker.

I'm also strong with DevOps—I've set up CI/CD pipelines with GitHub Actions, managed Kubernetes clusters, and implemented comprehensive logging and monitoring. At Digital Clinics, I worked with distributed training systems using Horovod processing 1.5TB of data weekly.

What I appreciate about You.com's stack is the focus on scalability and reliability—when you're processing 1B+ queries monthly, your backend has to be bulletproof. My experience shipping production systems at scale has taught me how to build systems that handle real-world load, failures, and edge cases."

8. "Tell me about a challenging technical problem you solved"

Response (2 minutes):

"The most challenging problem I solved was at Daiichi Sankyo when I built a document classification system for clinical trials that needed to run on-premise for HIPAA compliance, but also had to be fast enough for real-time use.

The challenge: Clinical trials generate massive amounts of documents—consent forms, adverse event reports, protocol amendments. Our scientists needed to automatically classify and route these documents, but we couldn't use cloud APIs due to patient privacy. Traditional rule-based systems were only 60% accurate.

My approach: I built a deep learning model for document classification, but the constraint was it had to run on-premise on CPU-only machines (no GPUs for inference). The base PyTorch model took 8 seconds per document—way too slow.

The solution: I optimized the model with ONNX which converts PyTorch models to a format optimized for CPU inference. I also implemented smart document chunking, model quantization (400MB to 100MB without accuracy loss), caching layer for common patterns, and asynchronous batch processing.

The result: Inference time dropped from 8 seconds to 1.2 seconds per document while maintaining 94% classification accuracy—better than cloud-based models we benchmarked against. We processed 1,000+ documents daily.

What I learned: The best solution isn't always the biggest model—it's understanding constraints (compliance, latency, cost) and making smart trade-offs. This applies directly to You.com's work:

you're optimizing for accuracy, latency (445ms p50), and cost simultaneously while processing 1B+ queries monthly. That's the kind of multi-objective optimization challenge I love solving."

9. "How do you handle scaling AI systems?"

Response (90 seconds):

"I approach scaling through three lenses: compute, data, and architecture.

Compute scaling: At SEO Content AI, when we hit 5,000 users in the first month, I migrated to AWS Lambda for stateless inference (automatically scales to thousands of concurrent executions) and ECS for stateful workflows. I also implemented smart caching with Redis—80% of queries hit cache, dramatically reducing compute costs.

Data scaling: At Digital Clinics, we processed 1.5TB of medical images weekly. I implemented distributed training with Horovod across multiple GPUs, which reduced training time from 3 weeks to 4 days. For inference, I built a data pipeline that streamed images from S3, preprocessed them in parallel, and batched them for efficient GPU utilization.

Architecture scaling: The key is designing for horizontal scalability from day one. At White Label Resell, I built microservices where each component could scale independently. When image generation became the bottleneck, we just added more Stable Diffusion workers without touching other services.

At scale, monitoring becomes critical. I instrument everything—latency percentiles, throughput, error rates, cost per query. At RIT, our ML workload management system makes routing decisions based on real-time cluster utilization.

For You.com's 1B+ queries monthly, I'd focus on intelligent caching, async processing for non-latency-sensitive queries, smart model routing, and comprehensive monitoring to catch performance degradation early."

10. "What's your experience with cloud infrastructure (AWS)?"

Response (90 seconds):

"I have extensive AWS experience. At SEO Content AI, I architected the entire cloud infrastructure from scratch:

Compute: Lambda for serverless inference, ECS for longer-running workflows, EC2 for model training with GPU instances

Storage: S3 for content storage (50,000+ articles monthly), DynamoDB for state management, RDS PostgreSQL for relational data

Networking: API Gateway for RESTful APIs, CloudFront CDN for content delivery, VPC configuration for security

Monitoring & Operations: CloudWatch for logs and metrics, X-Ray for distributed tracing, SNS for alerting, IAM for security

Cost optimization: I implemented lifecycle policies on S3, used spot instances for training, and right-sized EC2 instances. We reduced costs by 40% while scaling to 5,000 users.

At White Label Resell, I worked across AWS, GCP, and Azure, giving me perspective on multi-cloud architectures. I've also used Docker for containerization and managed Kubernetes clusters.

What I appreciate about You.com's scale is the infrastructure challenges. Processing 1B+ queries monthly requires sophisticated architecture—intelligent load balancing, multi-region deployment, robust monitoring. I'm excited to work on systems at this scale where infrastructure decisions directly impact business outcomes."

11. "Describe a time you worked autonomously on a project"

Response (90 seconds):

"My founding engineer roles required extreme autonomy. At SEO Content AI, I was brought in as the first AI Infrastructure Engineer when they had an idea but no technical product. I had complete ownership of technical strategy, implementation, and scaling.

I decided which AI models to use (GPT-3.5, BERT, FLAN-T5), what architecture (serverless AWS with Lambda + ECS), what tech stack (Python, FastAPI, Docker). I wrote the entire first version—infrastructure code, ML pipelines, API layer, monitoring, deployment scripts.

When we hit 5,000 users in the first month, I diagnosed bottlenecks, architected the scaling solution, and implemented it independently.

Result: Live platform in 6 weeks, scaling to 50,000+ articles monthly, securing \$50K+ in initial contracts.

The challenge with autonomy isn't just technical execution—it's knowing when to seek input. I regularly shared progress with stakeholders and validated technical decisions against business goals. But for day-to-day engineering decisions, I made those calls independently.

At White Label Resell, similar story—founding ML Engineer where I built the entire content automation platform from scratch. 10,000+ articles monthly in 50+ languages, multiple AI model integration, full AWS infrastructure.

What excites me about You.com is your mention of needing self-starters who thrive with autonomy. I've proven I can take ownership of complex technical problems, make good architectural decisions independently, and deliver production systems without hand-holding."

12. "What interests you about AI-powered search?"

Response (90 seconds):

"AI-powered search is solving one of the fundamental problems with LLMs—knowledge cutoffs and hallucinations. I've experienced this firsthand. When I built the NLP agent at RIT for federal submissions, I initially tried pure LLM approaches, but hallucinations were unacceptable. I had to build retrieval systems that grounded the AI in real source documents.

What You.com is doing with RAG at scale—combining real-time web search with LLM generation—is the right architecture. Your 77.8% accuracy on SimpleQA and 80% on FRAMES benchmark proves it. Traditional search gives you links but makes you do the synthesis. Pure LLMs give you synthesis but hallucinate facts. AI-powered search with proper RAG gives you both—synthesized answers grounded in real sources with citations.

The bigger opportunity is what you're building for the agentic era. AI agents need reliable real-time information to make decisions. A coding agent needs the latest React docs. A legal agent needs current case law. Without accurate, cited, real-time web intelligence, agents are flying blind.

What excites me about You.com specifically is you're not just building a consumer product—you're building infrastructure. Your APIs are powering DuckDuckGo, Databricks, Harvey AI, even OpenAI. You're the infrastructure layer enabling the next generation of AI applications. That's exactly the kind of foundational work I want to be building."

13. "How do you stay current with AI/ML developments?"

Response (60 seconds):

"I stay current through multiple channels:

Hands-on experimentation: I built ExpenseGenie Bot to learn Gemini's multimodal capabilities. I'm currently experimenting with fine-tuning smaller LLMs for domain-specific tasks at RIT. The best way to understand new technology is to build with it.

Reading research: I follow key AI researchers on Twitter, particularly Richard Socher's work. I read papers on Hugging Face Daily Papers and Arxiv. I followed your ARI Enterprise launch closely and studied your open-sourced DeepConsult benchmark—your 76% win rate over OpenAI was impressive.

Technical communities: I'm active in ML Discord servers and follow discussions on Hacker News and Reddit's r/MachineLearning. I use GitHub to track popular AI repositories.

Courses and learning: I've completed multiple Coursera specializations in deep learning and NLP. I recently finished Andrew Ng's course on LLMs and prompt engineering.

The field moves incredibly fast. What excites me about You.com is you're at the frontier—integrating new models within a day of release, pioneering composable APIs, setting benchmarks for the industry. That's the environment where I'd learn fastest."

14. "Tell me about your experience with distributed systems"

Response (90 seconds):

"I've built distributed systems at two levels: data processing and model training.

At Digital Clinics, I built a distributed training system using Horovod for deep learning on medical images. We were processing 1.5TB weekly—far too much for a single GPU. I implemented data parallelism across 8 GPUs, gradient aggregation with Horovod's ring-allreduce algorithm, distributed data loading from S3 with prefetching, and checkpointing for fault tolerance. This reduced training time from 3 weeks to 4 days and achieved 92% F1-score.

At RIT, I built an ML workload management system that routes computational tasks across GPU clusters. The challenge is load balancing—some tasks need 1 GPU for 10 minutes, others need 4 GPUs for 2 hours. I implemented priority queue system with different SLAs, resource scheduling algorithms, health monitoring, and async task distribution.

At SEO Content AI and White Label Resell, I built distributed microservices where different components ran independently and communicated via message queues. This allowed independent scaling of bottleneck services.

What I find exciting about You.com's 1B+ queries monthly is the distributed systems challenges at that scale: routing queries to the right model, load balancing across regions, handling partial failures gracefully, maintaining low latency. Those are the hard problems that get me excited."

15. "What's your approach to monitoring and evaluation of AI systems?"

Response (2 minutes):

"I've learned that monitoring AI systems requires both system health AND model quality metrics.

System health monitoring: At SEO Content AI, I instrumented everything with CloudWatch: latency percentiles (p50, p95, p99)—not just averages because outliers matter, throughput, error rates and types, cost per query, infrastructure health.

Model quality monitoring: At Daiichi Sankyo, for our document classification system, I tracked accuracy over time for model drift detection, confidence scores, human feedback loops where scientists could flag misclassifications, and category distribution to catch anomalies.

At RIT, for the federal submission agent, I implemented output validation (automated checks on AI outputs), A/B testing framework to compare different prompt strategies, cost tracking (tokens used, API costs), and hallucination detection by cross-referencing AI outputs against source documents.

Evaluation frameworks: I build eval datasets for every production AI system. At Digital Clinics, we maintained a gold-standard test set of 5,000 labeled images that we ran against every model version. This caught regressions before they hit production.

For You.com's scale, I'd focus on real-time accuracy monitoring, citation quality (are sources authoritative? are citations correct?), latency distribution by query type, cost per query by model, and user feedback signals.

The key principle: you can't fix what you can't measure. Every AI system I ship has comprehensive instrumentation from day one."

16. "Describe your experience collaborating with cross-functional teams"

Response (90 seconds):

"I've collaborated extensively with non-technical stakeholders throughout my career.

At Daiichi Sankyo, I worked with clinical trial scientists who needed document classification but didn't understand AI. I ran workshops where they explained document types, edge cases, and priorities. Then I translated that into training data requirements and model architecture. I presented results in terms of 'documents correctly classified per day' rather than 'F1-score'—metrics they cared about.

At RIT, I work with research administrators who submit federal proposals. They're not technical but have deep domain knowledge. I spent time shadowing their workflow, understanding pain points, and iteratively building the NLP agent based on their feedback. The key was showing progress weekly with demos they could test.

At SEO Content AI, I was the technical co-founder working with a non-technical CEO. I had to explain technical trade-offs in business terms: 'We can add this feature for \$5K/month in infrastructure costs, or optimize the current system to reduce costs by \$8K/month—which is higher priority?'

What I've learned: The best AI systems are built through close collaboration with domain experts. They understand the problem space better than I ever will. My job is to translate their needs into technical solutions and communicate progress clearly.

At You.com, I'd bring this same collaborative approach—whether working with product managers, sales teams, or research scientists."

17. "What do you know about composable APIs and the agentic era?"

Response (2 minutes):

"Composable APIs and the agentic era represent a fundamental shift in how we build AI systems, and You.com is at the center of this shift.

Composable APIs means building modular, mix-and-match components rather than monolithic solutions. In You.com's context: model flexibility (choose GPT-4, Claude, Gemini based on needs—not locked to one provider), data flexibility (combine web search, news, private RAG), and integration flexibility (REST API, Python SDK, Databricks Unity Catalog).

I've built primitive versions of this. At White Label Resell, I integrated GPT-3.5, BERT, FLAN-T5, and Stable Diffusion because different models excel at different tasks. I built routing logic to select the right model per task. But I was locked to specific providers. True composability means swapping GPT-4 for Claude with minimal code changes—that's what You.com enables.

The agentic era is the shift from passive AI assistants to autonomous agents that reason and plan (break complex goals into subtasks), use tools (call APIs, search databases, execute code), make decisions (choose actions based on context), and coordinate (work with other agents or humans).

Why You.com is critical infrastructure: Agents need real-time, verifiable information to make decisions. Example: A coding agent building a React app needs the latest React 19 docs. A legal agent needs current case law. Without accurate, real-time web intelligence, agents are limited to outdated training data.

That's why DuckDuckGo, Harvey AI, Windsurf, and Databricks use You.com APIs—their agents need reliable real-time web access. You're not building search for humans; you're building the sensory system for AI agents.

My experience: At RIT, my NLP agent for federal submissions is a simple agentic system—it breaks tasks into subtasks, uses tools, and coordinates different AI models. But it's limited by knowledge cutoff. With You.com's APIs, I could give it real-time access to federal regulations and recent successful proposals. That's the power of composable APIs for the agentic era."

18. "Where do you see AI search technology heading?"

Response (90 seconds):

"I see three major trends, all of which You.com is positioned perfectly for:

First, agentic AI will dominate. McKinsey and Bain reports show enterprises moving from experimentation to production agents. These agents need reliable, real-time web intelligence—exactly what You.com provides. Your partnerships with Databricks, Harvey AI, and Windsurf prove this is already happening. I think consumer search is just 10% of the opportunity; the 90% is B2B APIs powering agents.

Second, the end of single-model dominance. There won't be one winning LLM. We'll see specialized models for different tasks, regional models for compliance, custom fine-tuned models for domains. Model-agnostic infrastructure like You.com's becomes critical. Your 76% win rate over OpenAI's Deep Research proves that dynamic model selection beats locked-in solutions.

Third, hallucination elimination becomes table stakes. As AI moves into regulated industries like healthcare, legal, finance, hallucinations become unacceptable. RAG with proper citations isn't optional; it's required. Your 80% accuracy on FRAMES benchmark and focus on verifiable sources positions you as the trusted infrastructure layer.

The big picture: AI search won't replace Google for humans; it'll become the nervous system for AI agents. You.com is building that infrastructure. Five years from now, every AI agent will need real-time web access, and You.com's APIs will be the standard—like how AWS became the standard for web apps."

19. "What are your salary expectations?"

Response (30-45 seconds):

Option A (preferred if early in conversation): "I'm really focused on finding the right fit and working on impactful AI infrastructure at this stage of my career. I'm confident that if we're both excited about the match, we can find a compensation package that works. I'm curious—what range do you have budgeted for this role?"

Option B (if pressed): "Based on my research of AI Engineer roles at similar-stage companies and my 4+ years of experience building production AI systems at scale, I'm targeting a range of \$160,000 to \$200,000 base salary. But I'm also very interested in equity given You.com's growth trajectory—1B+ queries monthly and unicorn valuation—and the total compensation package. What range are you working with?"

20. "Do you have any questions for me?"

Select 3-4 questions based on conversation flow:

About Recent News:

1. "You just raised \$100M Series C in September at a \$1.5B valuation—congratulations! How is that funding changing the company's priorities or roadmap?"
2. "Congratulations on the 76% win rate over OpenAI's Deep Research—that's impressive. What technical breakthroughs enabled that, and how do you maintain that edge as competitors iterate?"

About Technology: 3. "I'm fascinated by your 445ms p50 latency at 1B+ queries monthly. What's the biggest technical challenge in maintaining that speed while processing 500+ sources for ARI Enterprise?"

4. "The model-agnostic architecture is fascinating. How do you decide which LLM to route a query to—is it rule-based, ML-based, or user-configurable?"

About the Role: 5. "You mentioned You.com is doubling headcount from 100 to 200 employees. How will the AI engineering team grow, and where does this role fit into that plan?"

6. "What does success look like for this role in the first 6 months? What would make you think, 'We made the right hire'?"

About Team & Culture: 7. "What does the day-to-day collaboration look like between AI engineers working on core infrastructure and Forward Deployed Engineers working with enterprise customers?"

8. "How does You.com balance the tension between building new capabilities and maintaining reliability at 1B+ queries monthly?"

About Strategy: 9. "I noticed partnerships with Databricks, DuckDuckGo, Harvey AI, and even OpenAI using your APIs. How do you prioritize enterprise customer needs vs. building platform capabilities?"

10. "As OpenAI, Anthropic, and Google build more search capabilities into their models, how does You.com maintain its competitive moat?"
-

PART 4: ADDITIONAL SMART QUESTIONS (10-15 TOTAL)

Company & Recent News:

- "Processing 1B+ queries monthly is impressive. What were the biggest infrastructure challenges in scaling from 100M to 1B queries?"
- "I saw you're expanding to SF and NYC offices. How is You.com thinking about remote-first culture as you double headcount?"

Product & Technology:

- "How does You.com's Private RAG handle the balance between integrating customer data and maintaining zero data retention?"
- "Can you walk me through a recent project the AI team shipped? What did the development lifecycle look like?"

Team Structure:

- "What does the AI engineering team structure look like? How many engineers work on core LLM infrastructure vs. search vs. enterprise features?"
- "How does You.com approach professional development? Are there opportunities to publish research or attend conferences?"

Competitive Landscape:

- "I noticed Bing API is sunseting and Google is restricting API access. Is You.com seeing increased demand from developers looking to migrate?"
- "The pivot from consumer search to enterprise AI infrastructure seems strategic. How does the consumer product fit into the broader vision now?"

Culture:

- "I saw on Glassdoor that employees mention You.com values self-starters and autonomy. Can you give an example of how that plays out day-to-day?"
 - "With Richard Socher and Bryan McCann's research backgrounds, how does You.com balance cutting-edge AI research with commercial product development?"
-

PART 5: KEY TALKING POINTS & HERO STORIES

Core Positioning Statement:

"I'm an AI Infrastructure Engineer who specializes in building production AI systems that scale reliably. I've deployed systems handling everything from 800 federal research proposals to 50,000+ articles monthly to 1.5TB of medical images weekly. I excel at the intersection of backend engineering and AI—building the infrastructure that makes AI work in the real world, not just demos. That's exactly what You.com does at scale with 1B+ queries monthly."

Key Talking Points:

1. **Scale Experience:** "I've built systems processing 800+ proposals, 50,000+ articles monthly, 1.5TB weekly—your 1B+ queries monthly excites me"
2. **AI Deployment Track Record:** "I've shipped multiple production AI systems with proven accuracy: 99.2% at RIT, 94% at Daiichi Sankyo, 92% F1-score at Digital Clinics"
3. **Backend + AI Combination:** "I don't just build models—I build full infrastructure: AWS Lambda/ECS, microservices, distributed systems, monitoring"
4. **Autonomous Work Style:** "Founding engineer at SEO Content AI and White Label Resell—I built entire platforms from scratch"
5. **LLM Infrastructure:** "I've worked with GPT-3.5, BERT, FLAN-T5, Mistral 7B, Gemini—I understand why model-agnostic architectures matter"

Five "Hero Stories":

Hero Story #1: Infrastructure Scaling (SEO Content AI)

Use for: Scale, backend systems, AWS, autonomous work questions

"At SEO Content AI, I was the founding AI Infrastructure Engineer. I built the entire infrastructure from scratch—AWS Lambda for inference, ECS for orchestration, DynamoDB, S3. We launched and hit 5,000 users in the first month. The initial architecture couldn't handle it. I architected the scaling solution: Redis caching (80% hit rate), optimized model selection, spot instances, smart batching. Result: Scaled to 50,000+ articles monthly, reduced costs by 40%, maintained sub-5-second generation times. Secured \$50K+ in contracts."

Why powerful: Shows you can build from scratch, scale quickly, optimize costs, deliver business value.

Hero Story #2: AI with Production Constraints (Daiichi Sankyo)

Use for: Shipping AI products, handling constraints, problem-solving

"At Daiichi Sankyo, I built a document classification system that had to run on-premise on CPU-only machines due to HIPAA compliance—no cloud APIs, no GPUs. The base PyTorch model took 8 seconds per document—too slow. I optimized with ONNX conversion, model quantization (400MB→100MB, no accuracy loss), smart chunking, and caching. Result: Reduced inference from 8 seconds to 1.2 seconds while achieving 94% classification accuracy. Processed 1,000+ documents daily."

Why powerful: Demonstrates understanding of production constraints (compliance, latency, cost), optimization skills, and reliability.

Hero Story #3: Autonomous Problem-Solving (RIT)

Use for: Autonomy, working independently, taking ownership

"At RIT, the challenge was automating federal research submissions—800+ proposals annually with zero tolerance for errors. I had complete ownership. I designed an AI-powered NLP agent that extracts requirements from RFPs, gathers data, generates compliant text, and validates outputs. I made all technical decisions independently—which models, how to architect for reliability, how to handle edge cases. I also built the ML workload management system routing tasks across GPU clusters. Result: Processing 800+ proposals with 99.2% accuracy, saving hundreds of hours."

Why powerful: Proves you can take ownership, make good decisions independently, deliver production systems.

Hero Story #4: System Architecture for Scale (Digital Clinics)

Use for: System architecture, distributed systems, scale

"At Digital Clinics, we built an AI for breast cancer diagnosis—life-or-death accuracy. The scale challenge was processing 1.5TB of medical images weekly. I architected a distributed training system using Horovod for data parallelism across 8 GPUs, efficient data loading from S3 with prefetching, and checkpointing for fault tolerance. The bigger challenge was accuracy—false negatives were catastrophic. I designed the system to optimize F1-score specifically with careful dataset balancing. Result: Reduced training from 3 weeks to 4 days, achieved 92% F1-score, published research papers."

Why powerful: Shows you can architect for both scale (1.5TB weekly, distributed systems) and accuracy—exactly You.com's balance.

Hero Story #5: Multi-Model Integration (White Label Resell)

Use for: LLM experience, model selection, composable systems

"At White Label Resell as founding ML Engineer, I built a content automation platform generating 10,000+ articles monthly in 50+ languages. The challenge was no single AI model was optimal for all tasks. I integrated GPT-3.5 for general text, BERT for classification, FLAN-T5 for instruction-following, and Stable Diffusion for images. I built routing logic that selected the right model based on task type—like a primitive version of You.com's model-agnostic routing. I also built full AWS microservices architecture where each component could scale independently. Result: 10,000+ articles monthly in 50+ languages, secured \$50K+ in contracts."

Why powerful: Directly parallels You.com's approach—multiple models for different tasks, composable architecture, proven at scale.

Quick Sound Bites:

- "I've deployed systems processing 800+ proposals, 50,000+ articles monthly, and 1.5TB weekly—I understand scale"
 - "I don't just build models; I build the infrastructure that makes AI work in production"
 - "I've integrated GPT-3.5, BERT, FLAN-T5, Mistral, Gemini—I understand why model-agnostic architectures matter"
 - "Your 1B+ queries monthly excites me—that's the scale where architecture decisions really matter"
 - "I've been following your ARI Enterprise launch—76% win rate over OpenAI proved you're building infrastructure, not just another search engine"
 - "As a founding engineer twice, I've proven I can take ownership and deliver production systems independently"
 - "I've optimized for accuracy (99.2%), latency (1.2s), and cost (40% reduction) at scale"
-

FINAL 20-MINUTE CALL STRATEGY

Time Allocation:

- **Minutes 0-3:** Intro, build rapport
- **Minutes 3-8:** Your background (tell me about yourself, why You.com)
- **Minutes 8-14:** Their questions (be concise, 1-2 min answers)
- **Minutes 14-18:** Your questions (3-4 strong questions)
- **Minutes 18-20:** Closing, next steps

Key Mindset:

✅ This is a **conversation**, not an interrogation ✅ Show **enthusiasm** for You.com specifically, not just "any AI job" ✅ Be **concise**—HR sees 20+ candidates, be memorable but efficient ✅ Demonstrate **research depth**—recent news, products, customers ✅ **Technical but accessible**—show expertise without overwhelming ✅ **Save deep technical details** for later rounds

Pre-Call Checklist (30 Minutes Before):

- [] Review You.com's recent news (Series C, ARI Enterprise, 1B+ queries)
- [] Review your resume and be ready to explain any project
- [] Practice "tell me about yourself" out loud (90 seconds)
- [] Have 5 questions written down
- [] Have your 5 hero stories ready with 1-minute versions
- [] Phone charged, quiet space, water nearby

During Call - Dos:

✅ Smile (they can hear it!) ✅ Reference specific You.com achievements (76% win rate, 1B+ queries, Series C) ✅ Connect your experience directly to their needs ✅ Show you understand their market position (B2B infrastructure, not consumer search) ✅ Express genuine excitement about the technology and mission ✅ Ask thoughtful questions showing research depth

During Call - Don'ts:

- ❌ Talk for more than 2 minutes on any answer
- ❌ Use excessive jargon with non-technical recruiter
- ❌ Badmouth previous employers
- ❌ Say "I don't know" without trying to answer
- ❌ Fail to ask questions
- ❌ Seem desperate or disinterested

Strong Closing:

"Thank you so much for this conversation, Justin. I'm even more excited about this opportunity than I was before. You.com is building exactly the infrastructure I want to work on—the APIs and systems powering the next generation of AI applications. My experience building production AI systems at scale—from the 800 federal proposals at RIT to 50,000+ articles monthly at SEO Content AI—has prepared me for the challenges You.com is tackling at 1B+ queries monthly. I'd love to move forward in the process. What are the next steps, and when might I expect to hear back?"

KEY SUCCESS FACTORS

What Separates Good from Great Candidates:

1. **Preparation is Obvious:** Deep company research beyond "About Us" page
2. **Communication is Clear:** Explain complex AI simply without condescension
3. **Enthusiasm is Genuine:** Passion for AI and this specific role comes through
4. **Examples are Concrete:** Specific projects and metrics at fingertips
5. **Questions are Thoughtful:** Informed questions showing research and strategic thinking
6. **Cultural Awareness:** Understand what company values and can speak to alignment
7. **Professional Maturity:** Handle difficult questions (salary, why leaving) with grace
8. **Technical Credibility:** Demonstrate AI expertise appropriate for non-technical audience

Remember:

- This is **mutual evaluation**—you're interviewing them too
- **Confidence without arrogance**—show you're good but eager to learn
- **Be memorable**—they screen many candidates; give them a reason to remember you
- **Trust the process**—if it's not the right fit, that's okay

Final Mindset:

The HR screening isn't about proving you're the smartest AI engineer in the world. It's about demonstrating you're:

- ☒ Qualified for the role (you are, or you wouldn't get the call)
- ☒ Excited about this specific opportunity
- ☒ Able to communicate effectively
- ☒ A good cultural fit
- ☒ Worth the team's time for deeper technical interviews

Approach it as a conversation, not an interrogation. Be yourself, be prepared, and let your genuine interest shine through.

YOU'RE READY! 🚀

You have:

-  Deep company intelligence (funding, products, customers, culture)
-  Clear understanding of role requirements
-  20 battle-tested Q&A responses tailored to your background
-  5 hero stories that perfectly align with You.com's needs
-  10-15 smart questions to ask
-  Clear strategy for the 20-minute call

Go into this call confident. You have the experience, the skills, and the preparation. Show them why you're the AI Infrastructure Engineer they need to help scale from 1B to 10B queries monthly. Good luck!

Techniques I'd master to fine-tune LLMs in production. Bookmark this

1. LoRA & QLoRA for parameter-efficient fine-tuning
2. PEFT library for adapter methods
3. Instruction tuning
4. Dataset formatting (ChatML, Alpaca, ShareGPT)
5. DeepSpeed ZeRO for memory optimization
6. Flash Attention 2 for efficient training
7. Gradient checkpointing for longer contexts
8. BitsAndBytes for 4-bit/8-bit quantization
9. RLHF & DPO for alignment
10. Tokenizer training & vocabulary extension
11. Evaluation metrics (perplexity, ROUGE, human eval)
12. Unsloth for 2x faster fine-tuning
13. Multi-GPU strategies (FSDP, DDP)

1. RAG
2. LoRA
3. DPO
4. Mixture of Experts (MoE)
5. Quantization
6. Function Calling
7. Mamba
8. QLoRA
9. vLLM
10. GQA

Techniques I'd master to build great evals for AI apps.

1. LLM-as-Judge
2. Reference-based similarity metrics
3. Pairwise comparison tournaments
4. Human-in-the-loop evaluation
5. Synthetic data generation
6. Adversarial test case creation
7. Multi-dimensional rubrics
8. Regression testing on golden datasets
9. A/B testing with live traffic
10. Statistical significance testing
11. Evaluation dataset curation & versioning
12. Domain-specific benchmarks
13. Red teaming & jailbreak testing
14. Latency & cost monitoring
15. User feedback loops
16. Calibration & confidence scoring

Bookmark this.

1. Bias-Variance Tradeoff
2. Cross-Validation Strategies
3. Regularization (L1, L2, Elastic Net)
4. Class Imbalance & Sampling Techniques
5. Feature Engineering & Selection
6. Overfitting vs Underfitting
7. Evaluation Metrics (beyond accuracy)

8. Hyperparameter Tuning
9. Train-Test Data Leakage
10. Ensemble Methods
11. Dimensionality Reduction
12. Model Interpretability (SHAP, LIME)
13. Gradient Descent Variants
14. Activation Functions & Neural Networks
15. Imbalanced Dataset Handling
16. Production Model Monitoring

Quantization

KV-Cache Quantization

Flash Attention

Speculative Decoding

LoRA

Pruning

Knowledge Distillation

Weight Sharing

Sparse Attention

Batching & Dynamic Batching

Model Serving Optimization

Tensor Parallelism

Pipeline Parallelism

Paged Attention

Mixed Precision Inference

Early Exit / Token-Level Pruning

And RAG All types of this

and Vector Databases related to agentic, chat bast, research agents etc

and embedding models simialrly for this

I cna spend 5-6 hours on this I need to learn it form perpsective of System design and talking about it to my interviewer

FedMob Final Demo - Complete Speaker Notes

SLIDE 1: Title Slide (30 seconds)

Good afternoon everyone. I'm Faseeh Ahmed Mohammad, and today I'm presenting FedMob - a privacy-preserving federated learning system for mobile devices. This project has been completed under the guidance of Professor Haibo Yang, and represents the culmination of my capstone work here at RIT.

SLIDE 2: Project Overview (1-2 minutes)

FedMob solves the challenge of collaborative machine learning while maintaining complete data privacy. The core idea is simple but powerful: instead of sending sensitive data to centralized servers, we train models locally on each device and only share the model improvements. This means raw data never leaves the device.

We built this as a React Native mobile application that enables true multi-device federated learning on real mobile hardware. Our goals were threefold: achieve multi-device coordination on actual phones, create a production-ready system using industry-standard frameworks, and ensure the system remains efficient, scalable, and privacy-preserving throughout.

SLIDE 3: Domain Model (2 minutes)

Our domain model clearly separates server coordination from client training, ensuring both privacy and scalability. On the server side, we have the FlowerServer which manages federated learning rounds, coordinates clients, and aggregates model updates. It maintains the GlobalModel that gets distributed to clients.

On the client side - which is the mobile app - we have the FlowerClient implementing the Flower interface, and the LocalModel which is the TensorFlow.js model that trains on local data. Communication between these components happens through a WebSocket Bridge that handles the React Native to Flower communication, while gRPC manages the Flower server protocol. This separation of concerns is critical for maintaining system stability and privacy guarantees.

SLIDE 4: System Architecture (2 minutes)

Our three-layer architecture bridges React Native with the Flower framework through a WebSocket server that handles protocol translation and model conversion.

At the bottom layer, we have the Mobile Client running React Native with TensorFlow.js for on-device training. We support two model architectures - one with 225,000 parameters and another with 108,000 parameters.

The middle layer is our WebSocket Bridge Server running on port 8082. This is critical - it handles model conversion between TensorFlow.js and Python formats, and coordinates client connections.

At the top, the Flower Server uses gRPC communication and implements FedAvg aggregation for managing multiple clients simultaneously. This architecture was essential for making React Native work with Flower's Python-based framework.

SLIDE 5: Architecture Updates - Evolution (2 minutes)

We evolved significantly from our initial planning. Back in DSCI-601, we had a Flask server using FedDisco, which was unstable and crashed frequently. Users had to manually upload images from their gallery, which wasn't practical.

Our current production system uses the Flower framework, which is industry-standard and has given us zero crashes for over six weeks. We implemented the two-server architecture you just saw for protocol bridging, switched to synthetic MNIST data generation for consistency, and now support multiple model architectures including LeNet-5.

The improvements are substantial: complete stability with zero crashes, 100% accurate weight conversion between formats, and multi-device coordination that actually works reliably. This evolution was driven by real-world testing and the need for production-grade reliability.

SLIDE 6: Deliverables Summary (2 minutes)

All four deliverables are complete, and we have a working, stable federated learning system running on real mobile devices.

Deliverable 1 was Flower Client Integration - our React Native WebSocket bridge is fully operational with working client registration and authentication.

Deliverable 2, the Flower Server Implementation, replaced Flask with the Flower framework. FedAvg aggregation with multi-client coordination has been stable since October 5th.

Deliverable 3 covered the Model Training Pipeline. We have two architectures working - Basic CNN with 225K parameters and LeNet-5 with 108K parameters. On-device TensorFlow.js training is functioning, and multi-client training is successful.

Finally, Deliverable 4 was Performance Optimization. We've achieved 95% training round completion through an adaptive timeout system, and we can coordinate three or more devices simultaneously. The system is production-ready.

SLIDE 7-8: Live Demo (5-7 minutes)

I'll now demonstrate the complete federated learning workflow with three actual devices training collaboratively.

[Setup explanation] I have three devices ready - two mobile phones and one laptop - all running the LeNet-5 model with 108,000 parameters. Each device will train on synthetic MNIST digit data.

[Connection phase] First, you'll see each device connect and register with the server. Watch as they establish WebSocket connections...

[Training initiation] Now I'm starting a federated learning round from the server. Each device will receive the global model and begin local training...

[Progress monitoring] You can see the training progress in real-time - watch the progress bars move from 0% to 100% as each device completes its local training epochs...

[Aggregation] As devices complete training, they send their weight updates back to the server. The server aggregates these updates using FedAvg to create an improved global model...

[Round completion] And there we have it - a complete federated learning round with three devices training collaboratively while keeping all their data private. The entire process took about 3 minutes.

SLIDE 9: Experiment Results (2 minutes)

Our experiments show significant performance improvements through optimization, achieving a 95% success rate in multi-device coordination.

Comparing our two models: LeNet-5 is 40% faster than the Basic CNN, taking only 1.5 minutes versus 2.5 minutes for training. It also uses 52% fewer parameters - just 108,000 compared to 225,000 - while maintaining similar accuracy. Memory usage is slightly better too, ranging from 1.2 to 1.4 megabytes versus 1.43 to 1.52 megabytes. Both achieve 100% accurate weight conversion, which is critical.

Looking at multi-device stability improvements, we increased round completion from 60% to 95%. Timeout issues dropped from frequent to rare - only about 5% of rounds experience timeouts now. Average round time decreased from 4-5 minutes to 3 minutes. The adaptive timeout system was key to these improvements, and 100% accurate weight conversion is absolutely critical for federated learning to work correctly.

SLIDE 10: Technical Challenges Overcome (2 minutes)

We solved several critical technical challenges to make this system work.

First, the React Native to Python bridge was complex. TensorFlow.js and Python use different weight formats. We built a ModelConverter using base64 encoding that achieves 100% accurate conversion - anything less would break the federated learning process.

Second, device heterogeneity was a major issue. Our laptop trains 3-4 times faster than phones, which would cause slower devices to time out. We implemented an adaptive timeout system that adjusts based on device performance, bringing round completion up to 95%.

Third, server stability was initially poor. The solution was migrating from Flask to Flower, which gave us zero crashes for over six weeks - a complete transformation in reliability.

Finally, data consistency was challenging for reproducible experiments. We solved this with synthetic MNIST generation, ensuring every experiment uses consistent data distributions.

SLIDE 11: System Achievements & Capabilities (1-2 minutes)

FedMob is production-ready with high reliability, efficient resource usage, and complete privacy preservation.

Our core achievements include that 95% training round success rate, coordination of three or more devices simultaneously, true privacy preservation since data never leaves devices, zero server crashes for over six weeks, 100% weight conversion accuracy, and a sub-2MB memory footprint.

For production features, we have a model library with persistent storage so trained models are saved, real-time progress monitoring so users can watch training happen, offline inference capability to use models without connectivity, and cross-platform support for both Android and iOS. This is a complete, production-ready system.

SLIDE 12: Future Extensions (1 minute)

The core system is complete and stable, with multi-device federated learning using the Flower framework ready for production deployment. All core functionality is working.

For potential future enhancements - and these are beyond the current scope - we could add support for larger models using optimization techniques like quantization or pruning, integrate with extended federated learning libraries for specialized algorithms, implement advanced aggregation strategies beyond FedAvg, and scale to production deployments with tens or hundreds of devices. But again, the current system accomplishes everything we set out to build.

SLIDE 13: Conclusion & Questions (1 minute)

FedMob has accomplished its mission. We've achieved all our project goals: a production-ready mobile federated learning system with 95% success in multi-device coordination, privacy-preserving architecture, and industry-standard integration with Flower.

Our key technical contributions include the two-server architecture for protocol bridging between React Native and Flower, the adaptive timeout system for handling device heterogeneity, and validation that TensorFlow.js is viable for mobile federated learning.

The bottom line is this: FedMob proves that federated learning works on real mobile devices while maintaining complete data privacy. Thank you, and I'm happy to take any questions.

Tab 77

